

A.A. 2026-2027

IngSw

Indice

0 Note sul corso 3

I Introduzione e Requisiti 4

1 Lezione 2-3 Software Processes and Software Quality 5
- La disciplina in una definizione 5
- Il modello a cascata 5
  - Limiti del modello 5
- Le fasi del modello a cascata 5
  - Requirements Engineering 5
  - System e software/UI design 5
  - Implementation 5
  - Verification e Validation 6
  - Operation and Maintenance 6
- La qualità del software 6
- Qualità del prodotto (ISO 25002) 6
  - Functional Suitability 6
  - Reliability 6
  - Performance Efficiency 7
  - Usability 7
  - Security 7
  - Compatibility 7
  - Maintainability 7
  - Flexibility 7
  - Safety 8
- Qualità nell'uso (Quality-in-use Model) 8
  - Usability (quality-in-use) 8
  - Safety (quality-in-use) 8
  - Flexibility (quality-in-use) 8

2 Lezione 4-5 Requirements Engineering, Elicitation e Analysis 9
- Il posto della RE nel ciclo di vita 9
- I requisiti software 9
- Livelli di requisiti 9
  - Ruolo contrattuale 9
- Requisiti funzionali e non funzionali 10
  - Criticità dei requisiti non funzionali 10
  - Tipi di requisiti non funzionali 10
  - Requisiti di dominio 11
- Proprietà dei buoni requisiti 11
  - I demoni dell'ambiguità 11
  - Requisiti e testabilità 11
  - Metriche per requisiti non funzionali 11
- Il processo di Requirements Engineering 12
- Elicitation e analisi dei requisiti 12
  - Sfide dell'elicitation 12
  - Il processo di elicitation e analisi 12
- Tecniche di elicitation 13
  - Interviste con gli stakeholder 13
  - Etnografia 13
  - Personas 13
  - Storie 14
  - Mockup a bassa fedeltà 14

3 Lezione 6-7 Requirements Engineering & Specification e Validation 15
- Specificazione dei requisiti 15
  - Requisiti e design 15
- Approcci di specificazione 15
- Linee guida per la specifica in linguaggio naturale 15
  - Problemi del linguaggio naturale 15
- Specificazioni strutturate 16
- Use case diagram 16
  - Attori 16
  - Use case ed ellissi 17
  - Associazioni 17
  - Attori secondari 17
  - Generalizzazione di attori 17
  - Errori comuni nei use case diagram 18
- Formati dei use case 18
  - Descrizioni testuali di uno use case 18
- Il template di Cockburn 18
  - Scenario principale ed estensioni 19
- Validazione dei requisiti 20
  - Controlli sui requisiti 20
  - Tecniche di validazione 20
  - Controlli in review 20

4 Lezione 8 UML Statecharts 21
- UML Statecharts 21
- Stati e transizioni 21
- Sintassi 21
  - Regioni, vertici e transizioni 21
  - Stati semplici 21
  - Pseudostati iniziali e stati finali 21
  - Sintassi delle transizioni 21
- Semantica 22
- Attività interne degli stati 22
- Stati compositi 23
  - Regioni parallele 23
  - Pseudostati di history 23
  - Pseudostati di fork e join 23
- Consigli di modellazione 23
- Model-driven Development 24
- Statecharts per la gestione delle UI 24

II Architettura di Sistema e Cloud 25

5 Lezione 10-11 System Design e Design Practices 26

La progettazione di un sistema 26
Scopo di System Design e Object Design 26
Il cambio di prospettiva 26
I passi del system design 26
Gli obiettivi qualitativi del sistema 27
Criteri di design 27
Design trade-offs 27
Concetti di base di buon design 28
La decomposizione 28
Il vantaggio della decomposizione 28
Sottosistemi, servizi e interfacce 28
Le responsabilità contano più dei dati 28
Indipendenza funzionale 28
La coesione 29
Il Single Responsibility Principle 29
Indicatori di alta coesione 29
L'accoppiamento 29
Tipi di accoppiamento in O-O 30
Il ruolo delle interfacce 30
La legge di Demetra 30
Eliminare il navigation code 31
Il responsibility-driven design 31
Le CRC cards 31
Localizzare le modifiche 31

6 Lezione 12-13 Standard Software Architectures 32

Architettura del software 32
Scopo della progettazione architetturale 32
Divide and conquer 32
L'architettura a livelli 33
La macchina virtuale 33
La layered architecture 33
L'architettura client-server 34
L'architettura a repository 34
Vantaggi del repository 34
Svantaggi del repository 34
Le architetture three-tier 34
Vantaggi delle three-tier 35
Svantaggi delle three-tier 35
Le architetture n-tier 35
La Service-oriented Architecture 36
Gli stili architetturali 36

7 Lezione 14 Virtualizzazione e Cloud 37

La virtualizzazione 37
L'evoluzione delle CPU 37
Le CPU multicore 37
L'infrastruttura tradizionale di un CED 37
Il fail di un server 37
Le macchine virtuali 38
L'hypervisor 38
Le proprietà delle macchine virtuali 38
La virtualizzazione enterprise ad alta disponibilità 38
I benefici della virtualizzazione 39
Il cloud computing 39
Scalabilità ed elasticità 39
Il dynamic provisioning 39
I service model 40
I deployment model 40
Public vs. private 40

8 Lezione 15-16 Modern Software Architectures 42

Il ciclo di vita di sviluppo del software 42
Architettura del sistema software 42
Perché l'architettura è importante 42
Proprietà influenzate dall'architettura 43
Scalabilità 43
Orizzontale, verticale e diagonale 43
Load balancing 44
Disponibilità 44
Quantificazione 44
Fail-over 44
Osservabilità e monitoraggio 45
Dati e consistenza 45
Proprietà ACID 45
Scalare un RDBMS 46
Database NoSQL 47
Elaborazione asincrona 48
Necessità dell'asincronismo 48
Message queue 48
Message broker 48
Code e stream 49
Microservizi 49
Architettura monolitica 49
Architettura a microservizi 49
Motivazioni 49
Scalare monoliti e microservizi 50
Quando adottare i microservizi 50
Progettazione con il Domain-Driven Design 50
Comunicazione tra servizi 51
Gestione dei dati 51
Trade-off 52
Architettura e organizzazione del team 52
Legge di Conway 52
Team a livelli e team cross-funzionali 52
Gestire la legge di Conway 52
Messaggio finale 52

9 Lezione 16.2 Containerization for Software Engineers 53

Tendenze del settore software 53
La complessità dei deployment 53
La «Matrix from Hell» 53
L'analogia del trasporto merci 53
Container per il codice 53
Container software 54

Macchine virtuali vs container 54
Docker 54
Docker come container engine 54
Architettura di Docker 54
Immagini Docker 55
Dockerfile 55
Comandi Docker essenziali 55
Immagini 55
Ciclo di vita di un container 55
Detach e trasferimento file 56
Applicazioni container-native 56
Architettura dell'applicazione 56
Fortune service 56
Uppercase service 56
Vista completa dell'applicazione 57
Diagramma di sequenza 57
Immagini Docker dei servizi 58
Orchestrazione con docker-compose 58

III Software and Design Pattern 60

10 Lezione 17-18 Design Patterns 61
Software Development Lifecycle 61
Riuso di codice e riuso di design 61
Che cos'è un design pattern 61
Classificazione dei pattern 62
Data Access Object 62
Pattern creazionali 63
Singleton 63
Builder 63
Factory Method 64
Abstract Factory 64
Pattern strutturali 65
Adapter 65
Composite 66
Decorator 67
Bridge 68
Facade 68
Pattern comportamentali 69
Observer 69
Strategy 70
State 70
Iterator 71
Dependency Injection 72
Dependency Injection e testing 72
Riuso, ereditarietà e delega 72
I due scopi dell'ereditarietà 72
Eredità di implementazione e di interfaccia 72
Delega 73
Osservazioni e conclusioni 73

11 Lezione 19 Refactoring 75
Definizione e motivazioni 75
Rinominare variabili, metodi e classi 75
Il catalogo di Fowler 75
Code smells 75
Tipologie di bad smells 76
Bloaters 76
Object-Orientation Abusers 76
Change Preventers 76
Dispensables 76
Couplers 76
Debito tecnico 76
Definizione 76
Analogia del debito 76
Debito strategico e intenzionale 77
Debito non strategico e non intenzionale 77
Conseguenze del debito 77
Misurazione 77
Il processo di refactoring 77
Metafora di base 77
Il ciclo di refactoring 77
Refactoring e unit test 77
Perché refactorare 78
Refactoring tipici 78
Extract Method 78
Push method down 78
Push method up 79
Add parameter to method 80
Replace Parameter with Method 80
Bad smells e rimedi 81
Duplicated Code 81
Long Method 81
Large Class 81
Long Parameter List 81
Divergent Change 81
Shotgun Surgery 81
Feature Envy 82
Data Clumps 82
Primitive Obsession 82
Switch Statements 82
Parallel Inheritance Hierarchies 82
Lazy Class 82
Speculative Generality 82
Temporary Field 83
Message Chains 83
Middle Man 83
Osservazioni finali 83
Refactoring vs restructuring 83

IV Progettazione Orientata agli Oggetti e Clean Code 84

V Verification, Validation & Testing 85

VI Processi di Sviluppo e Interfacce 86

Capitolo 0

Note sul corso

I seguenti appounti sono stati presi durante le lezioni del corso di Tecnologie Web.

Nota: Lezioni assenti nel documento Le lezioni 0 e 1 ha fatto solo introduzione, non era necessario includerle nel documento. La lezione 9 è stata saltata appositamente, dato che non è stato spiegato niente ma è stata fatta una esercitazione.

Parte I

Introduzione e Requisiti

Lezione 2-3 Software Processes and Software Quality

La disciplina in una definizione

> *«Software Engineering is the application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software»*
>
> – IEEE Standard Glossary

La disciplina è anche descritta come state of the art dello sviluppare software di qualità in tempo e entro budget (Bruegge e Dutoit).

Il modello a cascata

Il modello a cascata deriva dai modelli di processo ingegneristici usati nell'ingegneria di grandi sistemi militari.

- Il processo consiste in fasi **sequenziali**, in un processo **plan-driven** (guidato dal piano).
- Il risultato di ogni fase è un documento approvato (**signed-off**).
- La fase successiva non può iniziare finché il risultato della fase precedente non è completo.

Limiti del modello

L'approccio rigido e **feed-forward** ha senso per l'ingegneria hardware, dove i costi di produzione sono alti; per lo sviluppo software è troppo rigido.

- Le fasi possono sovrapporsi e alimentarsi di informazione: durante il system design possono emergere problemi nei requisiti, durante l'implementazione problemi nel design.
- I requisiti possono cambiare durante lo sviluppo.

Le fasi del modello a cascata

Requirements Engineering

L'obiettivo è capire cosa il software da sviluppare deve fare, non come implementarlo.

- Richiede un'analisi accurata dei bisogni dell'utente e del dominio del problema.
- Coinvolgono clienti, utenti finali e ingegneri del software.
- L'output principale è un **documento di specifica dei requisiti** (Software Requirements Specification).

System e software/UI design

L'obiettivo è progettare una struttura (architettura) adeguata per il software, su due livelli diversi.

- **System design**: architettura complessiva del sistema; scomposizione in moduli e componenti; allocazione delle funzionalità ai moduli e dei moduli ai componenti hardware; definizione di relazioni e collaborazioni tra moduli.
- **Software e UI design**: dettagli di come implementare ogni modulo; architettura software di basso livello (ad es. quali classi serviranno); prototipazione della interfaccia utente.

L'output è un insieme di specifiche di design, spesso formalizzate con linguaggi di design come UML.

Implementation

L'obiettivo è «tradurre» le specifiche di design in un linguaggio o tecnologia scelti: non una traduzione qualunque, ma di alta qualità. Il codice risultante deve essere:

- **leggibile** e **manutenibile**;

- **riusabile** ed **estendibile**;
- **testabile**;
- in una sola parola, **clean**.

Verification e Validation

Dopo l'implementazione occorre chiedersi se soddisfa i bisogni degli utenti: qui entrano verifica e validazione.

Definition

Verifica

Controllo che il sistema sia conforme alla propria specifica: "have we built the thing right?". Tipicamente fatta con il testing del programma: esecuzione del software in ambiente controllato e controllo che il comportamento sia corretto rispetto alle specifiche.

Accanto alla conformità alla specifica si controlla la conformità ai bisogni del cliente.

Definition

Validazione

Controllo che il sistema soddisfi le aspettative dei clienti: "have we built the right thing?". Tipicamente fatta definendo ed eseguendo acceptance test.

Operation and Maintenance

- **Operation**: il sistema viene distribuito e installato per i clienti e messo in uso effettivo.
- **Maintenance**: il software cambierà a un certo punto: i bisogni del cliente possono cambiare, il contesto d'uso può cambiare (es. una nuova legge che richiede procedure diverse) ed emergono bug sfuggiti alla fase di V&V.

La qualità del software

La qualità del prodotto è un concetto complesso e non univoco: giudizi diversi (efficienza, sicurezza, stabilità, usabilità) possono dare valutazioni diverse dello stesso prodotto. Lo standard **ISO/IEC 25002** definisce i modelli di qualità del software.

- **Product Quality Model**: 9 caratteristiche, ulteriormente suddivise in sotto-caratteristiche, relative alle proprietà di qualità dei prodotti; forniscono un modello di riferimento per specificare, misurare e valutare la qualità dei prodotti.
- **Quality-in-use Model**: 3 caratteristiche, ulteriormente suddivise in sotto-caratteristiche, che influenzano gli stakeholder quando i prodotti sono usati in un contesto d'uso specificato.

Qualità del prodotto (ISO 25002)

Functional Suitability

Grado in cui un componente o sistema fornisce funzioni che soddisfano bisogni espliciti e impliciti quando usato in condizioni specificate.

- **Functional completeness**: grado di copertura di tutti i task e obiettivi specificati.
- **Functional correctness**: grado di risultati accurati con gli utenti previsti.
- **Functional appropriateness**: grado in cui le funzioni facilitano il compimento di task e obiettivi specificati.

Reliability

Grado in cui il sistema esegue le sue funzioni in condizioni specificate per un periodo di tempo specificato.

- **Faultlessness**: grado di esecuzione senza guasti in operazione normale.
- **Availability**: grado di essere operativo e accessibile quando richiesto.
- **Fault tolerance**: grado di operare come previsto nonostante guasti hardware o software.
- **Recoverability**: grado di ripristino dopo un'interruzione o un guasto.

Performance Efficiency

Grado di esecuzione delle funzioni entro vincoli di risorse specificati, usando le risorse (CPU, memoria, storage, energia) in modo efficiente.

- **Time behaviour**: grado di soddisfare requisiti su tempi di risposta e throughput.
- **Resource utilization**: grado di soddisfare requisiti su quantità e tipi di risorse usate.
- **Capacity**: grado di soddisfare requisiti sui limiti massimi dei parametri.

Usability

Grado con cui un prodotto può essere interagito dai suoi utenti. L'usabilità può fare la differenza tra successo e fallimento di un prodotto, e persino tra vita e morte.

- **Recognizability**: grado in cui gli utenti riconoscono se il sistema è adatto ai loro bisogni.
- **Learnability**: grado in cui le funzioni possono essere imparate da utenti specificati entro un tempo specificato.
- **Operability**: grado in cui il sistema è facile da operare e controllare.
- **User error protection**: grado in cui il sistema previene gli errori.
- **Inclusivity**: grado in cui il sistema è usabile da persone di background diversi (età, abilità, culture, lingue, generi, condizioni economiche).

Security

Grado di difesa da attacchi di attori malevoli e di protezione di informazioni e dati con meccanismi di autorizzazione adeguati.

- **Confidentiality**: grado di rendere i dati accessibili solo a chi è autorizzato.
- **Integrity**: grado di proteggere stato e dati da modifiche o cancellazioni non autorizzate.
- **Non-repudiation**: grado di provare che azioni o eventi hanno avuto luogo, così che non possano essere ripudiate.
- **Accountability**: grado di tracciare univocamente le azioni di un'entità a quell'entità.
- **Authenticity**: grado di provare che l'identità di un soggetto o risorsa è quella dichiarata.

Compatibility

Grado in cui un sistema può scambiare informazioni con altri prodotti o componenti e/o eseguire le sue funzioni condividendo lo stesso ambiente e le stesse risorse di altri sistemi.

- **Co-existence**: grado di eseguire le funzioni richieste condividendo ambiente e risorse con altri prodotti senza impatti dannosi.
- **Interoperability**: grado di scambiare informazioni con altri prodotti e usare reciprocamente le informazioni scambiate.

Maintainability

Grado di efficacia ed efficienza con cui un prodotto può essere modificato per migliorarlo, correggerlo o adattarlo a cambiamenti di ambiente e requisiti.

- **Modularity**: grado di composizione in componenti discreti, così che una modifica a un componente abbia impatto minimo sugli altri.
- **Reusability**: grado con cui un software o modulo può essere usato come asset in più di un sistema.
- **Modifiability**: grado di modifica efficace ed efficiente senza introdurre difetti o degradare la qualità.
- **Testability**: grado con cui si possono stabilire criteri di test ed eseguire test per verificarli.

Flexibility

Grado con cui un prodotto può essere adattato a cambiamenti nei requisiti, nei contesti d'uso o nell'ambiente operativo.

- **Adaptability**: grado di adattamento o trasferimento efficace ed efficiente a diversi ambienti hardware, software o operativi.
- **Scalability**: grado di gestire carichi crescenti o decrescenti o adattare la capacità alla variabilità.
- **Installability**: grado di efficacia ed efficienza di installazione e/o disinstallazione.
- **Replaceability**: grado con cui un prodotto può sostituire un altro prodotto specificato per lo stesso scopo nello stesso ambiente.

Safety

Grado con cui un prodotto evita uno stato in cui vita, salute, proprietà o ambiente sono messi in pericolo.

- **Fail safe**: grado di porsi automaticamente in modalità operativa sicura o di tornare a una condizione sicura in caso di guasto.
- **Risk identification**: grado di identificare corsi di eventi o operazioni che portano a rischi inaccettabili.
- **Hazard warning**: grado di fornire avvisi di rischi inaccettabili in tempo utile perché si possa reagire.

Qualità nell'uso (Quality-in-use Model)

La qualità nell'uso misura il grado con cui un prodotto può essere usato da utenti specifici per raggiungere obiettivi specifici con efficacia, efficienza, libertà da rischi e soddisfazione in contesti d'uso specifici.

Usability (quality-in-use)

Misura fino a che punto gli utenti raggiungono i loro obietivi in modo efficiente e soddisfacente usando il sistema.

- **Effectiveness**: quanto bene gli utenti completano i task previsti con il sistema.
- **Efficiency**: le risorse (tempo, sforzo) richieste per portare a termine i task.
- **Satisfaction**: comfort ed esperienza positiva dell'utente nell'uso del sistema.

Safety (quality-in-use)

Valuta la capacità del sistema di prevenire danni a persone, ambiente e interessi commerciali.

- **Commercial damage**: quanto bene il sistema previene perdite finanziarie o danni al business.
- **Operator health and safety**: quanto bene protegge gli operatori da rischi per la salute e per la sicurezza.
- **Public health and safety**: prevenzione di rischi o danni al pubblico tramite l'uso del sistema.
- **Environmental harm**: il sistema deve evitare o minimizzare impatti negativi sull'ambiente.

Flexibility (quality-in-use)

Riferita alla capacità del sistema di adattarsi e operare efficacemente in contesti o ambienti diversi.

- **Context conformity**: capacità di adattarsi a requisiti e vincoli di contesti diversi.
- **Context extensibility**: potenziale di espansione o adattamento ad ambienti nuovi o in cambiamento senza modifiche significative.
- **Accessibility**: quanto efficacemente il sistema è usabile da tutte le persone, incluse quelle con disabilità, in ambienti diversi.

Capitolo 2

Lezione 4-5 Requirements Engineering, Elicitation e Analysis

Il posto della RE nel ciclo di vita

La Requirements Engineering è la prima fase del modello a cascata: i requisiti si raccolgono tramite interviste con stakeholder, personas e storie/scenari, e si specificano con use case, linguaggio naturale e modelli di dominio. L'output alimenterà il system design.

I requisiti software

Definition

Requisito
Descrizione di ciò che il sistema dovrebbe fare: i servizi che il sistema deve fornire ai suoi utenti e i vincoli operativi.

Esempi tipici: permettere di cercare corsi per parola chiave o iscriversi a corsi, implementare le interfacce come app native Android e iOS, gestire almeno 10k utenti concorrenti, autenticare anche con sensori di impronta digitali.

> *«The hardest single part of building a software system is deciding precisely what to build. No other part of the conceptual work is so difficult as establishing the detailed technical requirements [...]. No other part of the work so cripples the resulting system if done wrong. No other part is more difficult to rectify later.»*
>
> – Fred P. Brooks

Livelli di requisiti

Il termine "requisito" è usato in modo incoerente nell'industria del software: a volte indica una descrizione astratta, a volte una definizione formale e dettagliata.

Definition

User requirements
Descrizione astratta e di alto livello di un servizio che il sistema deve fornire o di un vincolo sul sistema, nella prospettiva degli utenti finali.

Lo stesso bisogno, visto dalla parte del sistema, diventa una definizione formale.

Definition

System requirements
Definizione più dettagliata e formale di una funzione di sistema, nella prospettiva del sistema da costruire.

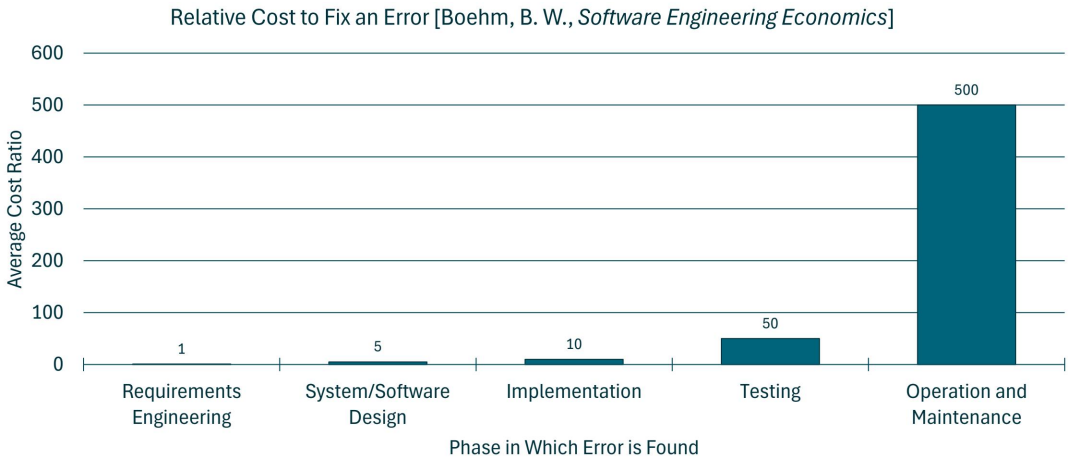
Il rapporto tra i livelli è di progressivo dettaglio: come user requirement "il sistema deve generare un report mensile delle iscrizioni ai corsi" si espande in system requirements che specificano il giorno di generazione, il contenuto per ogni corso e la restrizione dell'accesso ai soli course manager.

Ruolo contrattuale

L'ambiguità tra i livelli è inevitabile perché i requisiti servono a una doppia funzione.

- **User requirements**: possono essere la base di un'offerta per un contratto; il cliente definisce requisiti generali e contraenti diversi propongono modi diversi di soddisfarli.
- **System requirements**: possono essere la base del contratto stesso; una volta aggiudicata la commessa, l'appaltatore formula una serie dettagliata di system requirements perché il cliente capisca esattamente cosa farà il software e possa validare la proposta; una volta accettati e validati, entrano nel contratto finale e sono vincolanti.

Il costo relativo di correzione di un errore cresce lungo il ciclo di vita del progetto, quindi un errore nei requisiti è il più costoso da rettificare.



Definition

Requirements Engineering

Sotto-area dell'ingegneria del software che si occupa del processo di definizione dei requisiti di un software da sviluppare; fornisce a ingegneri del software metodi, tecniche e strumenti per capire e documentare cosa un sistema deve fare.

Requisiti funzionali e non funzionali

I requisiti sono spesso classificati in due grandi famiglie.

Definition

Requisiti funzionali

Servizi che il sistema deve fornire, e come il sistema deve reagire a particolari input o comportarsi in determinate situazioni.

I primi descrivono i servizi, i secondi i vincoli su quei servizi.

Definition

Requisiti non funzionali

Vincoli sui servizi o sulle funzioni offerte dal sistema: includono vincoli temporali, di processo o imposti da standard; spesso si applicano all'intero sistema più che a singole feature o servizi.

In pratica la distinzione non è netta: un requisito apparentemente non funzionale, sviluppato in maggiore dettaglio, genera requisiti chiaramente funzionali ("solo utenti autorizzati possono accedere" → login e autenticazione). I requisiti non sono indipendenti: uno spesso genera o vincola altri requisiti.

Espressi come requisiti utente, i requisiti funzionali si scrivono in linguaggio naturale per essere capiti da non tecnici (utenti, manager); come system requirements vanno dettagliati con accuratezza, descrivendo input, output ed eccezioni, così che gli ingegneri sappiano esattamente cosa implementare.

Criticità dei requisiti non funzionali

I requisiti non funzionali non riguardano servizi specifici: specificano o vincolano caratteristiche del sistema nel suo insieme, come implementarlo (es. usare il linguaggio Java) o altre proprietà (tempi di risposta, uso di memoria).

Warning

Criticità

Gli utenti possono trovare un modo per aggirare un'implementazione sub-ottimale di un requisito funzionale, ma non rispettare un requisito non funzionale può rendere il sistema inutilizzabile: un sistema non conforme al GDPR non può essere distribuito in Europa.

Per questo i requisiti non funzionali sono spesso più critici di quelli funzionali.

Tipi di requisiti non funzionali

I requisiti non funzionali possono derivare da tre fonti.

- **Product requirements**: caratteristiche richieste del prodotto (quanto deve essere veloce, quanta memoria richiedere, requisiti di usabilità, tassi di fallimento accettabili).

- **Organizational requirements**: organizzazioni del cliente e degli sviluppatori (processi operativi che descrivono come il sistema sarà usato, linguaggi di programmazione richiesti, requisiti sull'ambiente operativo).
- **External requirements**: fonti esterne (cosa serve per l'approvazione di un regulator di terze parti, requisiti legali, requisiti etici perché il sistema sia accettabile per utenti e pubblico).

Requisiti di dominio

Definition

> **Requisiti di dominio**
>
> **Requisiti** derivanti dallo scopo particolare o dal contesto industriale in cui un software è usato; possono essere funzionali o non funzionali.

Un esempio: il software di controllo di un dispositivo medico deve garantire la sicurezza secondo lo standard IEC 60601-1 (Medical Electrical Equipment). Il requisito vincola sia il design del sistema sia il processo di sviluppo: un software perfettamente funzionale ma non sviluppato secondo lo standard non sarebbe utilizzabile in pratica.

Proprietà dei buoni requisiti

Un buon requisito dovrebbe avere le seguenti proprietà.

- **Chiaro e facile da capire**: in particolare per gli user requirements.
- **Non ambiguo**: l'ambiguità porta a dispute con i clienti.
- **Completo**.
- **Consistente**: non deve entrare in conflitto con altri requisiti.
- **Testabile**: deve essere possibile determinare senza ambiguità se il sistema lo soddisfa; la mancanza di testabilità porta a dispute con i clienti.

I demoni dell'ambiguità

Un requisito apparentemente chiaro nasconde spesso interpretazioni multiple. Due tecniche per far emergere le aree sfumate:

- **Dissezionare la descrizione**: le parole sono aperte a interpretazioni diverse (un'espressione innocua può ammettere più letture molto diverse tra loro).
- **Poke at the borders**: i termini mappano imperfettamente l'uno sull'altro, quindi occorre trovare le aree sfumate (come si controlla che un servizio remoto sia in esecuzione? Si legge un flag? Il flag può essere impostato anche a servizio non attivo?).

Requisiti e testabilità

Se non è possibile determinare senza ambiguità se il sistema soddisfa un requisito, gli sviluppatori possono ritenerlo soddisfatto mentre il cliente non è affatto d'accordo. Per questo:

- i **system functional requirements** devono essere il più dettagliati possibile e non lasciare spazio a interpretazioni;
- i **system non-functional requirements** devono includere indicatori quantitativi quando possibile.

Non testabili: "il sistema deve essere affidabile", "facile da usare", "veloce e reattivo". Le stesse esigenze in forma testabile:

- **Uptime**: disponibilità mensile del 99,9%.
- **Facilità d'uso**: tutte le funzioni usabili dopo al massimo due ore di training, con non più di due errori utente in media per ora d'uso.
- **Reattività**: tempo medio di risposta non superiore a 100 ms.

Metriche per requisiti non funzionali

Alle proprietà tipiche si associano metriche quantitative.

Proprietà	Metriche
Performance	Operazioni elaborate al secondo, tempo di risposta a utenti/eventi, frequenza di refresh dello schermo
Size	Megabyte

Proprietà	Metriche
Ease of use	Tempo di training richiesto, tasso di errori utente, numero di richieste di supporto
Reliability	Tempo al guasto, tasso di disponibilità (uptime)
Robustness	Tempo di ripristino dopo un guasto, probabilità di perdita dati in caso di guasto

Il processo di Requirements Engineering

Il processo ha tre attività chiave.

1. **Elicitation e analisi**: scoprire i requisiti interagendo con gli stakeholder.
2. **Specification**: convertire i requisiti in una forma standardizzata.
3. **Validation**: controllare che i requisiti definiscano davvero il sistema che il cliente vuole.

In pratica il processo non è lineare: le attività sono intrecciate in un processo **iterativo** a diversi livelli di granularità.

- Prima iterazione: **requisiti di business** (perché serve il progetto? chi ne beneficia? quando e dove avrà luogo?).
- Poi: **user requirements**.
- Infine: **system requirements**.

Ogni iterazione include fasi di elicitation e analisi, specification e validation; le iterazioni si appoggiano su feasibility study, prototyping e review.

Elicitation e analisi dei requisiti

L'obiettivo è scoprire cosa i clienti vogliono e di cui hanno bisogno: più facile a dirsi che a farsi. L'elicitation è considerata la parte più critica della RE perché coinvolge la collaborazione tra stakeholder diversi.

Definition

Stakeholder

Persone o gruppi affetti in qualche modo da un progetto di sviluppo software: utenti finali (anche un gruppo eterogeneo, ad es. vendita, logistica e customer care per un order management system), chi usa gli output del software, management ed executive del cliente, staff tecnico del cliente che opera il sistema quotidianamente.

Sfide dell'elicitation

Gli stakeholder pongono sfide peculiari al processo.

- Non sanno cosa vogliono dal software se non nei termini più generali, non sanno cosa è fattibile e possono fare richieste irrealistiche.
- Sono esperti del proprio dominio: esprimono i requisiti nel proprio gergo, con conoscenza implicita del loro lavoro, dando per scontati dettagli che non lo sono.
- Stakeholder diversi esprimono requisiti diversi in modi diversi: l'ingegnere dei requisiti deve districare comunanze e conflitti.
- Fattori politici possono influenzare il processo: manager possono chiedere requisiti che accrescano la loro influenza nell'organizzazione.
- Hanno visioni diverse (e a volte in conflitto) su importanza e priorità dei requisiti; chi si sente ignorato può cercare deliberatamente di minare il processo.
- I requisiti cambiano durante l'elicitation: nuovi requisiti emergono da stakeholder non inizialmente considerati.

Il processo di elicitation e analisi

Il processo si articola in quattro passi.

1. **Discovery e understanding**: interagire con gli stakeholder per scoprire i requisiti.
2. **Classificazione e organizzazione**: i requisiti correlati vengono raggruppati e organizzati.
3. **Prioritizzazione e negoziazione**: risolvere i conflitti derivanti da bisogni contrastanti di stakeholder diversi.
4. **Documentazione**: tenere traccia dei requisiti scoperti per l'iterazione successiva del processo.

Tecniche di elicitation

Approcci diversi possono essere usati e combinati per scoprire i requisiti: interviste con stakeholder, osservazione ed etnografia, personas, storie e scenari, mock-up e prototipi.

Interviste con gli stakeholder

Le interviste formali o informali sono fondamentali in RE: il team chiede agli stakeholder del loro lavoro, del sistema attualmente usato e del sistema da sviluppare; i requisiti derivano dalle risposte.

- **Closed**: lo stakeholder risponde a una lista di domande predefinite.
- **Open**: non c'è un'agenda predefinita.
- In pratica si combinano: le interviste completamente aperte raramente funzionano bene; meglio partire da domande definite in anticipo, che portano ad altri temi discussi in modo meno strutturato.

Le persone parlano volentieri del proprio lavoro, ma il loro tempo va rispettato minimizzando l'impatto delle interviste. Sfide tipiche:

- gli stakeholder usano gergo e danno per scontati alcuni requisiti;
- conoscono il proprio lavoro ma possono avere una visione parziale o distorta di quello dei colleghi di altre aree o dipartimenti;
- possono essere riluttanti a discutere requisiti e vincoli organizzativi per sottili dinamiche di potere nell'organizzazione.

Etnografia

Definition

Etnografia

Tecnica osservativa per capire i processi operativi: l'ingegnere dei requisiti si immerge nell'ambiente di lavoro degli utenti finali, dove il sistema sarà usato, osservando il lavoro quotidiano e annotando i task effettivi e i partecipanti coinvolti.

Il software raramente esiste in isolamento: è usato in un ambiente sociale e organizzativo che genera o vincola i requisiti; spesso i processi realmente usati in un'organizzazione differiscono da quelli formali che dovrebbero essere usati. L'etnografia è particolarmente efficace per scoprire:

- requisiti derivanti da come le persone lavorano realmente, non da come le definizioni dei processi dicono che dovrebbero lavorare;
- requisiti derivanti dalla cooperazione e dalla consapevolezza delle attività altrui (es. prima di inviare un sondaggio, un dipendente verifica con i colleghi che l'ordine sia stato consegnato).

Personas

Conoscere gli utenti è importante per scoprire i requisiti e, in seguito, per progettare interfacce efficaci. A volte gli utenti sono noti (es. strumenti per altri ingegneri del software), ma in generale il software è sviluppato per altre persone con bisogni diversi: gli utenti reali sono diversi da chi sviluppa, e l'esperienza personale non fornisce un quadro completo.

Definition

Persona

Archetipo ipotetico di utenti reali: non è una persona reale, ma la rappresenta; sebbene immaginaria, è definita con rigore e precisione e viene scoperta come sottoprodotto del processo di elicitation, non inventata di sana pianta.

Non esiste un modo standard per rappresentarla; gli elementi comunemente usati sono i seguenti.

- **Personalizzazione**: nome, età, breve biografia.
- **Informazioni lavorative**: qual è il lavoro dell'individuo.
- **Educazione**: dettagli di formazione ed esperienza.
- **Goals**: qual è il suo interesse nel software.
- **Frustrazioni/pain point**: difficoltà che il software può contribuire a risolvere.

Le personas sono uno strumento per acquisire una comprensione completa degli utenti e promuovere l'empatia. Sono particolarmente utili quando non ci sono stakeholder specifici da intervistare: sviluppo di software off-the-shelf per il pubblico generale, o software custom i cui end-user sono il pubblico. Dall'analisi di una persona possono emergere requisiti nuovi non scoperti con le interviste.

Storie

Definition

Storia

Descrizione di come il sistema può essere usato per un particolare task: cosa fa l'utente, quali informazioni usa, di quale output ha bisogno.

Le storie si scrivono come testo narrativo e presentano una descrizione di alto livello dell'uso del sistema.

- È più facile relazionarsi a esempi di vita reale che ad astrazioni.
- Gli stakeholder non sono bravi a enunciare requisiti di sistema, ma descrivono efficacemente come fanno le cose o come le vorrebbero fare con un nuovo modo di lavorare.
- Sono efficaci nel presentare goal di "quadro generale" a cui è facile relazionarsi.

Mockup a bassa fedeltà

Definition

Low-fidelity mockup (wireframe)

Schizzo semplificato delle interfacce utente del sistema: modo rapido ed economico per visualizzare struttura e flussi di base; l'attenzione è su funzionalità e flusso, non sull'estetica.

I mockup a bassa fedeltà portano benefici concreti al processo.

- **Facilitano la comprensione**: aiutano gli stakeholder a afferrare la funzionalità di base e sono un punto di partenza per la discussione.
- **Aiutano a scoprire nuovi requisiti**: è più facile ragionare su interfacce concrete che su affermazioni astratte.
- **Engagement e feedback iterativo**: promuovono il coinvolgimento degli stakeholder e permettono iterazione e feedback rapidi senza investimenti significativi nel design.
- **Fondazione per lo sviluppo**: base solida per evolvere verso design a fedeltà maggiore e per lo sviluppo finale.

Strumenti possibili: carta e matita per gli schizzi iniziali; strumenti digitali commerciali come Balsamiq o Figma; alternative open-source come mydraft.cc, draw.io e Pencil Project.

Lezione 6-7 Requirements Engineering & Specification e Validation

Specificazione dei requisiti

Definition

Specificazione dei requisiti
Processo di scrittura degli user e system requirements in un documento dei requisiti.

- Gli **user requirements** devono essere comprensibili a utenti finali e clienti senza background tecnico.
- I **system requirements** sono più dettagliati e possono includere informazioni tecniche.
- I requisiti possono far parte di un contratto per lo sviluppo del sistema: è quindi importante che siano completi e dettagliati il più possibile.

Requisiti e design

In linea di principio i requisiti dicono **cosa** il sistema deve fare e il design descrive **come** lo fa; in pratica requisiti e design sono inseparabili.

- I requisiti possono essere strutturati e organizzati sulla base di un'architettura di sistema ad alto livello.
- Il sistema può interoperare con altri sistemi che generano requisiti di design.
- L'uso di un'architettura specifica per soddisfare requisiti non funzionali può essere un requisito di dominio.

Approcci di specificazione

Sono possibili approcci diversi, con formalità crescente.

- **Linguaggio naturale**: requisiti come frasi numerate, una frase per requisito; espressivo, intuitivo e universale, comprensibile a utenti e clienti, può esprimere requisiti funzionali e non.
- **Linguaggio naturale strutturato**: uso di un form o template standardizzato.
- **Notazioni semi-formali e modelli**: UML use case diagram e altri modelli di dominio, tipicamente integrati da annotazioni in linguaggio naturale.
- **Formal specification**: notazioni basate su concetti matematici come macchine a stati (finite e infinite) e logiche temporali.

Approcci diversi si usano in domini diversi: nei sistemi safety-critical è comune usare specifiche formali e linguaggio naturale strutturato; in un'applicazione piccola si può usare linguaggio naturale non strutturato; nei sistemi informativi di media-grande taglia le notazioni semi-formali sono un buon compromesso.

Linee guida per la specifica in linguaggio naturale

Il linguaggio naturale rende difficile la precisione, quindi servono linee guida di scrittura.

- Definire un formato "standard" e usarlo per tutti i requisiti (es. `shall`).
- Usare il linguaggio in modo consistente: `shall` per i requisiti obbligatori, `should` per quelli desiderabili.
- Usare l'evidenziazione del testo per identificare le parti chiave del requisito.
- Evitare il gergo informatico.
- Includere una spiegazione (**rationale**) del perché un requisito è necessario.

Problemi del linguaggio naturale

La specifica in linguaggio naturale soffre di tre problemi tipici.

- **Lack of clarity**: la precisione è difficile senza rendere il documento difficile da leggere.
- **Requirements confusion**: i requisiti funzionali e non funzionali tendono a mescolarsi.
- **Requirements amalgamation**: requisiti diversi possono essere espressi insieme nella stessa frase.

Specificazioni strutturate

Definition

Specificazione strutturata

Approccio in cui la libertà del redattore dei requisiti è limitata e un modo standardizzato di scriverli è imposto; funziona bene per certi tipi di requisiti (es. sistemi embedded di controllo) ma a volte è troppo rigida per i requisiti di sistemi aziendali.

Example

Esempio classico di form strutturato (software di controllo di una pompa insulinica, requisito 3.3.2 del SRS).

Campo	Contenuto
Funzione	Calcolo della dose di insulina: livello di zucchero sicuro
Descrizione	Calcola la dose di insulina da erogare quando il livello di zucchero misurato è tra 3 e 7 unità
Input	Lettura corrente `r_2`, le due letture precedenti `r_0` e `r_1`
Sorgente	Lettura corrente dal sensore; le altre letture dalla memoria
Output	`CompDose`: la dose di insulina da erogare
Azione	`CompDose` è zero se il livello è stabile o in calo, o se cresce con ritmo in diminuzione; se livello e ritmo crescono, si calcola la differenza tra livello attuale e precedente divisa per 4, arrotondata; se il risultato è zero, si imposta la dose minima erogabile
Richieste	Le due letture precedenti, per calcolare il tasso di variazione del livello di zucchero
Precondizione	Il serbatoio di insulina contiene almeno la dose singola massima consentita
Postcondizione	In memoria `r_0` è sostituito da `r_1`, poi `r_1` da `r_2`
Effetti collaterali	Nessuno

Use case diagram

I **use case** descrivono le interazioni tra utenti e sistema usando un modello grafico e testo in linguaggio naturale strutturato; sono una parte chiave dell'UML (Unified Modelling Language) e rappresentano l'insieme dei requisiti funzionali di un sistema.

- Identificano **attori** (categorie di utenti, non necessariamente umane) e **use case** (tipi di interazioni o feature offerte dal sistema).
- Informazioni aggiuntive sulle interazioni si forniscono con descrizioni testuali (strutturate) o modelli semi-formali (es. UML sequence diagram o statechart).
- Supportano la comunicazione con il cliente per definire le funzionalità: vanno tenuti il più semplici possibile, definiscono cosa il sistema deve fare e non come è implementato, dal punto di vista degli utenti (il sistema è una black box).
- Sono spesso documentati con un use case diagram ad alto livello.

Attori

Definition

Attore

Entità esterna che interagisce con il sistema in sviluppo (SUD): classi di utenti umani, altri sistemi, ambiente fisico. È rappresentato con uno stick figure e ha un nome unico.

Gli attori sono più a grana grossa delle personas: un attore può essere associato a più personas. Per identificarli ci si può chiedere:

- quali gruppi di utenti sono supportati dal SUD nel loro lavoro?

- quali gruppi eseguono le funzioni principali offerte dal SUD?
- quali gruppi eseguono le funzioni secondarie, come l'amministrazione?
- il SUD interagirà con sistemi o software esterni? Ogni sistema esterno con cui il SUD interagisce sarà un attore.

Un attore non corrisponde a una singola entità: rappresenta una classe di utenti con lo stesso ruolo. Un utente può avere ruoli diversi nello stesso sistema: l'amministratore di un sito può anche visitarvisi come utente non autenticato.

Use case ed ellissi

Gli use case sono rappresentati come **ellissi nominate** e corrispondono a feature del sistema che offrono beneficio o utilità agli attori; modellano i requisiti funzionali.

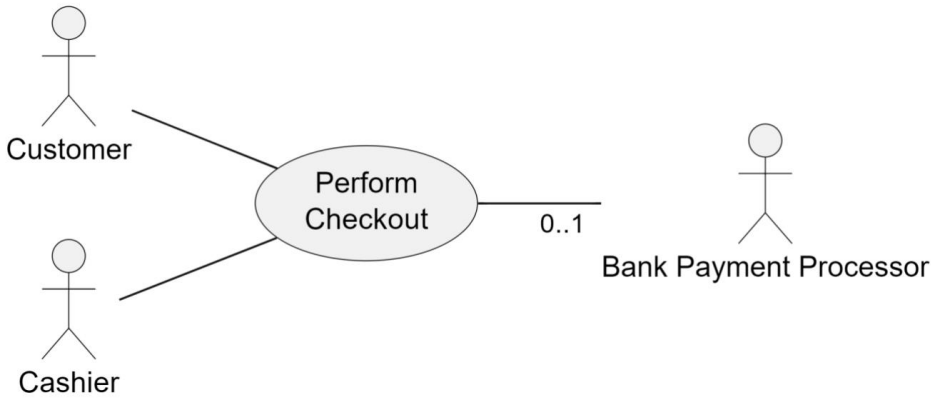
- Un use case astrae molti possibili **scenari** (sequenze di azioni) per una data feature: uno scenario è un'istanza di un use case.
- Uno use case rappresenta una classe di scenari che mirano a usare la stessa funzionalità (es. ci possono essere più modi di iscriversi a un corso).

Associazioni

Un'**associazione** tra un attore e uno use case indica che l'attore può eseguire quel use case; graficamente è una linea che collega l'attore e l'ellisse.

Attori secondari

Uno use case può essere associato a più attori: la semantica è che gli attori devono **collaborare** in qualche modo per eseguirlo. Le cardinalità sono supportate in UML 2.5: un attore esterno può partecipare in modo opzionale (es. cardinalità `0..1`).



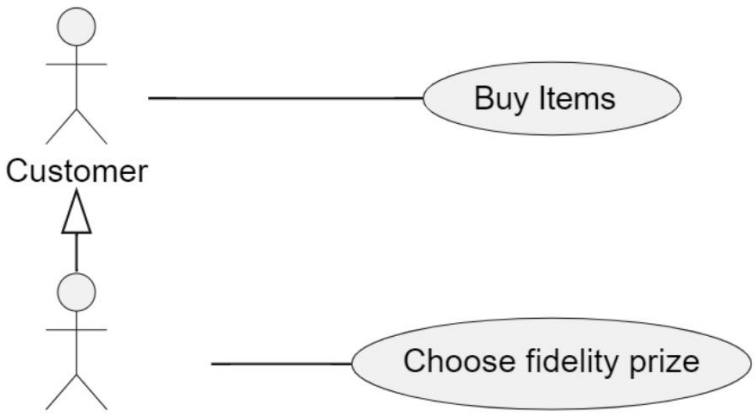
Example

Customer e Cashier devono cooperare per eseguire il checkout, e opzionalmente può essere coinvolto un Bank Payment Processor (sistema esterno), ad esempio quando si paga con carta.

I diagrammi UCD non hanno strumenti per specificare come gli attori coinvolti interagiscono e le diverse responsabilità: servono descrizioni aggiuntive.

Generalizzazione di attori

La **generalizzazione** tra attori si applica quando un attore è sotto-tipo di un altro: stesso concetto e notazione dei class diagram UML, graficamente una freccia con punta "vuota". L'attore specializzato può eseguire tutti gli use case che il padre può eseguire.



Errori comuni nei use case diagram

Error

Errori da principianti

- Gli use case devono dare un beneficio all'attore o aiutarlo a raggiungere un obiettivo: i nomi degli use case dovrebbero contenere un verbo, quelli degli attori un sostantivo.
- Due attori associati allo stesso use case con cardinalità non nulla sono coinvolti (e devono collaborare) in ogni istanza dello use case: non significa che entrambi possano eseguirlo indipendentemente.
- Attenzione alle generalizzazioni improprie: ogni attore dovrebbe avere use case propri, oltre a quelli ereditati; se l'attore specializzato non ha use case propri, la generalizzazione è inutile o mancano use case.
- I diagrammi non devono diventare troppo complessi: relazioni tra use case e generalizzazioni con moderazione, modellazione a un grado di astrazione sufficientemente alto, ordine grafico (evitare linee che si incrociano). Un diagramma complesso è l'indicazione di un analista scarso.

Formati dei use case

I use case si scrivono in formati e livelli di formalità diversi.

- **Brief**: riassunto stringato in un paragrafo, di solito dello scenario di successo principale.
- **Casual**: formato a paragrafi informale; più paragrafi che coprono scenari vari.
- **Fully-dressed**: tutti i passi e le variazioni scritti in dettaglio, con sezioni di supporto come precondizioni e garanzie di successo.

Brief e casual si usano nelle fasi iniziali della specifica per farsi un'idea rapida di soggetto e scope; i fully-dressed si sviluppano più avanti, come base di un contratto e per specificare in maggiore dettaglio il comportamento del sistema da sviluppare.

Descrizioni testuali di uno use case

Una descrizione testuale include in genere quattro parti.

1. Cosa si aspettano sistema e utenti quando lo use case inizia.
2. Il flusso normale degli eventi (**main scenario**).
3. Cosa può causare errori e come i problemi risultanti vengono gestiti.
4. Lo stato del sistema al completamento dello use case.

Il template di Cockburn

Per le descrizioni fully-dressed esistono formati diversi; un riferimento è il template proposto da Alistair Cockburn.

Campo	Contenuto
`USE CASE #`	Numero e nome dello use case
`Goal in Context`	Obiettivo dello use case

Campo	Contenuto
`Preconditions`	Condizioni che devono valere per avviarlo
`Success End Condition`	Stato del sistema se lo use case ha successo
`Failed End Condition`	Stato del sistema se lo use case fallisce
`Primary Actor`	Attore principale
`Trigger`	Azione dell'attore principale che avvia lo use case
`Main Scenario`	Tabella dei passi: numero, azioni degli attori, risposta del sistema
`Extensions`	Varianti ed errori, in tabelle analoghe al main scenario
`Open Issues`	Aspetti ancora da chiarire; alla consegna del documento deve essere vuota

Scenario principale ed estensioni

Definition

Scenario principale

Sequenza di azioni che si verifica quando tutto nello use case procede come previsto (main success scenario).

Non tutto però procede sempre come previsto.

Definition

Estensione

Descrizione di una sequenza alternativa di azioni che può verificarsi nello use case: un'altra via per eseguirlo o un errore a un certo punto.

Gli utenti possono eseguire lo use case in modi diversi (es. autenticazione con PIN o con impronta) e può verificarsi un errore; nel definire il comportamento funzionale del sistema vanno descritte anche queste sequenze alternative tramite estensioni. Tipicamente c'è molto più testo nelle estensioni che nello scenario principale.

Example

Un esempio completo mostra la struttura: use case "Withdraw money" di un ATM, con attore primario Customer.

Passo	Customer	System
1	Tocca lo schermo	
2		Mostra M1
3	Inserisce la carta	
4		Mostra M2
5	Tocca il pulsante "PIN"	
6		Mostra M3
7	Inserisce il PIN	
8		Mostra M4
9	Tocca il pulsante "Withdraw"	
10		Mostra M5
11	Tocca il pulsante "50 €"	
12		Eroga il denaro, espelle la carta, mostra M6

Le estensioni coprono gli errori e le varianti: PIN non corretto, saldo insufficiente, contanti insufficienti nell'ATM, carta rubata o illeggibile, autenticazione biometrica al posto del PIN, ricevuta cartacea opzionale.

Passo	Customer	System
7a `<PIN errato inserito>`	Inserisce il PIN	
8a		Mostra M7 e termina lo use case

Il messaggio M7 informa che l'autenticazione è fallita e invita a recuperare la carta; altre estensioni, come il saldo insufficiente, possono chiudersi tornando a un passo dello scenario principale.

Validazione dei requisiti

La validazione si preoccupa di dimostrare che i requisiti definiscano il sistema che il cliente vuole davvero.

Warning

Costo degli errori nei requisiti

I costi degli errori nei requisiti sono alti: correggere un errore di requisito dopo la consegna può costare fino a 100 volte la correzione di un errore di implementazione, quindi la validazione è molto importante.

Controlli sui requisiti

La validazione esamina cinque proprietà dell'insieme dei requisiti.

- **Validità**: il sistema fornisce le funzioni che supportano al meglio i bisogni del cliente?
- **Consistenza**: ci sono requisiti in conflitto tra loro?
- **Completezza**: tutte le funzioni richieste dal cliente sono incluse?
- **Realismo**: i requisiti sono implementabili con il budget e la tecnologia disponibili?
- **Verificabilità**: i requisiti possono essere controllati?

Tecniche di validazione

- **Requirements reviews**: analisi manuale sistematica dei requisiti; review regolari durante la formulazione della definizione, con coinvolgimento sia dello staff del cliente sia dell'appaltatore; le review possono essere formali (con documenti completati) o informali, e una buona comunicazione tra sviluppatori, clienti e utenti risolve i problemi in stadio iniziale.
- **Prototyping**: uso di un modello eseguibile semplificato del sistema per controllare i requisiti; anche visual prototyping, con mockup o wireframe.
- **Test-case generation**: sviluppo di test per i requisiti per verificarne la testabilità.

Controlli in review

Durante una review si controllano quattro proprietà di ogni requisito.

- **Verificabilità**: il requisito è realisticamente testabile?
- **Comprensibilità**: il requisito è compreso correttamente?
- **Tracciabilità**: l'origine del requisito è dichiarata chiaramente?
- **Adattabilità**: il requisito può cambiare senza un grande impatto sugli altri requisiti?

Capitolo 4

Lezione 8 UML Statecharts

UML Statecharts

Definition

UML Statechart

Diagramma comportamentale (noto anche come UML *Behavioural State Machine*), estensione degli *Statecharts* di Harel, usato per modellare gli aspetti dinamici di un sistema.

- **Sistemi reattivi**: sistemi che reagiscono a eventi esterni o interni; esempi tipici sono il software con GUI, gli oggetti nella programmazione orientata agli oggetti e i controllori embedded (es. ABS di un'automobile).
- **Usage industriale**: gli Statecharts sono usati anche oltre la modellazione; il software *embedded* viene spesso generato automaticamente da Statecharts, e la logica delle interfacce utente moderne può essere gestita tramite Statecharts.

Stati e transizioni

- **Stato**: rappresenta una situazione in cui vale una condizione invariante; può essere **statico** (il sistema attende che qualcosa accada) o **dinamico** (il sistema sta eseguendo un compito specifico).
- **Transizione**: rappresenta un possibile cambiamento di stato.

Sintassi

Regioni, vertici e transizioni

- **Statechart UML**: contiene una **regione top-level**; una **regione** contiene **vertici** e **transizioni**.
- **Vertice**: rappresenta uno stato; ne esistono diversi tipi con semantiche diverse.
- **Transizione**: disegnata come arco diretto (*edge*) tra due vertici.

Stati semplici

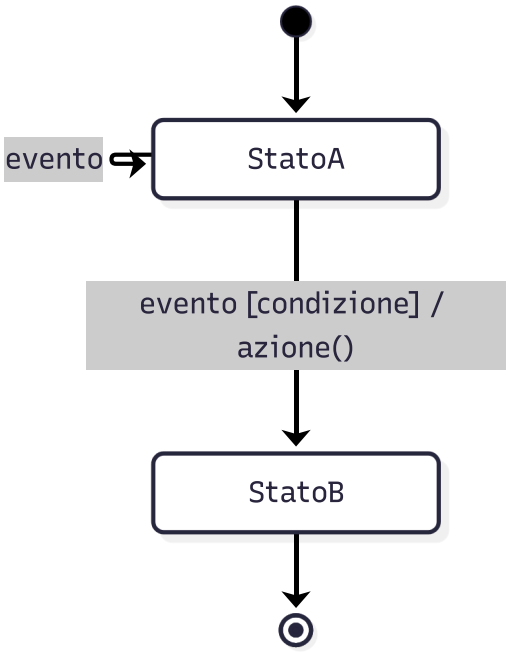
- **Stato semplice**: rappresenta stati di sistema non strutturati; è disegnato come rettangolo con angoli arrotondati.
- **Name compartment**: contiene il nome (opzionale) dello stato, come stringa.

Pseudostati iniziali e stati finali

- **Pseudostato iniziale**: marca lo stato di default (iniziale); una regione può contenerne al massimo uno.
- **Stato finale**: modella una situazione in cui il calcolo è completato, cioè il sistema non processerà altri eventi.

Sintassi delle transizioni

- **Etichetta di transizione**: ha la forma `triggers [guard ] / actions`, dove ogni parte è opzionale.
- **Trigger**: lista di eventi che possono indurre un cambiamento di stato.
- **Guard**: condizione booleana.
- **Actions**: lista di operazioni da eseguire quando la transizione scatta (*fires*).
- **Self-transition**: una transizione può puntare allo stesso stato di partenza.



Semantica

Una transizione è **fireable** quando:

- sono stati generati eventi che corrispondono a **tutti** i trigger;
- la condizione nella **guard** valuta `TRUE`.
- **Transizione spontanea**: transizione senza trigger né guard.
- **Esecuzione delle azioni**: dopo lo scatto di una transizione viene eseguita la lista di azioni associata.
- **Non-determinismo**: se più transizioni sono fireable, ne scatta una sola, determinata non-deterministicamente.

```java
public class Counter {
    private int count = 0;
    private String mode = "increment";
    public void flick() {
        if (mode.equals("increment")) count++;
        else count--;
        if (count == 10) mode = "decrement";
        else if (count == 0) mode = "increment";
    }
}
```

Il contatore illustra come gli stati (la **mode**) e le transizioni siano codificati in un oggetto: variabili di istanza per lo stato, ramo condizionale per la transizione.

Attività interne degli stati

Uno stato può contenere, opzionalmente, una lista di **attività interne**, ciascuna caratterizzata da un'etichetta che indica quando invocarla.

- `entry /`: attività eseguita all'ingresso nello stato.
- `do /`: attività eseguita per tutto il tempo in cui il sistema resta nello stato (dopo il completamento delle attività `entry`).
- `exit /`: attività eseguita all'uscita dallo stato.

Stati compositi

- **Stato composito**: uno stato che contiene una **regione interna**; può avere name compartment, attività interne e una o più regioni interne.
- **Substate**: gli stati di una regione interna di un composito.
- **Gerarchia**: gli stati compositi permettono di definire una struttura gerarchica; la regione interna dettaglia il comportamento dello stato a cui appartiene.
- **Nascondere la complessità**: offrono un modo elegante e conciso di modellare comportamenti complessi e di nasconderla quando non necessaria.

Regioni parallele

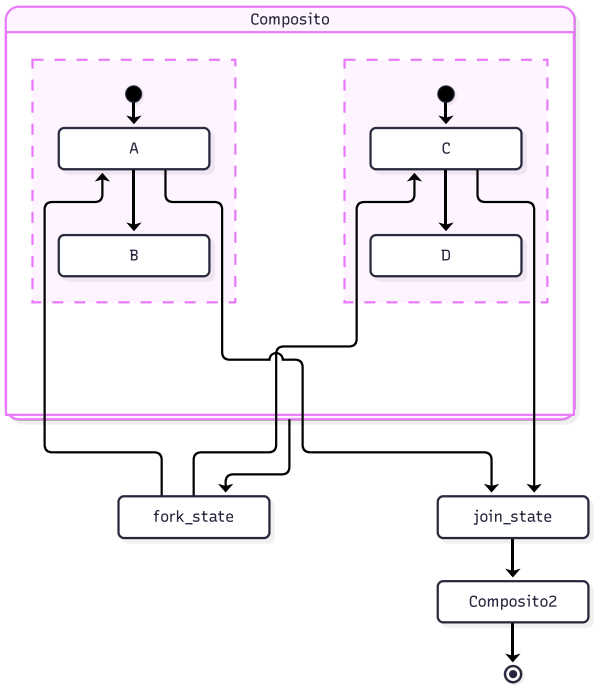
- **Regioni parallele**: un composito può contenere più regioni, che rappresentano comportamenti che possono avvenire in parallelo.
- **Uscita dal composito**: all'uscita da uno stato composito tutte le sue regioni vengono terminate.

Pseudostati di history

- **Shallow history** (`H`): rappresenta lo stato più recente attivo in un composito, ma **non** i substate di quel substate; presente solo negli stati compositi, al massimo uno per regione.
- **Deep history** (H_*): come lo shallow history, ma ripristina l'intera configurazione della regione, substate dei substate inclusi.

Pseudostati di fork e join

- **Fork**: divide una transizione in ingresso in più transizioni che entrano in vertici di regioni ortogonali.
- **Join**: fonde transizioni uscenti da vertici di regioni ortogonali in un'unica transizione.
- **Equivalenza**: fork e join sono semanticamente equivalenti alle transizioni che entrano/escono singolarmente dai vertici delle regioni parallele.



Consigli di modellazione

- **Transizioni di ingresso/uscita**: ogni stato dovrebbe avere tipicamente almeno una transizione in ingresso e una in uscita.

- **Senso di lettura**: i diagrammi si leggono tipicamente dall'alto a sinistra verso il basso a destra; posizionare pseudostati iniziali/finali di conseguenza.
- **Stati comuni**: se più stati condividono una condizione di ingresso e/o uscita, considerare l'uso di stati compositi.
- **Non-determinismo**: assicurarsi di non modellarlo, a meno che non sia davvero voluto.
- **Stato attivo**: al più uno stato può essere attivo in una regione in un dato istante.

Model-driven Development

Info

Model-driven Development

Passo successivo nella tendenza all'astrazione crescente; standard de facto in molti domini di software *embedded* (es. automotive). Con strumenti come **Simulink** è possibile simulare modelli Statechart, generare automaticamente codice e test, e applicare metodi formali.

Vantaggi e limiti del Model-driven Development:

- **Convenienza**: in alcuni domini è tipicamente più economico, più rapido e porta a qualità più alta.
- **Comprensibilità**: i modelli sono comprensibili dagli esperti di dominio.
- **Documentazione**: i modelli sono essi stessi documentazione.
- **Indipendenza**: minore dipendenza dalla tecnologia e dal personale.

Warning

Limiti del MDD

- **Costo degli strumenti**: i tool sono costosi.
- **Flessibilità**: non abbastanza flessibile per alcune applicazioni.
- **Code generation**: tipicamente supportata per un numero limitato di piattaforme.

Statecharts per la gestione delle UI

- **Guida alla logica di GUI**: gli Statecharts possono «guidare» la logica dell'interfaccia, essendo più facili da capire del codice.
- **Disaccoppiamento**: il comportamento è disaccoppiato dai componenti della GUI.
- **WHEN vs WHAT**: separare il *quando* (codificato nello Statechart) dal *cosa* deve accadere (codificato nel componente UI).
- **Scalabilità**: gli Statecharts scalano bene al crescere della complessità; studi hanno mostrato un numero di difetti inferiore per controllori di GUI basati su statechart.
- **XState**: libreria open-source JavaScript per creare, interpretare ed eseguire modelli statechart; si integra con librerie UI come React, Vue e Svelte, gestisce lo stato dell'interfaccia tramite Statecharts e supporta il testing. Disponibile su `https://xstate.js.org/`.

Parte II

Architettura di Sistema e Cloud

Capitolo 5

Lezione 10-11 System Design e Design Practices

La progettazione di un sistema

Definition

Software Design

Insieme dei task svolti dall'ingegnere del software per trasformare il modello di analisi nel modello di design del sistema.

L'obiettivo ultimo di System e Object Design è definire un nuovo documento, scritto in linguaggio tecnico e formale, da dare ai programmatori affinché siano in grado di implementare il software descritto nel Documento dei Requisiti Software.

Scopo di System Design e Object Design

Definition

System Design

Attività di progettazione con tre scopi: definire gli obiettivi di design del progetto; definire l'architettura del sistema, decomponendolo in sottosistemi più piccoli che possano essere realizzati in parallelo da team individuali; selezionare le strategie per costruire il sistema (hardware/software, gestione dei dati persistenti, flusso di controllo globale, politiche di controllo degli accessi).

Definition

Object Design

Attività che, data l'architettura del sistema, specifica il dettaglio realizzativo dei sottosistemi più piccoli e seleziona le strategie ottimali per l'implementazione (design pattern).

Il cambio di prospettiva

Il system design cambia radicalmente il punto di vista rispetto alla fase di analisi.

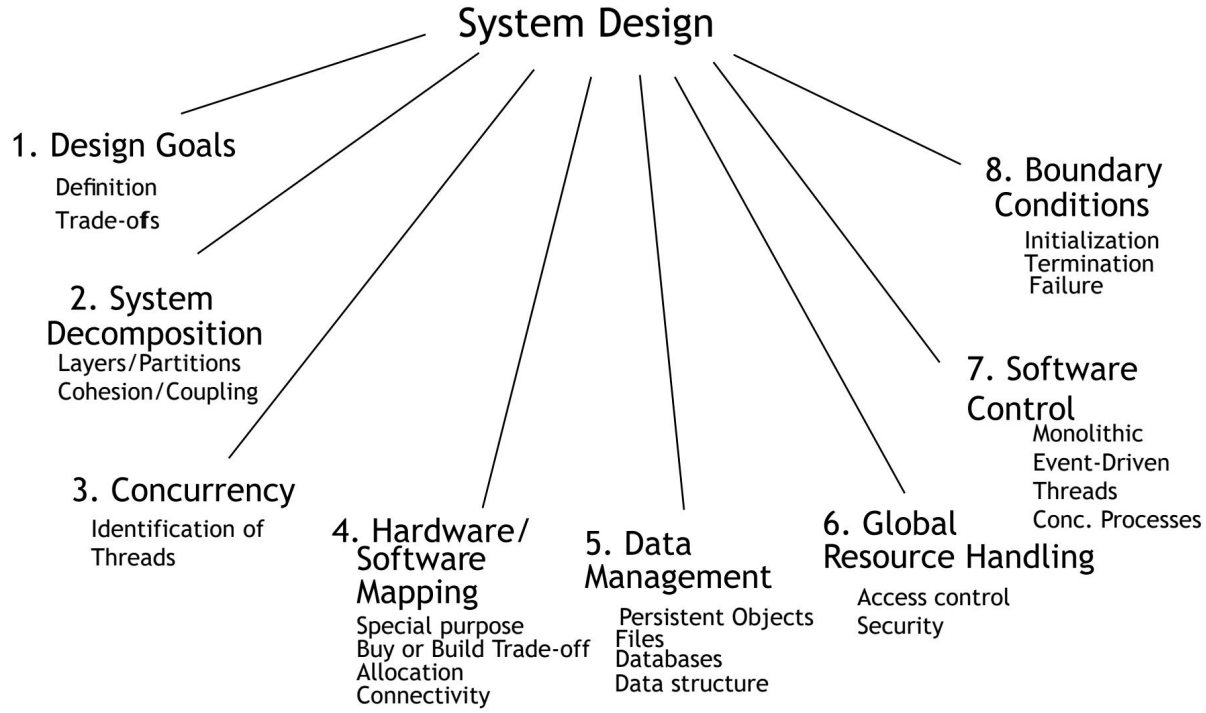
- Finora si è lavorato per interagire col cliente: linguaggio per profani, poco formale.
- Nel system design gli interlocutori sono le squadre di programmatori che dovranno implementare il sistema: chi leggerà i documenti è un esperto di informatica, quindi il linguaggio è per esperti tecnici.## Le attività del system design

Il system design è costituito da tre macro-attività.

1. **Identificare gli obiettivi di design**: gli sviluppatori identificano quali caratteristiche di qualità ottimizzare e ne definiscono le priorità.
2. **Progettare la decomposizione in sottosistemi**: basandosi sugli use case e sui modelli di analisi, gli sviluppatori decompongono il sistema in parti più piccole, usando stili architetturali standard.
3. **Raffinare la decomposizione**: quella iniziale di solito non soddisfa gli obiettivi di design; gli sviluppatori la raffinano finché gli obiettivi non sono soddisfatti.

I passi del system design

Il processo completo si articola in otto passi, dal riconoscimento degli obiettivi qualitativi fino alla definizione dell'architettura.



Gli obiettivi qualitativi del sistema

Identificare gli obiettivi qualitativi è il primo passo del system design: identifica le qualità su cui il sistema deve essere focalizzato.

- Molti design goal si ricavano dai requisiti non funzionali o dal dominio di applicazione; altri sono forniti dal cliente.
- È importante formalizzarli esplicitamente: ogni importante decisione di design deve seguire lo stesso insieme di criteri.

Criteri di design

Gli obiettivi si selezionano da una lunga lista di qualità desiderabili, organizzate in cinque gruppi.

- **Performance**: tempo di risposta, throughput, requisiti di memoria.
- **Affidabilità**: robustezza, disponibilità, tolleranza ai fault, sicurezza.
- **Costi**: sviluppo, manutenzione, gestione.
- **Mantenimento**: estendibilità, modificabilità, adattabilità, portabilità.
- **Usabilità**.

Design trade-offs

Quando si definiscono gli obiettivi di design, solo un piccolo sottoinsieme di questi criteri può essere tenuto in considerazione (non è realistico sviluppare software simultaneamente sicuro ed economico). Gli sviluppatori devono dare priorità agli obiettivi, tenendo conto anche di aspetti manageriali come schedule e budget. Casi tipici di trade-off:

- **Spazio vs. velocità**: se il software non rispetta i requisiti di tempo di risposta e throughput, si usa più memoria per velocizzarlo (caching, ridondanza); se non rispetta i requisiti di memoria, si comprime a discapito della velocità.
- **Tempo di rilascio vs. funzionalità**: con tempi stringenti si rilasciano meno funzionalità di quelle richieste, ma nei tempi giusti.
- **Tempo di rilascio vs. qualità**: con tempi stretti si può rilasciare nei tempi prefissati con dei bug da correggere in seguito, oppure rilasciare in ritardo con meno bug.
- **Tempo di rilascio vs. staffing**: si possono aggiungere risorse al progetto per accrescere la produttività.

Concetti di base di buon design

Molti criteri di design dipendono strettamente dal problema trattato: scegliere il miglior bilanciamento dei vincoli è compito dell'analista, che opera per esperienza e sensibilità. Esistono però criteri generici, validi per qualunque software object-oriented: ben applicati, migliorano notevolmente la qualità interna del codice senza introdurre particolari svantaggi.

Un principio guida trasversale è **pensare in avanti**: quando si progetta una classe bisogna sforzarsi di immaginare quali cambiamenti saranno richiesti in futuro, perché le scelte iniziali devono facilitare l'evoluzione del software.

La decomposizione

Per ridurre la complessità della soluzione si decompone il sistema in parti più piccole, chiamate **sottosistemi**.

- Un sottosistema corrisponde alla parte di lavoro svolgibile autonomamente da un singolo sviluppatore o da un team.
- Sottosistemi relativamente indipendenti permettono ai team di lavorare con un minimo overhead di comunicazione.
- I sottosistemi complessi si decompongono ulteriormente in sottosistemi più semplici.

Il vantaggio della decomposizione

Siano `P` un problema, `C(P)` la sua complessità e `E(P)` lo sforzo per risolverlo. Dati due problemi `P1` e `P2`:

- se `C(P1) > C(P2)` allora `E(P1) > E(P2)`;
- ma `C(P1+P2) > C(P1) + C(P2)`, quindi `E(P1+P2) > E(P1) + E(P2)`: affrontare i sottoproblemi separatamente costa meno che affrontare il problema intero.

Sottosistemi, servizi e interfacce

La modellazione dei sottosistemi ha primitive precise nei linguaggi object-oriented.

- **Subsystem** (in UML: package): collezione di classi, associazioni, operazioni e vincoli correlati.
- **Primitive linguistiche**: Java fornisce i package; C++ e C# usano il costrutto di namespace per lo stesso concetto.

Un sottosistema è caratterizzato dai servizi che fornisce agli altri.

- L'insieme delle operazioni di un sottosistema disponibili agli altri sottosistemi forma l'**interfaccia** del sottosistema: nomi delle operazioni, parametri, tipi e valori di ritorno.
- Il system design si focalizza sulla definizione dei servizi di ogni sottosistema in termini di operazioni, loro parametri e loro comportamento ad alto livello.

Le responsabilità contano più dei dati

Un esempio classico di scelta di design riguarda l'accesso al database: se tutti i sottosistemi accedono direttamente al DB, non si riescono a specificare chiaramente le responsabilità di ogni modulo e tutti diventano vulnerabili ai cambiamenti dell'interfaccia del sottosistema Database (un cambio di DBMS richiede modifiche in tutte le classi che accedono al DB). Introducendo un sottosistema DataAccess che gestisce l'I/O dal database, le responsabilità diventano chiare e al cambio del DBMS si modifica solo il sottosistema Storage.

Warning

Attenzione all'astrazione

Ridurre le relazioni tra sottosistemi può aumentare la complessità: con l'obiettivo di ridurre le relazioni si rischia di aggiungere livelli di astrazione che consumano tempo di sviluppo e tempo di elaborazione.

Indipendenza funzionale

La suddivisione in sottosistemi può essere guidata da due fattori qualitativi fondamentali: **coesione** e **accoppiamento**. L'obiettivo è l'indipendenza funzionale dei sottosistemi, che si massimizza con alta coesione e basso accoppiamento.

La coesione

Definition

Coesione

Misura di quanto siano fortemente correlate e mirate le responsabilità (o i servizi offerti) di un modulo. Se ciascuna unità è responsabile di un singolo compito, l'unità ha alta coesione.

L'alta coesione è una proprietà desiderabile del codice perché permette di comprendere, riusare e manutenere meglio una classe, di limitare e focalizzare i cambiamenti nel codice e di usare nomi appropriati, efficaci e comunicativi. La coesione si valuta a due granularità.

- **Coesione dei metodi**: un metodo dovrebbe essere responsabile di un solo compito ben definito.
- **Coesione delle classi**: ogni classe dovrebbe rappresentare un singolo concetto ben definito.

Il Single Responsibility Principle

Definition

Single Responsibility Principle

Principio per cui una classe dovrebbe avere una sola responsabilità e un solo motivo per cambiare: una responsabilità è un "motivo di cambiamento", e le responsabilità di una classe sono assi di cambiamento.

Se una classe ha due responsabilità, queste sono accoppiate nel design e devono cambiare insieme; una classe con più responsabilità diventa fragile quando cambia uno qualunque dei requisiti. I benefici del SRP: **reuse** (classi più facilmente riusabili), **clarity** (il codice è più pulito), **naming** (un'unica responsabilità rende facile scegliere un buon nome), **readability** (chiarezza, nomi migliori e file più corti).

Una classe con responsabilità multiple viola il principio; il caso tipico è una classe `Report` che raccoglie dati, li formatta e li stampa.

C#

```
class Report
{
  private IList<ReportDataElement> GetData() { ... }
  private void FormatReport() { ... }
  public void Print()
  {
    GetData();
    FormatReport();
    Console.WriteLine("Printing Report...");
  }
}
```

La soluzione separa le responsabilità in classi distinte: `DataAccess` per raccogliere i dati, `ReportFormatter` per la formattazione, `ReportPrinter` per la stampa.

Indicatori di alta coesione

Un'alta coesione si riconosce da segnali concreti.

- La classe ha responsabilità moderate, limitate a una singola area funzionale.
- Collabora con altre classi per completare i task.
- Ha un numero limitato di metodi, con funzionalità altamente legate tra loro.
- Tutti i metodi sembrano appartenere a uno stesso insieme, con un obiettivo globale simile.
- La classe è facile da comprendere.

L'accoppiamento

Definition

Accoppiamento

Misura di quanto fortemente una classe è connessa con, ha conoscenza di, o si basa su altre classi. Due o più classi sono accoppiate quando è impossibile riusare l'una senza dover riusare anche le altre.

Se due classi dipendono strettamente e per molti dettagli l'una dall'altra sono **fortemente accoppiate**; altrimenti sono **debolmente accoppiate**. Per un codice di qualità si punta a un **basso accoppiamento**.

- Un basso accoppiamento permette di capire una classe senza leggere i dettagli delle altre, di modificarla senza conseguenze sulle altre e di riusarla senza dover importare altre classi: migliora la manutenibilità del software.
- Un certo livello di accoppiamento è però insito nel concetto di scambio di messaggi del paradigma object-oriented: è necessario al funzionamento del sistema e va limitato ove possibile.
- Linea guida: definire le responsabilità delle classi in modo da ottenere un basso accoppiamento, anche con la legge di Demetra.

Tipi di accoppiamento in O-O

Date due classi `X` e `Y`, sono forme di accoppiamento:

- `X` ha un attributo di tipo `Y`;
- `X` ha un metodo con un elemento (parametro, variabile locale, ecc.) di tipo `Y`;
- `X` è una sottoclasse, anche indiretta, di `Y`: lo scenario che porta al massimo accoppiamento in assoluto;
- `X` implementa un'interfaccia di tipo `Y`.

Il caso tipico di accoppiamento forte è una classe con un attributo del tipo concreto di cui ha bisogno, come un `Traveler` con un attributo di tipo `Car`: riusarla con un altro mezzo di trasporto o cambiare i metodi di `Car` richiede modifiche. Introducendo un'interfaccia l'accoppiamento si spezza completamente.

Java

```java
class Traveler
{
  Vehicle v;
  void startJourney() { v.move(); }
}

interface Vehicle { void move(); }
```

Le conseguenze dell'introduzione dell'interfaccia `Vehicle`:

- `Traveler` è ora accoppiato con un'interfaccia, non con una classe concreta.
- Tutte le implementazioni dell'interfaccia funzionano con `Traveler` senza modifiche al codice.
- Sarà possibile usare implementazioni di `Vehicle` non ancora realizzate oggi (anticipare il cambiamento).
- Serve un'entità esterna che faccia l'inject della reale implementazione dell'interfaccia.
- La complessità del codice aumenta.

Il ruolo delle interfacce

Le interfacce portano due benefici chiave, alla base del loro uso estensivo negli SDK come Java.

- **Standard e contratto**: un'interfaccia definisce un contratto che promuove il riuso; se un oggetto implementa un'interfaccia, promette di conformarsi a uno standard. L'oggetto che ne usa un altro è detto consumer, e l'interfaccia è un contratto tra un oggetto e il suo consumer.
- **Astrazione**: permettono di discutere il comportamento generale del codice senza entrare nei dettagli specifici, rendendo i programmi più facili da capire.

La legge di Demetra

Definition

Legge di Demetra

Linea guida chiave per il basso accoppiamento, nata nel progetto Demetra (uno dei primi grandi sistemi object-oriented, sviluppato all'Università di Boston): spinge al massimo l'information hiding, evitando catene di chiamate tipo `Foo.getA().getB().getC()`. Più è lungo il percorso di chiamate attraversato dal programma, più il programma è fragile ai cambiamenti.

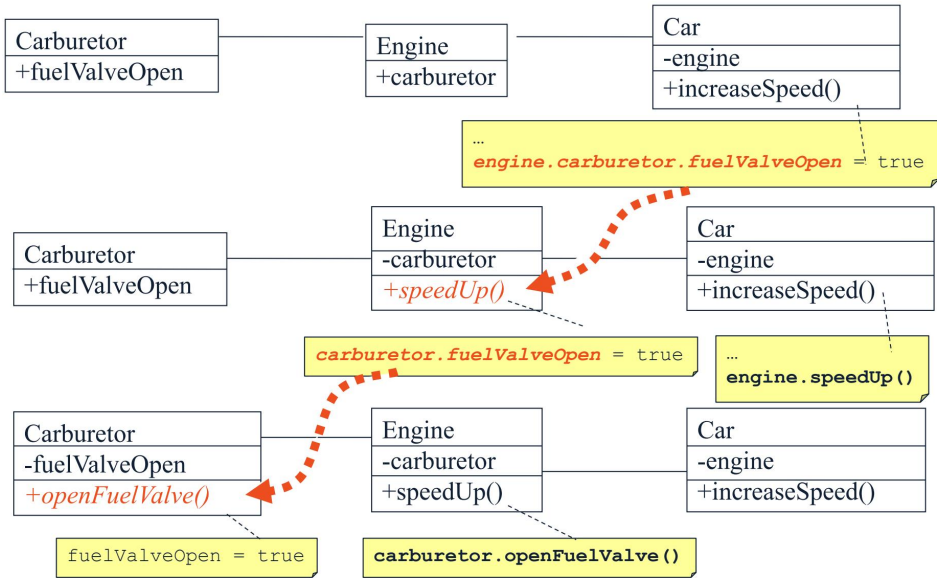
Un metodo `M` di un oggetto `O` dovrebbe mandare messaggi solo a:

- l'oggetto contenente (`this`);
- una variabile di istanza di `this`;
- un parametro del metodo;

- un oggetto creato all'interno del metodo;
- i propri oggetti componenti diretti.

Eliminare il navigation code

Applicare la legge porta a eliminare il navigation code, delegando l'operazione al componente che la possiede: invece di attraversare la catena `Car` → `Engine` → `Carburetor` per aprire la valvola del carburatore, il `Car` chiama `engine.speedUp()` e l'`Engine` chiama `carburetor.openFuelValve()`.



Il responsibility-driven design

Il **responsibility-driven design** (RDD) pensa alla progettazione in termini di classe, responsabilità e collaborazioni.

- Ciascuna classe dovrebbe avere responsabilità e ruoli ben precisi → alta coesione.
- La classe che possiede i dati dovrebbe essere responsabile di processarli → basso accoppiamento.
- Ciascuna classe dovrebbe essere responsabile di gestire i propri dati → incapsulamento.
- Il metodo è facilitato dall'uso di CRC cards.

Le CRC cards

Definition

CRC Card

Scheda da associare a ogni classe, introdotta da Kent Beck e Ward Cunningham per insegnare la progettazione object-oriented: riporta il nome della classe, le superclassi e sottoclassi, le responsabilità (conoscenza che la classe mantiene o servizi che fornisce) e i collaboratori (altre classi di cui sfruttare conoscenza o servizi per ottemperare alle responsabilità).

Le CRC cards spingono verso un design con responsabilità chiare e controllo sul numero di classi associate.

- Se non si riesce a specificare chiaramente le responsabilità di una classe, c'è un errore di design.
- Se una classe ha bisogno di troppe classi collaboratrici, c'è un potenziale errore di design.

Localizzare le modifiche

Il design RDD con CRC cards porta a una **localizzazione delle modifiche**: quando è necessario operare una modifica, dovrebbe essere coinvolto il minor numero possibile di classi. La qualità del proprio codice si osserva proprio quando sorge l'esigenza di fare modifiche.

Capitolo 6

Lezione 12-13 Standard Software Architectures

Architettura del software

Definition

Architettura del software

Struttura o strutture del sistema, comprendenti gli elementi software, le loro proprietà visibili esternamente e le relazioni tra loro. Le strutture esatte da considerare e i modi di rappresentarle variano secondo gli obiettivi ingegneristici.

La definizione ha diverse implicazioni.

- L'architettura è un'**astrazione** del sistema: definisce gli elementi e come interagiscono, sopprimendo le informazioni puramente locali (i dettagli privati degli elementi non sono architetturali).
- Le proprietà esternamente visibili sono le assunzioni che un elemento può fare su un altro: servizi forniti e richiesti, caratteristiche di performance, gestione dei fault, uso delle risorse.
- **Ogni sistema ha un'architettura**, anche nel caso più semplice di un singolo elemento; ma avere un'architettura è diverso dall'avere un'architettura nota a tutti (l'architettura vs. la specifica dell'architettura): se non la si sviluppa esplicitamente, la si ottiene comunque, e potrebbe non piacere.
- I disegni di box-and-line da soli **non sono** architetture, ma solo un punto di partenza: serve aggiungere specifiche e proprietà.
- I sistemi hanno molte strutture (**viste**): nessuna struttura singola può essere "l'architettura", e l'insieme delle strutture candidate non è fisso né prescritto, si scelgono quelle utili per analisi, comunicazione o comprensione.

Scopo della progettazione architetturale

L'architettura si definisce nella prima fase di System Design (progettazione architetturale) e ha come scopo primario la scomposizione del sistema in sottosistemi.

- La realizzazione di più componenti distinti è meno complessa della realizzazione di un sistema come monolite.
- Permette di parallelizzare lo sviluppo.
- Favorisce modificabilità, riusabilità, portabilità.

Definire un'architettura significa **mappare funzionalità su moduli** (es. modulo di interfaccia utente, modulo di accesso al database, modulo di gestione della sicurezza). Anche la definizione delle architetture deve seguire i concetti di **alta coesione** e **basso accoppiamento**: ogni sottosezione fornisce servizi altamente correlati, limitando il numero di moduli con cui è legata.

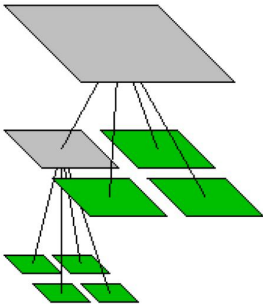
La definizione avviene da due punti di vista che portano alla soluzione finale:

- **identificazione e relazione dei sottosistemi**;
- **definizione delle politiche di controllo**.

Entrambe le scelte sono cruciali: è difficile modificarle quando lo sviluppo è partito, perché molte interfacce dei sottosistemi dovrebbero essere cambiate.

Divide and conquer

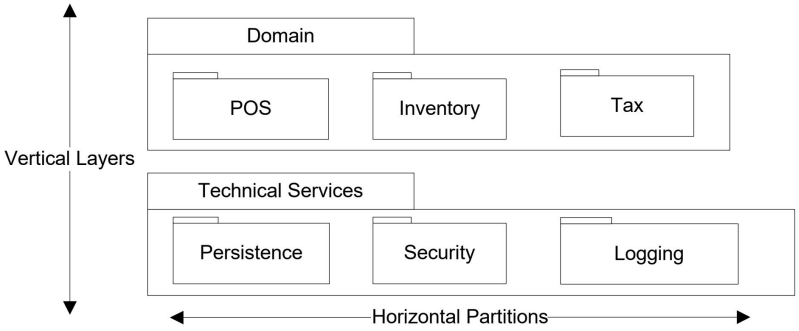
Un principio guida della decomposizione è **divide and conquer**: affrontare qualcosa di grande tutto insieme è più difficile che affrontare un insieme di cose più piccole.



- Ogni componente individuale è più piccolo e quindi più facile da capire.
- Le parti possono essere sostituite o modificate senza dover sostituire o cambiare estesamente le altre.
- Persone separate possono lavorare su parti separate, e il singolo ingegnere può specializzarsi.

Un sistema si divide gerarchicamente: **sistema** → **layer e sottosistemi** → **package** → **classi** → **metodi**. La decomposizione segue due dimensioni: quella funzionale (sottosistemi) e quella di astrazione/implementazione (layering).

Nell'architettura logica, layer e partizioni si combinano: i layer dividono il sistema lungo la dimensione dell'astrazione, le partizioni lungo la dimensione funzionale.



L'architettura a livelli

Definition

Layer

Raggruppamento di sottosistemi che forniscono servizi correlati, eventualmente realizzati usando servizi di altri layer. Una decomposizione gerarchica di un sistema consiste in un insieme ordinato di layer.

- Un layer può dipendere solo dai layer di livello più basso e non ha conoscenza dei layer superiori.
- **Architettura chiusa**: ogni layer può accedere solo al layer immediatamente sotto di esso; design goal: alta manutenibilità e portabilità.
- **Architettura aperta**: un layer può anche accedere ai layer di livello più basso; design goal: efficienza a runtime.

La macchina virtuale

La prima formalizzazione dell'architettura a layer è la **macchina virtuale** (Dijkstra, 1965): un sistema dovrebbe essere sviluppato come insieme di macchine virtuali, ognuna costruita in termini di quelle al di sotto di essa.

La layered architecture

Nella layered architecture i sottosistemi sono organizzati in layer: ogni layer usa i servizi del layer sottostante e fornisce servizi al layer superiore tramite interfacce ben definite. I termini **tier** e **layer** sono, per la maggior parte delle persone, intercambiabili. Bilancio di pro e contro:

- + astrazione delle preoccupazioni (abstraction of concerns);
- + isolamento dei livelli bassi e alti l'uno dall'altro (decoupled);
- + riusabilità;
- — overhead nell'attraversare i diversi layer;
- — complessità.

L'architettura client-server

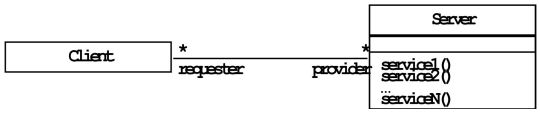
Definition

Client-server

Architettura distribuita dove dati ed elaborazione sono distribuiti su una rete di nodi di due tipi: i server, processori potenti e dedicati che offrono servizi specifici (stampa, gestione di file system, compilazione, gestione del traffico di rete, calcolo); i client, macchine meno prestazionali su cui girano le applicazioni utente che usano i servizi dei server.

Le regole di conoscenza sono asimmetriche.

- I client devono conoscere i nomi e la natura dei server; i server non devono conoscere identità e numero dei client.
- I client chiamano il server, che realizza alcuni servizi e restituisce il risultato; i client conoscono l'interfaccia del server, i server non conoscono le interfacce dei client.
- La risposta è in generale immediata; gli utenti interagiscono solo con il client.



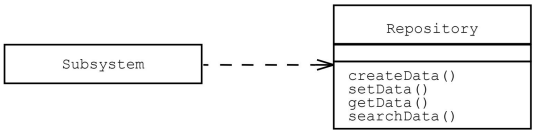
L'architettura a repository

Definition

Repository

Architettura in cui i sottosistemi accedono e modificano una singola struttura dati chiamata repository; i sottosistemi sono relativamente indipendenti, perché interagiscono solo attraverso il repository.

Il flusso di controllo è dettato o dal repository (un cambiamento nei dati memorizzati) o dai sottosistemi (flusso di controllo indipendente). Esempi tipici: i database management systems e i compilatori moderni.



Vantaggi del repository

- Modo efficiente di condividere grandi moli di dati: write once for all to read.
- Un sottosistema non deve preoccuparsi di come i dati sono prodotti o usati dagli altri sottosistemi.
- Gestione centralizzata di backup, sicurezza, controllo degli accessi e recovery da errori.
- Il modello di condivisione dei dati è pubblicato come repository schema: è facile aggiungere nuovi sottosistemi.

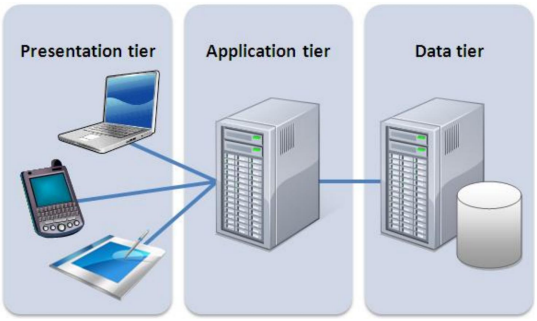
Svantaggi del repository

- I sottosistemi devono concordare su un modello dati di compromesso → performance ridotte.
- **Data evolution**: adottare un nuovo modello dati è difficile e costoso, perché deve essere applicato a tutto il repository e tutti i sottosistemi devono essere aggiornati.
- Diversi sottosistemi possono avere requisiti diversi su backup e sicurezza, non supportati.
- È difficile distribuire efficientemente il repository su più macchine continuando a vederlo come logicamente centralizzato: problemi di ridondanza e consistenza dei dati.

Le architetture three-tier

Le architetture three-tier sono state introdotte negli anni Novanta e trattano la business logic in modo esplicito, su tre livelli.

- **Livello 1 — gestione dei dati**: comunicazione verso DBMS, file XML, ecc.
- **Livello 2 — business logic**: processamento dei dati.
- **Livello 3 — interfaccia utente**: presentazione dei dati, servizi.



Ogni livello ha obiettivi e vincoli di design propri, e nessun livello fa assunzioni sulla struttura o sull'implementazione degli altri: il livello 2 non assume nulla sulla rappresentazione dei dati né sull'implementazione dell'interfaccia utente, e il livello 3 non assume nulla su come opera la business logic.

- **Non c'è comunicazione diretta tra livello 1 e livello 3**: l'interfaccia utente non riceve né inserisce direttamente dati nel livello di data management; tutti i passaggi di informazione nei due sensi vengono filtrati dalla business logic.
- I livelli operano senza assumere di essere parte di una specifica applicazione: le applicazioni sono viste come collezioni di componenti cooperanti, e ogni componente può essere contemporaneamente parte di applicazioni diverse.

Vantaggi delle three-tier

- **Flessibilità e modificabilità**: componenti utilizzabili in sistemi diversi; la modifica di una componente non impatta sul resto del sistema (a meno di cambiamenti nelle API); ricerca di bug più focalizzata per la separazione e l'isolamento delle funzionalità; l'aggiunta di funzionalità implica l'estensione delle sole componenti coinvolte o l'aggiunta di nuove componenti.
- **Interconnettività**: le API delle componenti superano il problema degli adattatori del modello client-server, così che interfacce diverse possano essere connesse allo stesso servizio; l'accesso a dati comuni da applicazioni diverse è facilitato.
- **Gestione di sistemi distribuiti**: la business logic di applicazioni distribuite (es. sistemi informativi con server replicati e client remoti) è aggiornabile senza richiedere l'aggiornamento dei client.

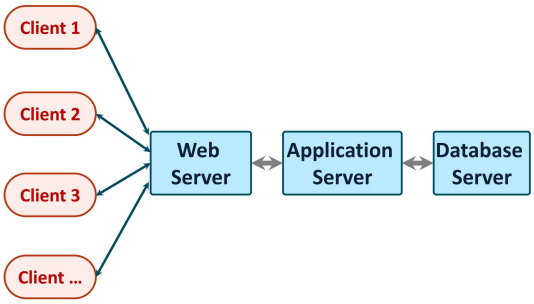
Svantaggi delle three-tier

- **Dimensioni delle applicazioni ed efficienza**: pesante uso della comunicazione in rete, che introduce latenza nel servizio; la comunicazione tra componenti richiede librerie per lo scambio di informazioni, con codice voluminoso.
- **Integrazione di legacy software**: molte imprese usano software vecchio basato sul modello monolitico; è difficile applicare il modello three-tier alle nuove applicazioni e servono adattatori per interfacciare il software legacy.

Le architetture n-tier

Le architetture **n-tier** sono l'evoluzione delle 3-tier su N livelli e permettono configurazioni diverse. Gli elementi fondamentali sono:

- **Interfaccia utente (UI)**: gestisce l'interazione con l'utente; può essere un web browser, una mobile app, un'interfaccia grafica.
- **Presentation logic**: definisce cosa la UI presenta e come gestire le richieste dell'utente.
- **Business logic**: gestisce le regole di business dell'applicazione.
- **Infrastructure services**: forniscono funzionalità supplementari alle componenti dell'applicazione (messaging, supporto alle transazioni).
- **Data layer**: il livello dei dati dell'applicazione.



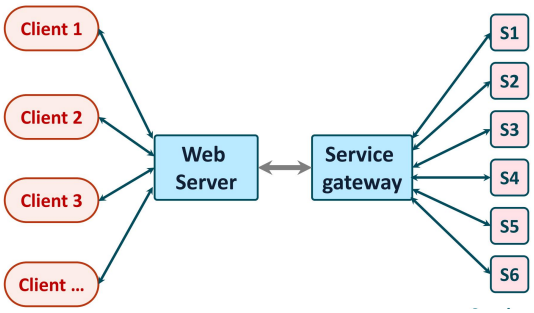
La Service-oriented Architecture

Definition

Service-oriented Architecture

Architettura in cui i servizi sono componenti stateless: possono essere replicati e migrare da un computer a un altro.

- Molti server possono essere coinvolti nell'erogazione dei servizi.
- Una SOA è in genere più facile da scalare quando la domanda cresce ed è resiliente ai guasti.



Gli stili architetturali

Nell'ingegneria del software sono stati definiti vari **stili architetturali** usabili come base per uno specifico sistema da sviluppare: Client/Server, Repository, Peer-to-Peer, Model/View/Controller (nelle sue declinazioni), Multi-tiered, Microservices.

Capitolo 7

Lezione 14 Virtualizzazione e Cloud

La virtualizzazione

Definition

Virtualizzazione

Tecnica per aumentare l'efficacia e l'efficienza di Centri Elaborazione Dati complessi: consiste nella creazione di una versione virtuale di una risorsa normalmente fornita fisicamente e appartenente a un sistema. Risalente agli anni Sessanta, è tornata in auge grazie all'evoluzione tecnologica delle CPU.

L'evoluzione delle CPU

I microprocessori hanno sempre avuto un rapidissimo incremento delle prestazioni e una diminuzione dei costi; la strategia storica per migliorarle è stata aumentare la "velocità" del processore.

- **1978**: Intel 8086, 4 MHz.
- **1985**: Intel 386, 25 MHz (~6x).
- **1995**: Intel Pentium Pro, 200 MHz (~8x).
- **2005**: Intel Pentium 4, 3,8 GHz (~19x).

Questa strategia si è arrestata nel 2005: processori recenti (es. Intel Core Ultra 9 285K, 2,5–4,6 GHz) girano su frequenze comparabili.

Le CPU multicore

Raggiunti i limiti fisici del silicio, la soluzione è stata aumentare il numero di unità di elaborazione (**multicore**) per aumentare la potenza di calcolo (es. AMD Ryzen 9 9955HX, 2,5–5,4 GHz, 32 unità di calcolo). Il problema: i programmi sono tradizionalmente scritti per un computer con una singola CPU e la stragrande maggioranza delle applicazioni è sequenziale, come sfruttare più unità in parallelo?

L'infrastruttura tradizionale di un CED

Il modello tradizionale di Centri Elaborazione Dati prevede singoli server che offrono singoli servizi o applicazioni (**one workload, one box**): un nuovo servizio o applicazione richiede un nuovo server, con proprio storage. Le conseguenze:

- **Sottoutilizzo delle risorse**: la maggior parte dei server sfrutta in media tra il 5% e il 15% della propria capacità computazionale; per i servizi critici serve ridondanza (computer di riserva normalmente non usati).
- **Aumento dei costi di installazione e gestione**: rilascio di aggiornamenti su scala nazionale molto costoso; backup da effettuare su ogni macchina.
- **Servizi intrinsecamente interconnessi**: es. il server di dominio, che gestisce le autenticazioni, è interrogato da tutti gli altri servizi.

Il fail di un server

In caso di guasto di un server si possono bloccare i servizi degli altri server in comunicazione con esso, con conseguenze pesanti.

- Interruzione dell'operatività collegata a quel servizio; nei casi più gravi, interruzione dell'intera operatività aziendale.
- Aumento dei costi: costi di fermo e di ripristino.
- Tempi di fermo non sempre prevedibili.

Le macchine virtuali

Un server fisico consiste di risorse hardware, un sistema operativo e uno o più software applicativi. L'idea della virtualizzazione: se la CPU del server ha più unità di calcolo parallele, si può "simulare" l'esistenza di più computer, indipendenti dal vero hardware sottostante.

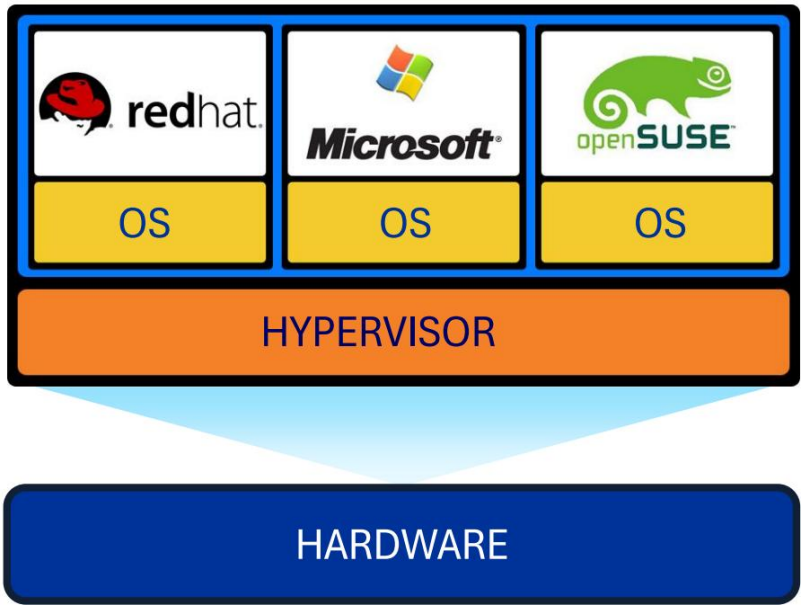
L'hypervisor

Definition

Hypervisor

Strato software che parte prima del sistema operativo e simula l'esistenza di più computer virtuali su un server fisico: offre le risorse fisiche ai computer virtualizzati e le gestisce in modo concorrente a favore delle macchine virtuali.

Su ogni computer virtualizzato si installa un normalissimo sistema operativo, che non ha modo di sapere se sta funzionando su un computer reale o virtuale.



Le proprietà delle macchine virtuali

- **Indipendenza dall'hardware**: l'hypervisor simula una configurazione standard di un server, nascondendo il vero hardware fisico; le VM possono funzionare su server fisici diversi senza bisogno di essere configurate.
- **Memorizzazione**: una VM è un file, contenente sistema operativo, applicativi, dati e rappresentazione della memoria del server; un file memorizza lo stato di un server virtuale e ne permette la copia su un numero arbitrario di server fisici. Esistono strumenti commerciali che trasformano il contenuto di un server fisico in una VM.
- **Isolamento**: ogni VM è totalmente isolata dalle altre VM in esecuzione sullo stesso server fisico; in caso di attacco o crash di una VM, le altre non sono intaccate.
- **Gestione centralizzata**: tutte le VM di un CED possono essere gestite attraverso un'unica finestra, semplificando notevolmente la gestione dell'intero centro.

La virtualizzazione enterprise ad alta disponibilità

Un CED enterprise deve disporre di almeno **due server fisici** e di un'unità di dischi rigidi (**SAN**, Storage Area Network), collegati tra loro ad altissima velocità (fibra ottica). Nella virtualizzazione enterprise le macchine virtuali risiedono nello storage SAN.

La virtualizzazione garantisce la **business continuity** tramite funzioni dedicate.

- **HA (High Availability, alta disponibilità)**: se un server ha un guasto, le macchine virtuali presenti sono istantaneamente migrate sugli altri server disponibili e fatte ripartire, minimizzando il downtime al tempo di restart delle VM.

- **FT (Fault Tolerant)**: garantisce la continuità di funzionamento delle VM dopo il guasto di un server, azzerando il tempo di fermo; è al momento quanto di meglio si possa avere per la business continuity.
- **Live migration**: la migrazione "a caldo" delle VM tra i server fisici consente operazioni di manutenzione o di bilanciamento del carico mantenendo l'operatività dei server virtuali.

I benefici della virtualizzazione

Con l'introduzione della virtualizzazione si parla di **consolidamento** di un CED: sfruttando le risorse di computazione parallela dei nuovi processori, si riduce significativamente il numero di server necessari per erogare i servizi richiesti da un ente. Gli altri benefici:

- **Ottimizzazione delle risorse HW**: esecuzione di distinti sistemi operativi contemporaneamente in una singola macchina.
- **Ottimizzazione dello spazio disco**: a ogni VM viene assegnato lo spazio strettamente necessario, e il resto del volume resta disponibile per nuove macchine virtuali.
- **Semplificazione**: backup e disaster recovery (l'intero server, SO, applicazioni, dati, dispositivi e stato, non è altro che un file); indipendenza dall'hardware; rilasci semplificati; possibilità di clonare "live" le VM per test e aggiornamenti senza compromettere l'integrità dell'originale.
- **Riduzione dei costi**: meno server significa risparmio sui costi energetici, di condizionamento e sugli spazi fisici richiesti dal CED; la gestione centralizzata attraverso un'unica interfaccia di amministrazione incrementa la produttività del reparto IT.

Il cloud computing

Definition

Cloud computing

Modello in cui le risorse IT sono fornite come servizio: calcolo, storage, database, code. Sfrutta le economie di scala dell'hardware commodity (storage economico, reti ad alta banda, processori multicore) e data center geograficamente distribuiti; offerte da Microsoft, Amazon, Google e altri.

Scalabilità ed elasticità

Le due proprietà chiave del cloud sono strettamente correlate.

Definition

Scalabilità

Proprietà desiderabile di un sistema, una rete o un processo, che indica la capacità di gestire quantità crescenti di lavoro in modo graduale o di essere ingrandito prontamente.

La scalabilità statica non basta: le risorse devono seguire la domanda in tempo reale.

Definition

Elasticità

Capacità di far crescere o ridurre dinamicamente le risorse dell'infrastruttura in base al bisogno, adattandosi in modo autonomo ai cambiamenti del carico di lavoro e massimizzando l'uso delle risorse.

Queste proprietà si ottengono con il **dynamic provisioning** e il **multi-tenant design**.

Il dynamic provisioning

Definition

Dynamic provisioning

Modalità di un ambiente complesso di server in rete in cui le istanze di calcolo vengono provisionate o deployate da una console amministrativa o da un'applicazione client, da parte dell'amministratore di server o di rete o di qualsiasi altro utente abilitato.

Nel modello di calcolo tradizionale ci sono due problemi comuni: la sottostima dell'utilizzo del sistema, che porta a un under-provisioning, e la sovrastima, che porta a un basso utilizzo. La soluzione è provisionare dinamicamente le risorse del cloud, per:

- far fronte alle variazioni stagionali della domanda;
- far fronte alle variazioni di domanda tra settori diversi;
- gestire il burst demand di eventi straordinari.

I service model

I servizi cloud si organizzano in tre modelli, che si distinguono per quanta infrastruttura è gestita dal provider e quanta resta sotto il controllo del consumatore.

Definition

Infrastructure as a Service (IaaS)

Modello in cui il provider offre elaborazione, storage, reti e altre risorse di calcolo fondamentali, e il consumatore può deployare ed eseguire software arbitrario, inclusi sistemi operativi e applicazioni. Il consumatore non gestisce né controlla l'infrastruttura cloud sottostante, ma ha controllo su sistemi operativi, storage, applicazioni deployate ed eventuale controllo limitato di alcune componenti di rete.

Un gradino più in alto, il provider fornisce anche la piattaforma di sviluppo ed esecuzione.

Definition

Platform as a Service (PaaS)

Modello in cui al consumatore è data la capacità di deployare sull'infrastruttura cloud applicazioni da lui create o acquisite, usando librerie e strumenti supportati dal provider. Il consumatore non gestisce l'infrastruttura sottostante (rete, server, sistemi operativi, storage), ma ha controllo sulle applicazioni deployate ed eventualmente sulle configurazioni dell'ambiente di hosting.

Al gradino più alto, il consumatore usa direttamente applicazioni già pronte.

Definition

Software as a Service (SaaS)

Modello in cui al consumatore è data la capacità di usare le applicazioni del provider in esecuzione su un'infrastruttura cloud, accessibili da vari dispositivi client tramite thin client come un browser web (es. email web-based). Il consumatore non gestisce l'infrastruttura né le singole capacità applicative, salvo limitate impostazioni di configurazione user-specific.

Esempi tipici: per IaaS, Amazon EC2, Azure Compute, OpenNebula; per PaaS, Microsoft Azure SQL Database, Amazon AWS Lambda, Hadoop; per SaaS, Google Apps (Gmail, Google Docs, Google Sites) e Amazon AWS QuickSight.

I deployment model

I modelli di deployment primari del cloud sono tre, public, private e hybrid, che insieme al community cloud compongono l'ecosistema cloud. Le loro differenze stanno principalmente nello scope e nell'accesso dei servizi cloud pubblicati verso i consumatori.

Definition

Public cloud

Infrastruttura cloud resa disponibile al grande pubblico o a un grande gruppo industriale e di proprietà di un'organizzazione che vende servizi cloud; nota anche come cloud esterno o multi-tenant cloud, rappresenta un ambiente apertamente accessibile.

Caratteristiche di base: infrastruttura omogenea, politiche comuni, risorse condivise e multi-tenant, infrastruttura affittata, economie di scala.

Definition

Private cloud

Infrastruttura cloud gestita esclusivamente per un'organizzazione; può essere gestita dall'organizzazione o da terzi e risiedere on premise o off premise. Nota anche come cloud interno o on-premise cloud, limita intenzionalmente l'accesso alle risorse ai consumatori della stessa organizzazione proprietaria.

Caratteristiche di base: infrastruttura heterogenea, politiche personalizzate, risorse dedicate, infrastruttura interna, controllo end-to-end.

Public vs. private

	Public Cloud	Private Cloud
Infrastruttura	Omogenea	Heterogenea
Modello di policy	Comuni e definite	Personalizzate

	Public Cloud	Private Cloud
Modello di risorse	Condivise e multi-tenant	Dedicate
Modello di costi	Operational expenditure	Capital expenditure
Modello economico	Grandi economie di scala	Controllo end-to-end

Definition

Hybrid cloud

Composizione di due o più cloud (private, community o public) che restano entità uniche ma sono legate insieme da tecnologie standard o proprietarie che abilitano la portabilità di dati e applicazioni (es. cloud bursting per il bilanciamento del carico tra cloud).

Capitolo 8

Lezione 15-16 Modern Software Architectures

Il ciclo di vita di sviluppo del software

Lo sviluppo di un sistema software articolato attraversa fasi successive.

1. **Ingegneria dei requisiti**: i requisiti si raccolgono tramite interviste con gli stakeholder, *personas*, storie e scenari; si specificano con casi d'uso e linguaggio naturale.
2. **Progettazione del sistema e della UI**: i requisiti vengono allocati ai sottosistemi software e i sottosistemi alle risorse hardware (pattern architetturali); si definiscono poi gli oggetti necessari a realizzare ogni sottosistema (design pattern, ingegneria dell'usabilità, *wireframe* ad alta fedeltà).
3. **Implementazione**: ogni sottosistema viene realizzato con codice sorgente e altri artefatti, curando *clean code*, framework, ORM e qualità del software.
4. **Testing**: si verifica che il software soddisfi i clienti, tramite modelli di dominio, ispezioni del codice, testing funzionale (unitario, di integrazione, di sistema) e test di usabilità.
5. **Esercizio e manutenzione**: il sistema entra in uso pratico; la manutenzione serve a correggere errori non scoperti nelle fasi precedenti e ad adattare il software a cambiamenti nei requisiti o nel suo ambiente.

Architettura del sistema software

Definition

Architettura del sistema software

Insieme delle decisioni di progetto principali assunte sul sistema: un progetto di riferimento (*blueprint*) per la sua costruzione ed evoluzione. Queste decisioni hanno un impatto significativo su struttura, comportamento, interazioni e proprietà non funzionali del sistema.

«Principale» implica un grado di importanza superiore, che riconosce a una decisione di progetto uno «stato architetturale»; non esiste però un modo oggettivo per stabilire quali decisioni siano principali.

> *«Architecture is about the important stuff. Whatever that is».*

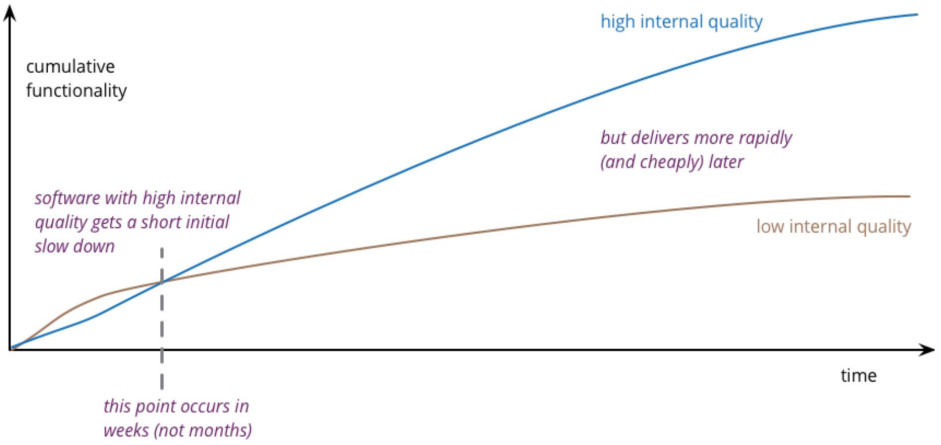
Pensare da architetti significa decidere cosa è importante e spendere energia per tenere quegli elementi in buono stato. Per diventare architetto, uno sviluppatore deve saper riconoscere gli elementi importanti, cioè quelli che, se non controllati, produrrebbero problemi seri.

L'architettura riguarda ciò che è difficile cambiare in seguito: scegliere tra MariaDB e MySQL non è una decisione architetturale, mentre scegliere tra un database relazionale e uno NoSQL lo è.

Perché l'architettura è importante

L'architettura è un tema ostico per clienti e utenti perché non è direttamente percepibile; ha però un impatto significativo su struttura, comportamento, interazioni e proprietà non funzionali del sistema.

Una scarsa architettura produce sistemi molto più difficili da realizzare ed evolvere, con impatti su costi e consegna di nuove funzionalità. L'**alta qualità interna** costa di più all'inizio, ma ripaga nel lungo periodo.



Proprietà influenzate dall'architettura

Le proprietà non funzionali influenzate dall'architettura sono **scalabilità**, **disponibilità**, **consistenza**, **performance**, **latenza** e **throughput**.

Scalabilità

Definition

Scalabilità

Proprietà di un servizio le cui prestazioni crescono in modo proporzionale alle risorse aggiunte.

Aumentare le prestazioni significa in genere servire più unità di lavoro, ma anche saper gestire unità di lavoro più grandi, come nella crescita dei dataset.

Info

Performance e scalabilità

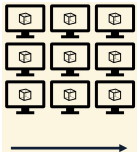
- **Problema di performance**: il sistema è lento già per un singolo utente.
- **Problema di scalabilità**: il sistema è veloce per un singolo utente, ma lento sotto carico pesante.

La scalabilità è anche una questione di denaro: a parità di risorse pagate, cresce il volume servito (richieste gestite, transazioni, utenti concorrenti…).

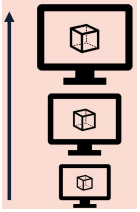
Orizzontale, verticale e diagonale

Esistono tre strategie per servire più unità di lavoro.

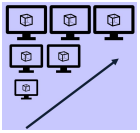
- **Scalabilità orizzontale**: aumenta il numero di nodi di calcolo.



- **Scalabilità verticale**: aumenta la potenza di calcolo del nodo.



- **Scalabilità diagonale**: aumenta insieme numero e potenza dei nodi.



Load balancing

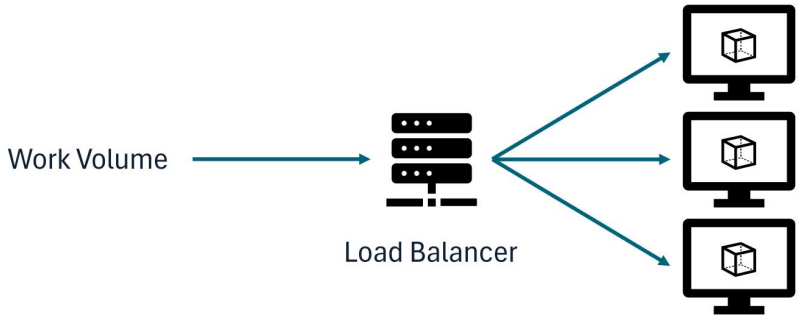
Scalando orizzontalmente (o diagonalmente) è necessario un componente dedicato che distribuisca le richieste in arrivo ai nodi di calcolo.

Definition

Load Balancer

Componente che distribuisce le richieste in arrivo ai nodi di calcolo quando si scala orizzontalmente o diagonalmente.

Il Load Balancer instrada il traffico secondo strategie diverse: **random** (traffico diretto a un nodo casuale), **least loaded** (al nodo meno caricato), **round robin**, **Layer 4** (guarda i dettagli del livello di trasporto, es. indirizzo IP sorgente), **Layer 7** (guarda i dettagli del livello applicativo, es. cookie e sessioni).



Warning

Limiti del load balancer

Un load balancer può diventare un collo di bottiglia per le prestazioni, e un singolo load balancer è un singolo punto di guasto (*single point of failure*).

Disponibilità

Definition

Disponibilità

Proporzione di tempo in cui un sistema o servizio è operativo e accessibile all'uso.

La disponibilità è cruciale per i sistemi la cui operatività continua è vitale: i sistemi *business-critical* (perdite finanziarie, danno reputazionale…) e quelli *safety-critical*.

Le cause di indisponibilità possono essere: problemi software (l'applicazione smette di rispondere), carichi eccessivi (risorse computazionali insufficienti), perdita della connessione di rete, guasti hardware.

Quantificazione

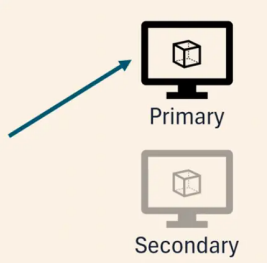
La disponibilità si quantifica come percentuale di tempo di attività (*uptime*) ed è misurata in numero di 9: un servizio con disponibilità pari al 99,99% si dice avere «quattro 9».

Numero di 9	Disponibilità	Downtime accettabile in un anno
Uno	90%	circa 36 giorni
Due	99%	circa 3 giorni e 15 ore
Tre	99,9%	circa 8 ore e 41 minuti
Quattro	99,99%	circa 52 minuti
Cinque	99,999%	circa 5 minuti
Sei	99,9999%	circa 31 secondi

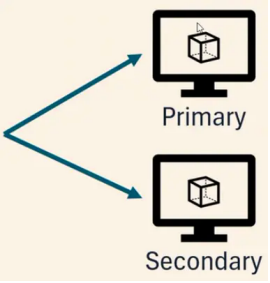
Fail-over

L'alta disponibilità si persegue con meccanismi di *fail-over*, in due varianti.

- **Fail-over attivo-passivo**: al guasto di un componente primario, un'istanza secondaria ne prende il lavoro; la secondaria è generalmente passiva e non fa alcun lavoro finché la primaria non fallisce.



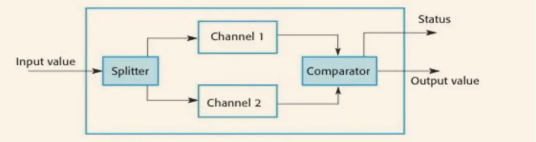
- **Fail-over attivo-attivo**: due o più istanze del componente esistono da subito e si dividono il lavoro (come nello scaling orizzontale); al guasto di una, le altre ne raccolgono anche la sua parte.



Example

Fault Tollerance

Software e dei sistemi per garantire la tolleranza ai guasti



Osservabilità e monitoraggio

Con i meccanismi di fail-over, il downtime dipende da diversi fattori.

- Quanto in fretta si nota il guasto di un componente? Tipicamente lo si rileva tramite **heartbeat**: segnali o controlli periodici che verificano il funzionamento normale del sistema.
- Quanto tempo impiega l'istanza secondaria a partire? La secondaria può essere in standby «caldo» (*hot*) o «freddo» (*cold*); nel fail-over attivo-attivo le istanze secondarie sono già in esecuzione.

La **osservabilità**, il grado in cui si può capire cosa accade in un sistema software, è una caratteristica importante dell'architettura; per il monitoraggio si usano spesso soluzioni come OpenTelemetry.

Dati e consistenza

Scalare orizzontalmente le istanze applicative è relativamente facile: basta un nuovo server con l'applicazione e un load balancer. La maggior parte delle applicazioni richiede però anche persistenza dei dati, e scalare i dati in un database relazionale non è altrettanto facile.

Proprietà ACID

Definition

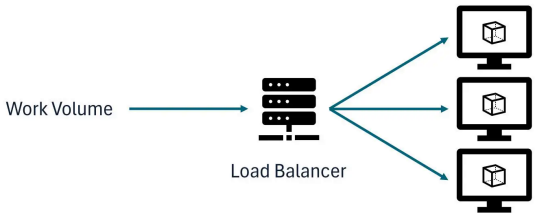
ACID

Insieme di proprietà delle transazioni nei database relazionali.

- **Atomicità**: ogni transazione è tutto o niente.
- **Consistenza**: ogni transazione porta il database da uno stato valido a un altro.
- **Isolamento**: l'esecuzione concorrente di transazioni produce gli stessi risultati dell'esecuzione in serie.
- **Durabilità**: una volta confermata, una transazione rimane tale.

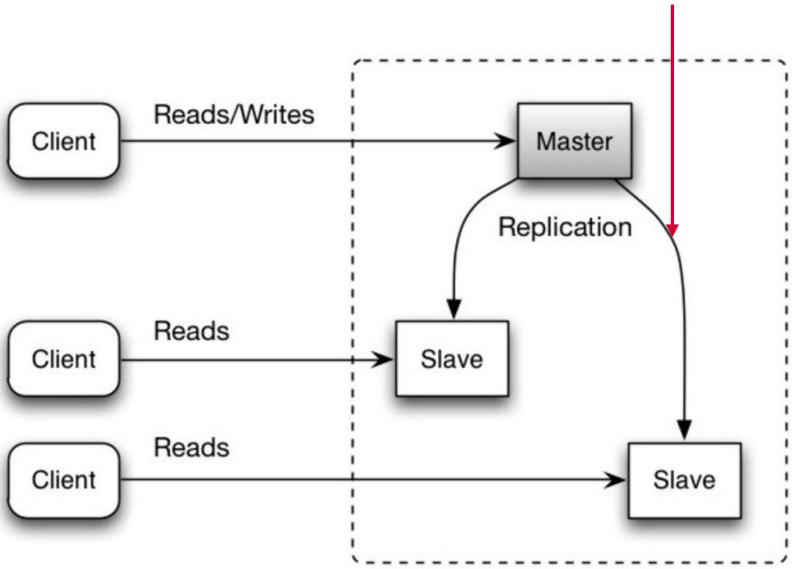
Scalare un RDBMS

Le proprietà ACID rendono impossibile scalare orizzontalmente un RDBMS in modo semplice. Le strade percorribili sono lo scaling verticale (più risorse di calcolo sullo stesso nodo) e tecniche specifiche di replica e partizionamento.



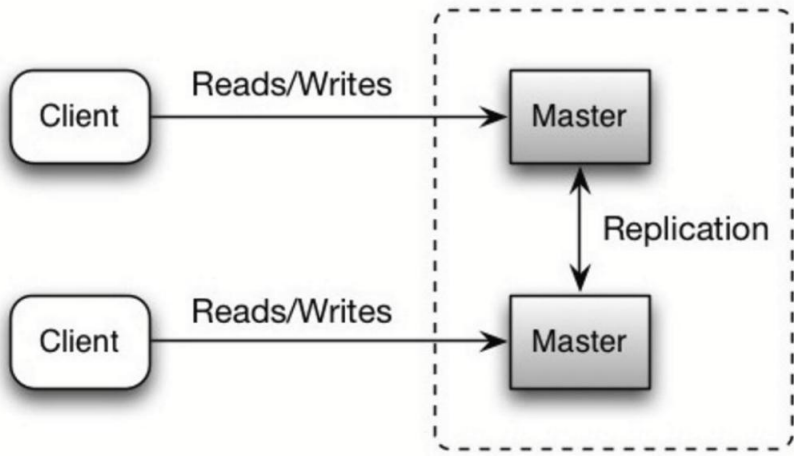
Master-Slave

Più istanze dell'RDBMS: un **master** e uno o più **slave**. Il master serve letture e scritture, replicando ogni scrittura su ciascuno slave; gli slave agiscono da repliche in sola lettura. Se il master fallisce, il sistema può continuare a operare in sola lettura (uno slave può eventualmente essere promosso). L'approccio funziona bene con molte letture e poche scritture; la replica introduce un *overhead*.



Master-Master

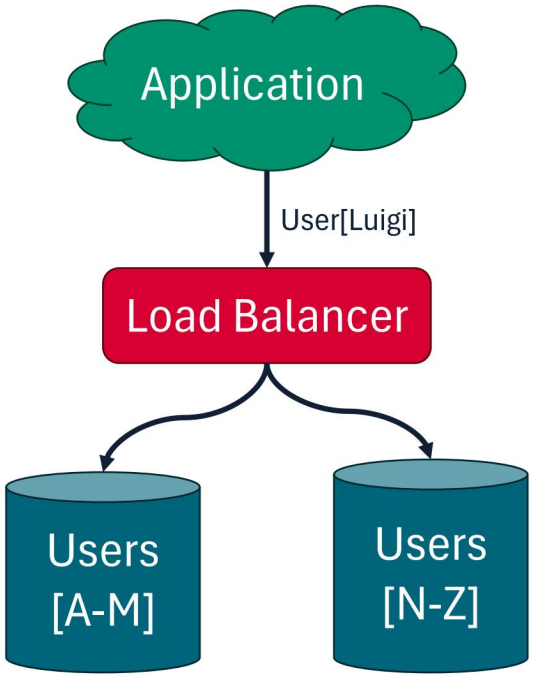
Tutte le istanze sono master e servono sia letture che scritture: per ogni scrittura i master si coordinano e sincronizzano, mentre le letture sono gestite indipendentemente. Se un master fallisce, il sistema continua a operare con gli altri; la sincronizzazione introduce un *overhead*.



Federation

Un singolo grande database può essere suddiviso in più database per funzione (es. un database prodotti, uno utenti, uno ordini). Presenta alcuni limiti: non è efficace se lo schema include una tabella molto grande; rende più difficili i join; rende più difficile imporre vincoli tra tabelle diverse (es. chiavi esterne); può richiedere denormalizzazione.

Sharding



Lo **sharding** distribuisce i dati su più database in modo che ciascuno gestisca solo un sottoinsieme dei dati. Conseguenze: la logica applicativa diventa più complessa per gestire gli shard; gli shard possono diventare sbilanciati; i join sono difficili; imporre vincoli è più difficile.

Denormalizzazione

La **denormalizzazione** cerca di migliorare le prestazioni in lettura a spese di parte delle prestazioni in scrittura e della perdita di normalizzazione: si scrivono copie ridondanti dei dati in più tabelle per evitare join costosi. Mantenere i dati consistenti può essere un problema; alcuni RDBMS come PostgreSQL supportano le **viste materializzate**, che si occupano di memorizzare le informazioni ridondanti e di mantenere coerenti le copie.

Database NoSQL

I modelli di database NoSQL sono più scalabili, al costo di rinunciare alle vere transazioni ACID (e ad alcune prestazioni di interrogazione). Le loro proprietà si descrivono con l'acronimo BASE.

Definition

BASE

Acronimo delle proprietà dei database NoSQL.

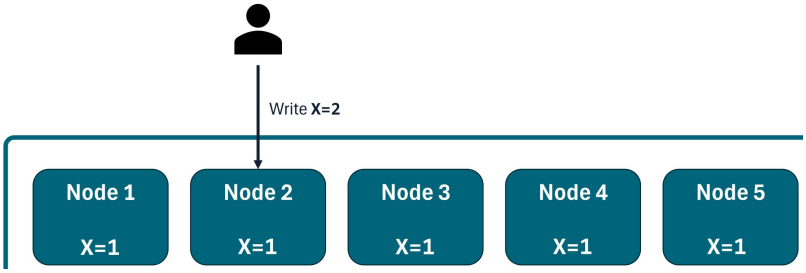
- **Basically Available**: il sistema garantisce la disponibilità.
- **Soft state**: lo stato del sistema può cambiare nel tempo, anche senza input.
- **Eventual consistency**: il sistema diventa consistente dopo un periodo di tempo, purché non riceva input durante quel periodo.

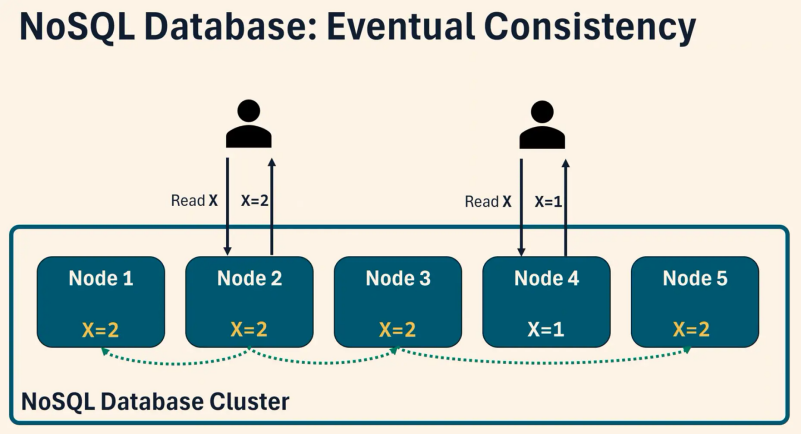
Definition

Consistenza finale

Garanzia informale per cui, se non vengono effettuati nuovi aggiornamenti a un dato, prima o poi tutti gli accessi a quel dato restituiranno l'ultimo valore aggiornato.

In un cluster NoSQL un aggiornamento si propaga progressivamente ai vari nodi, che per un certo tempo possono restituire valori diversi.





Esistono diversi modelli di database NoSQL.

- **Key-value store** (Redis, Memcached): tipicamente operano in memoria.
- **Document store** (MongoDB): centrati su documenti (XML, JSON…) che contengono tutte le informazioni di un dato oggetto.
- **Graph database** (Neo4J): ogni nodo è un record e le relazioni tra record sono rappresentate come archi.
- **Vector database** (Milvus): specializzati nell'archiviazione e nel reperimento di grandi vettori, spesso provenienti da applicazioni di AI.

Elaborazione asincrona

Necessità dell'asincronismo

Con sistemi sempre più grandi, a un certo punto può servire un'elaborazione che non può essere eseguita in linea durante la gestione di una richiesta, perché creerebbe una latenza inaccettabile. L'asincronismo aiuta a ridurre i tempi di risposta.

Message queue

Definition

Message Queue

Componente progettato per ricevere, trattenere e consegnare messaggi.

Quando un task è troppo lento per essere eseguito in linea (sincronamente), il flusso asincrono è il seguente: l'applicazione pubblica sulla MQ un messaggio con i dettagli del task da eseguire; un **worker**, in modo asincrono e possibilmente più tardi, agisce da consumatore, preleva il messaggio dalla MQ ed esegue davvero il task. L'utente non resta bloccato in attesa: il task è completato in background quando uno dei worker (eventualmente molti) diventa disponibile.

Message broker

Definition

Message Broker

Componente che fa da intermediario tra produttori e consumatori, instradando i messaggi verso le code.

Popolari message broker open-source sono RabbitMQ, Apache ActiveMQ e BlazingMQ.

Info

I broker non sono oggetti banali

- Sono tipicamente organizzati internamente in architetture distribuite complesse.
- Le strategie di instradamento dei messaggi possono essere più complesse del semplice polling da parte dei consumatori.
- Broker e configurazioni diversi garantiscono proprietà diverse: la consegna può essere garantita o meno, e un messaggio può essere consegnato a più di un consumatore.
- Questi aspetti vanno valutati con attenzione rispetto al problema da risolvere.

Code e stream

Alcuni message broker (es. RabbitMQ) supportano anche gli **stream**, che svolgono lo stesso compito delle code, accumulare messaggi dai produttori, letti in seguito dai consumatori, ma differiscono in due modi importanti.

- **Come i messaggi sono memorizzati**: gli stream modellano un log in sola aggiunta (*append-only*) che può essere riletto ripetutamente finché i messaggi non scadono; sono sempre persistenti e replicati, con semantica di consumo non distruttiva.
- **Come i messaggi sono consumati**: lo stesso messaggio può essere letto da più consumatori, quante volte vogliono, e i consumatori possono collegarsi in qualsiasi punto dello stream.

Tre casi d'uso tipici degli stream sono i seguenti.

- **Grandi fan-out**: per consegnare lo stesso messaggio a più sottoscrittori servirebbe una MQ dedicata per ciascun consumatore (potenzialmente inefficiente); gli stream permettono a un numero arbitrario di consumatori di leggere gli stessi messaggi dalla stessa coda in modo non distruttivo, eliminando la necessità di più code.
- **Grandi backlog**: le MQ sono progettate e ottimizzate per convergere verso lo stato vuoto, e le prestazioni degradano con milioni di messaggi in coda; gli stream sono progettati per memorizzare quantità maggiori di dati.
- **Replay**: gli stream permettono ai consumatori di rileggere i messaggi.

Microservizi

Architettura monolitica

Definition

Architettura monolitica

Stile architetturale in cui componenti e moduli diversi sono strettamente impacchettati in un'unica unità: singola codebase, singola unità di deployment autosufficiente, business concern accoppiati.

All'inizio, o per software a bassa complessità, il monolite è vantaggioso: i deployment sono facili (si costruisce un solo artefatto) e lo sviluppo è facile (singola codebase, gestione dei dati semplice).

Quando la complessità cresce, il monolite diventa problematico.

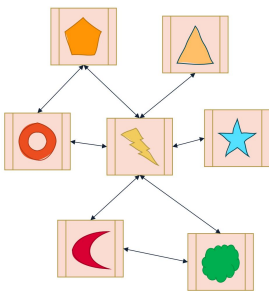
- L'architettura interna è sempre più difficile da mantenere ed evolvere.
- Le nuove release richiedono tempi sempre più lunghi (mesi).
- Aggiungere nuove funzionalità richiede molto tempo.
- Il ciclo di build, test e release diventa lungo e sempre più complesso.
- I deployment diventano sempre più difficili; è problematico capire chi ha «rotto la build» o chi è il proprietario del modulo fallito.
- Si innova con più difficoltà.

Architettura a microservizi

Definition

Architettura a microservizi

Stile architetturale in cui il software è strutturato come un insieme di servizi: ciascun servizio ha la propria codebase (eventualmente poliglotta), è deployabile in modo indipendente ed è organizzato attorno alle capability di business; i servizi sono debolmente accoppiati.



Motivazioni

Il mondo odierno è volatile e cambia rapidamente: le aziende devono essere agili e innovare più in fretta, e il software deve essere consegnato rapidamente, frequentemente e in modo affidabile. Le metriche DORA misurano questa capacità di consegna.

Metrica DORA	Significato
Deployment Frequency	Frequenza dei rilasci in produzione
Lead Time for Changes	Tempo necessario perché un commit arrivi in produzione
Changes Failure Rate	Percentuale di deployment che falliscono in produzione
Time to Restore Service	Tempo per ripristinare il servizio dopo un guasto in produzione

Suddividere il monolite in più microservizi aiuta a rendere gestibili sistemi più grandi.

- Ogni microservizio è significativamente più piccolo dell'intero sistema: più facile da capire, mantenere, evolvere, testare.
- Ogni microservizio può essere costruito e deployato in modo indipendente.
- Le nuove funzionalità dovrebbero interessare probabilmente un solo microservizio.
- I deployment sono più frequenti: per aggiungere una funzionalità si ridistribuisce un solo microservizio.

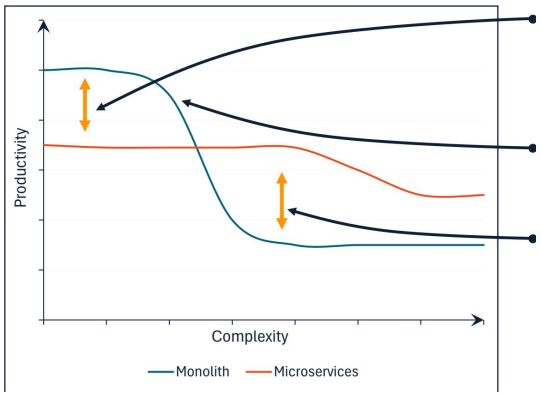
Scalare monoliti e microservizi

Le due architetture scalano in modo diverso: i monoliti mettono tutte le funzionalità in un singolo processo e scalano replicando il monolite su più server; i microservizi mettono ogni funzionalità in un processo distinto e scalano distribuendo i servizi su server diversi, replicandoli secondo necessità.

Quando adottare i microservizi

I microservizi non vanno nemmeno presi in considerazione se il sistema non è complesso. La complessità che spinge verso i microservizi può derivare da: dimensioni considerevoli (grandi team di sviluppo), multi-tenancy, necessità di supportare diversi modelli di interazione utente, necessità di scalare.

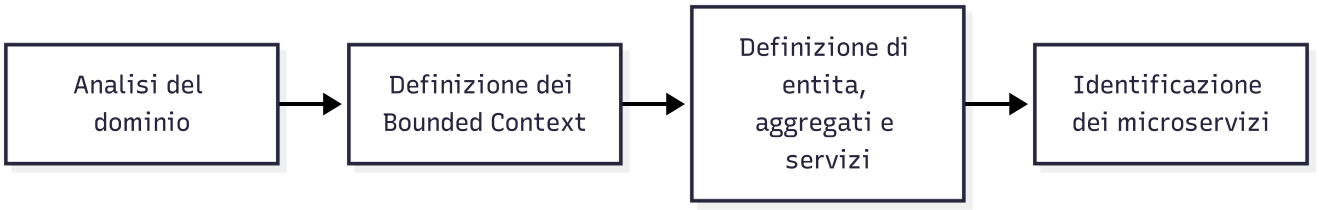
Riguardo alla produttività: per sistemi meno complessi l'overhead dei microservizi la riduce; quando la complessità cresce, la produttività del monolite crolla; il minore accoppiamento dei microservizi riduce l'attenuazione della produttività.



L'adozione richiede di affrontare quattro punti chiave: come organizzare i team di sviluppo, come progettare i microservizi, come gestire la comunicazione tra servizi e come gestire i dati.

Progettazione con il Domain-Driven Design

Definire i confini di ciascun servizio è una delle sfide maggiori: la regola generale è che un servizio deve fare «una sola cosa», e nessun processo meccanico garantisce il risultato giusto. Viene in aiuto il **Domain-Driven Development** (DDD), articolato in quattro passi.



- **Analisi del dominio**: comprendere i requisiti funzionali; l'output è una definizione informale del dominio del sistema, raffinabile in un insieme di modelli di dominio.
- **Definizione dei Bounded Context**: ciascuno contiene un modello di dominio che rappresenta un particolare sottodominio; l'unificazione totale del dominio potrebbe non essere fattibile né conveniente.
- **Definizione di entità, aggregati e servizi**: per ogni Bounded Context si applicano i pattern DDD.
- **Identificazione dei microservizi**: si parte da un Bounded Context; la funzionalità di un microservizio non dovrebbe estendersi su più context.

I pattern DDD definiscono tre costrutti per ogni Bounded Context.

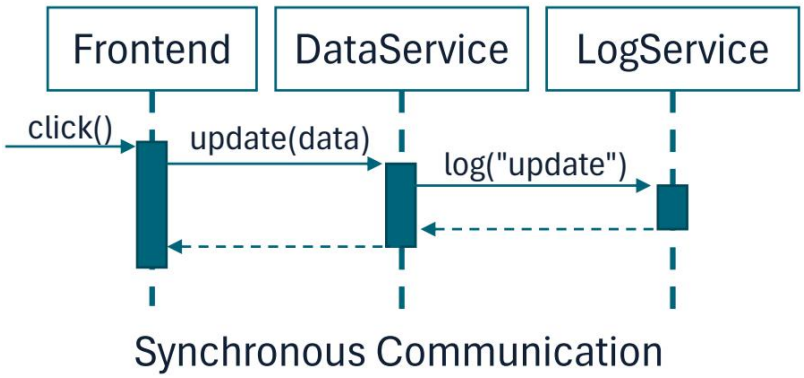
- **Entità**: oggetti con un'«identità» che persiste nel tempo.
- **Aggregati**: confini di consistenza attorno a una o più entità; modellano gli invarianti transazionali.
- **Servizi di dominio**: oggetti che implementano logica senza possedere alcuno stato.

Aggregati e servizi di dominio sono buoni candidati per diventare microservizi.

Comunicazione tra servizi

Nei monoliti la comunicazione tra componenti avviene tramite invocazioni di metodi; in un'architettura a microservizi deve essere **agnostica rispetto alla tecnologia**, e spesso si sfruttano i protocolli su cui è costruito il World Wide Web.

- **Sincrona**: REST (API su risorse HTTP); Apache Thrift, che supporta più protocolli (binari, basati su JSON…).
- **Asincrona**: message queue.



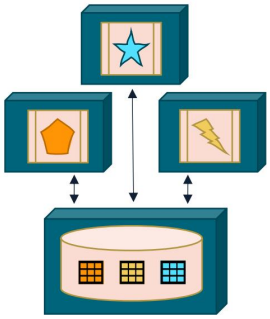
Warning

Effetto moltiplicativo del downtime

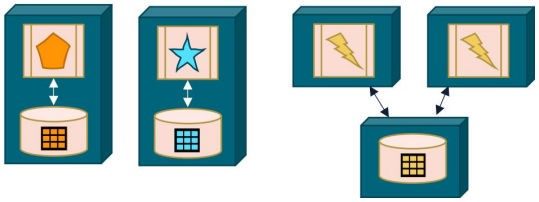
Le chiamate sincrone ostacolano le prestazioni: in presenza di molte chiamate sincrone, il downtime del sistema diventa il prodotto dei downtime dei singoli servizi. Occorre rendere asincrone le comunicazioni, oppure gestire i downtime.

Gestione dei dati

I monoliti sono tipicamente associati a un singolo database centralizzato, spesso per modelli di licenza e costi; anche i microservizi possono lavorare con un database centralizzato, ma questo richiede di coordinare i cambi di schema tra i team.



L'alternativa è un **singolo database per servizio**: ogni microservizio (e team) è davvero indipendente, e si abilita la **persistenza poliglotta**.



Nei sistemi distribuiti garantire la consistenza forte diventa sempre più difficile: ci si può accontentare della consistenza finale, per cui a un certo punto nel futuro tutte le letture su un'entità restituiranno il valore aggiornato.

Trade-off

I microservizi offrono vantaggi, ma non gratis.

- **Vantaggi**: forti confini tra moduli, deployment indipendenti, diversità tecnologica.
- **Costi**: maggiore complessità, consistenza finale, complessità operativa.

Bisogna assicurarsi di avere una buona ragione per adottare i microservizi.

Architettura e organizzazione del team

Legge di Conway

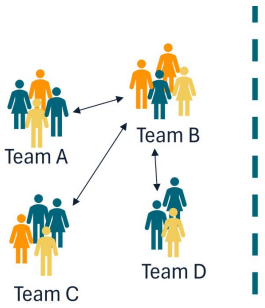
> *«Any organization that designs a system will produce a design whose structure is a copy of the organization's communication structure».*
>
> – Melvin Conway

L'accoppiamento software è permesso e favorito dalle comunicazioni umane: se si può parlare con l'autore di un codice è più facile capirlo davvero, e quindi è più facile far interagire, cioè accoppiare, il proprio codice con quello.

Team a livelli e team cross-funzionali

Spesso la gestione si concentra sul livello tecnologico: i team funzionali organizzati a livelli portano ad architetture a livelli. Il limite è che aggiungere una nuova funzionalità richiede modifiche alla logica ovunque, in tutti i livelli (UI, logica, database).

I microservizi favoriscono piccoli team indipendenti e **cross-funzionali**, che includono l'intera gamma di competenze necessarie.



Gestire la legge di Conway

Nei confronti della legge di Conway sono possibili tre atteggiamenti.

- **Ignorarla**: non tenerne conto, perché non la si conosce o non si pensa che si applichi, «spoiler»: si applica.
- **Accettarla**: riconoscerne l'impatto e assicurarsi che l'architettura non entri in conflitto con i pattern comunicativi del team.
- **Inverse Conway Maneuver**: cambiare i pattern di comunicazione del team di sviluppo per incoraggiare l'architettura desiderata.

Messaggio finale

La decomposizione del sistema e la decomposizione dell'organizzazione di sviluppo non possono avvenire in modo indipendente, non solo all'inizio del progetto ma lungo tutto il suo ciclo di vita: evolvere l'architettura e riorganizzare l'organizzazione umana devono andare di pari passo.

Lezione 16.2 Containerization for Software Engineers

Tendenze del settore software

Il settore del software è passato da un modello monolitico a servizi disaccoppiati distribuiti su molti ambienti.

- **Prima**: software **monolitico**, cicli di sviluppo lunghi, un solo ambiente di destinazione, scalata lenta.
- **Ora**: servizi **disaccoppiati**, miglioramenti rapidi e iterativi, molti ambienti di destinazione, scalata rapida necessaria.

La complessità dei deployment

Ogni servizio indipendente combina molti stack e deve raggiungere molti ambienti di destinazione.

- **Molteplicità di stack**: ogni servizio usa linguaggi, framework e database propri (es. Python, Java, PostgreSQL, Redis).
- **Molteplicità di ambienti**: ambienti di sviluppo, pre-produzione, QA, staging e produzione (on-premises, cloud pubblico, soluzioni ibride), più il disaster recovery.

Due domande definiscono la sfida: i servizi interagiscono correttamente tra loro? È possibile migrare senza intoppi tra gli ambienti?

La «Matrix from Hell»

Definition

Matrix from Hell

Matrice ottenuta incrociando gli N stack (Web App, API Endpoint, Analytics, App DB, Queue, Preprocessing) con gli M ambienti (dev laptop, production server, disaster recovery, public cloud, QA server): prima dei container ogni cella è un punto interrogativo, cioè una combinazione stack-ambiente da configurare e verificare separatamente.

L'analogia del trasporto merci

Il problema del software ricalca quello del trasporto merci prima degli anni Sessanta.

- **Molteplicità delle merci**: beni diversi che possono interagire tra loro.
- **Molteplicità dei mezzi**: metodi di trasporto e stoccaggio diversi, con passaggi difficili dall'uno all'altro.

Definition

Container da trasporto

Contenitore standardizzato per le merci che risolve la molteplicità dei beni e dei mezzi di trasporto.

I container da trasporto hanno queste proprietà.

- **Standardizzati**: hanno tutti le stesse dimensioni.
- **Versatili**: possono essere caricati con praticamente qualsiasi merce.
- **Preparati dai responsabili della spedizione**, che garantiscono che dentro non avvengano interazioni indesiderate.
- **Sigillati** fino alla consegna finale.
- **Uniformi durante il trasporto**: durante il viaggio tutti i container sono uguali, facili da caricare, scaricare e impilare.

Container per il codice

La stessa soluzione vale per il software: gli **stack** corrispondono alle merci, gli **ambienti hardware** ai mezzi di trasporto e stoccaggio.

Container software

I container risolvono la «Matrix from Hell»: ogni combinazione stack-ambiente riceve lo stesso contenitore standardizzato. Separano inoltre le responsabilità tra chi sviluppa e chi gestisce l'esercizio.

- **Sviluppatori**: si curano solo di ciò che sta **dentro** il container; setup dell'ambiente di sviluppo semplificato; nessun conflitto di librerie e dipendenze; *build once, run anywhere*.
- **Operations**: si curano solo di ciò che sta **fuori** dal container; ogni container si gestisce allo stesso modo; gestione del ciclo di vita semplificata; *configure once, run anything*.

Warning

Limiti della portabilità

La portabilità dei container vale entro la **stessa architettura e lo stesso kernel**: *build once, run anywhere* significa ovunque con architettura uguale e kernel Linux moderno; *configure once, run anything* significa qualsiasi software costruito sulla stessa architettura e kernel.

Macchine virtuali vs container

Le macchine virtuali risolvono gli stessi problemi dei container, ma sono meno leggere.

- Ogni VM porta con sé un **Guest OS** completo per ogni applicazione; il container condivide il sistema operativo dell'host tramite il container engine.
- I container consentono deployment, riavvio ecc. **significativamente più rapidi**.

La differenza di struttura è riassunta nella tabella.

Strato	Macchine virtuali	Container
Per ogni app	App + Bins/Libs + Guest OS	App + Bins/Libs
Strato condiviso	Hypervisor	Container engine
Base	Infrastructure	Operating System + Infrastructure

Docker

Docker come container engine

Definition

Docker

Container engine de facto standard per la containerizzazione del software, progetto nato nel 2013.



Docker è diffusissimo, ma non è l'unico.

- È usato da più di **13 milioni di sviluppatori**, con più di **9 milioni di applicazioni dockerizzate**.
- Esistono alternative: LXD, BuildKit, Buildah, Podman.

Architettura di Docker

L'architettura separa il client, l'host Docker e il registry delle immagini.

- **Client**: tool a riga di comando o Docker Desktop GUI; invia al daemon comandi come `build`, `pull`, `run`.
- **Docker host**: ospita il **daemon** (`dockerd`), che costruisce le immagini ed esegue i container.
- **Registry**: archivio di immagini (DockerHub, JFrog, Quay.io) da cui il daemon scarica le immagini.

Il flusso di comandi e immagini è sintetizzato dal diagramma.

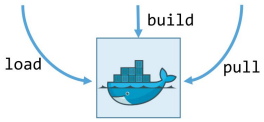


Immagini Docker

Definition

Immagine Docker
Template portatile e in sola lettura che contiene tutte le istruzioni per creare un container.

Un'immagine si ottiene in tre modi.



- **Caricata** da un file archivio tar.
- **Scaricata** da un registry.
- **Costruita** estendendo un'immagine esistente con una lista di istruzioni scritte in un Dockerfile.

Dockerfile

Definition

Dockerfile
Insieme di comandi per assemblare un'immagine, a partire da un'immagine base.

I comandi principali del Dockerfile sono i seguenti.

- `FROM`: immagine base di partenza.
- `RUN`: esegue comandi (es. installazione di pacchetti).
- `COPY`: copia file dalla directory del Dockerfile nel filesystem del container.
- `EXPOSE`: espone una porta del container verso il Docker host.
- `ENV`: imposta variabili d'ambiente.
- `CMD`: comando di default eseguito all'avvio del container.

Un Dockerfile minimo estende `ubuntu:20.04` installando pacchetti e copiando un file.

Dockerfile

```
1 FROM ubuntu:20.04
2 RUN apt update -qq && apt install -y -q fortune cowsay
3 COPY ./file.txt /home/file.txt
4 CMD bash
```

Comandi Docker essenziali

La gestione di immagini e container avviene tramite la CLI `docker`.

Immagini

Le immagini si ottengono, ispezionano e costruiscono con questi comandi.

- **`docker pull <immagine>`** scarica un'immagine dal registry (es. `docker pull ubuntu:20.04`).
- **`docker image list`** elenca le immagini locali con repository, tag, ID e dimensione.
- **`docker build -t <nome> .`** costruisce un'immagine dal Dockerfile della directory corrente; i layer già costruiti vengono riutilizzati dalla cache.

Ciclo di vita di un container

Un container attraversa creazione, avvio, esecuzione e rimozione.

- **`docker run -it --name <nome> <immagine>`** crea e avvia un container con terminale interattivo.
- **`docker container list --all`** elenca tutti i container, inclusi quelli arrestati.
- **`docker start <container>`** riavvia un container arrestato.

- `docker attach <container>` collega il terminale alla sessione principale del container.
- `docker exec -ti <container> <comando>` esegue un comando in un container in esecuzione senza collegarsi alla sua sessione principale.
- `docker kill <container>` termina un container in esecuzione.
- `docker rm <container>` elimina un container arrestato.

Detach e trasferimento file

Durante l'esecuzione ci si può scollegare dal terminale e scambiare file con l'host.

- Il **detach** dal terminale interattivo si esegue premendo `CTRL+P` seguito da `CTRL+Q`, senza arrestare il container.
- `docker cp <sorgente> <destinazione>` copia file tra host e container in entrambe le direzioni (es. `docker cp ./file.txt my-first-container:/home/file.txt`).

Applicazioni container-native

Definition

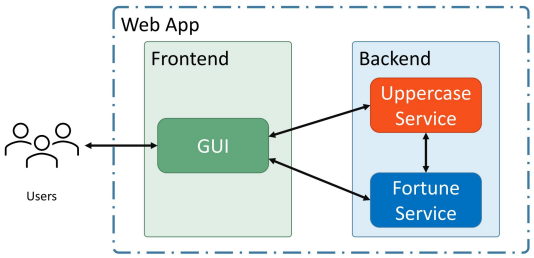
Software container-native

Software per cui i container sono le unità di infrastruttura di primo livello (non server o VM), progettato per i container fin dall'inizio.

Architettura dell'applicazione

L'applicazione di riferimento è una web app a microservizi con tre funzionalità: ottenere un messaggio fortune, ottenere la versione maiuscola di qualsiasi stringa, ottenere la versione maiuscola del proprio messaggio fortune.

- **Frontend**: pagina web con un po' di JavaScript.
- **Backend**: due microservizi che comunicano tramite **HTTP**, un protocollo web standard.
- **Fortune service**: Nodejs ed Express.
- **Uppercase service**: Python e Flask.



Fortune service

Il fortune service offre una sola funzione: restituisce un messaggio fortune scelto per l'utente corrente.

- L'utente invia una richiesta **GET** all'endpoint `/fortune`, cioè `http://{fortune-service-location}/fortune`.
- Il servizio risponde con un JSON contenente il campo `fortune`.

```JavaScript
app.get('/fortune', (req, res) => {
  const fortunes = ["Great fortune!", "Meh, average fortune",
                    "Terrible fortune! Be careful!"]
  var fortune = fortunes[Math.floor(Math.random()*fortunes.length)]
  res.send({ 'fortune': fortune })
})
```

Uppercase service

L'uppercase service offre due funzioni: rendere maiuscola una stringa e restituire un fortune maiuscolo.

- **Maiuscole**: l'utente invia una **POST** all'endpoint `/uppercase` con body JSON contenente il campo `message`; il servizio risponde con un JSON con i campi `original` e `uppercase`.

- **Fortune maiuscolo**: l'utente invia una **GET** all'endpoint `/uppercase-fortune`; il servizio recupera un fortune dal fortune service, lo rende maiuscolo e lo restituisce nel campo `fortune`.

```python
@app.route('/uppercase', methods=['POST'])
def uppercase():
    data = request.json           # dati JSON nel body della richiesta
    message = data["message"]
    return jsonify({'original': message,
                    'uppercase': message.upper()})
```

Il secondo snippet mostra la **composizione tra servizi**: l'uppercase service invoca il fortune service via HTTP e combina il risultato.

```python
@app.route('/uppercase-fortune')
def uppercase_fortune():
    req = requests.get(f"{fortune_baseurl}/fortune")
    data = req.json()
    return jsonify({'fortune': data['fortune'].upper()})
```

Vista completa dell'applicazione

Nella vista completa gli utenti raggiungono l'app dal browser: il frontend è servito da un web server **Nginx**, che espone la GUI verso gli utenti, mentre i due servizi stanno nel backend.

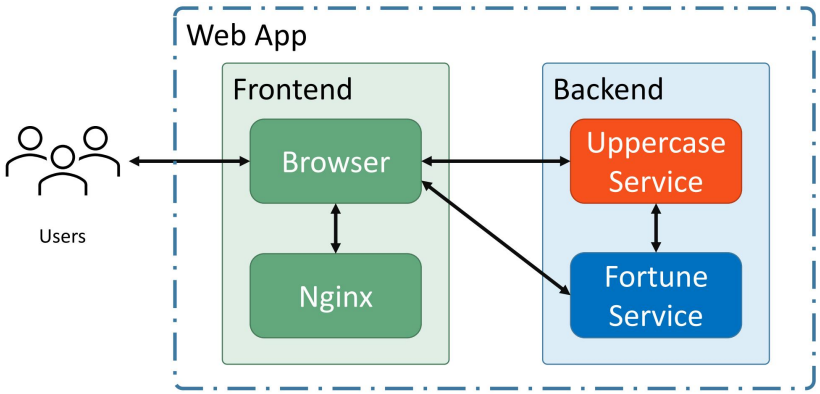
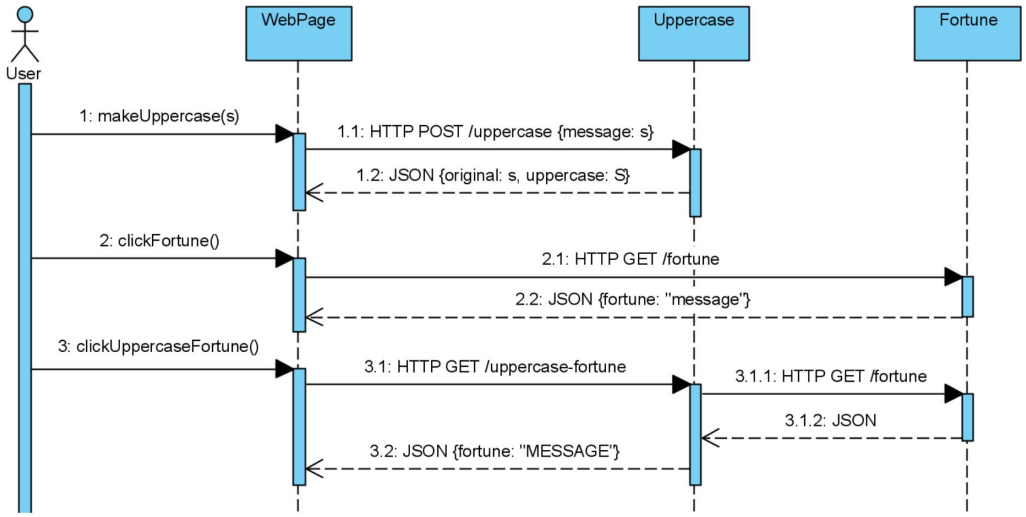


Diagramma di sequenza

Le interazioni tra utente, frontend e servizi seguono questo diagramma di sequenza.



Immagini Docker dei servizi

Ogni componente dell'app viene containerizzato con un proprio Dockerfile.

Il fortune service parte dall'immagine base Nodejs 14; **`WORKDIR`** imposta la directory di lavoro base del container.

Dockerfile

```
FROM node:14
WORKDIR /home/app
COPY package*.json .
COPY app.js .
RUN npm install
EXPOSE 3000
CMD node app.js
```

L'uppercase service parte dall'immagine base Python 3.8; `--host 0.0.0.0` rende l'app raggiungibile dall'esterno del container.

Dockerfile

```
FROM python:3.8
COPY ./requirements.txt /home/requirements.txt
RUN pip3 install -r /home/requirements.txt
COPY app.py /home/app.py
EXPOSE 5000
CMD flask --app /home/app.py run --host 0.0.0.0
```

Il web server che ospita la GUI parte dall'immagine Nginx e copia la pagina nel document root predefinito.

Dockerfile

```
FROM nginx:latest
COPY ./website /usr/share/nginx/html
```

Orchestrazione con docker-compose

Definition

Docker Compose
Strumento di orchestrazione che definisce e avvia applicazioni multi-container a partire da un unico file YAML.

Il file compose descrive i servizi dell'applicazione.

YAML

```
version: "3.9"
services:
  frontend:
    build: ./frontend/
    container_name: "uppercase-fortune-frontend"
    depends_on:
      - fortune-service
      - uppercase-service
    ports:
      - "8080:80"
  fortune-service:
    build: ./fortune/
    container_name: "fortune-service-container"
    ports:
      - "3000:3000"
  uppercase-service:
    build: ./uppercase/
    container_name: "uppercase-service-container"
    environment:
```

```
20        FORTUNE_URL: http://fortune-service:3000
21      depends_on:
22        - fortune-service
23      ports:
24        - "5000:5000"
```

Le direttive chiave del file sono le seguenti.

- **`build`**: directory contenente il Dockerfile del servizio.
- **`depends_on`**: impone l'ordine di avvio dei servizi (frontend dopo i due servizi di backend, uppercase service dopo fortune service).
- **`ports`**: mappa le porte `host:container` (es. `"8080:80"` espone la porta 80 del container sulla porta 8080 dell'host).
- **`environment`**: variabili d'ambiente del container; l'uppercase service raggiunge il fortune service tramite il nome del servizio nell'URL.
- **`container_name`**: nome assegnato al container.

L'avvio dell'intero progetto avviene con `docker-compose -p "uppercase-fortune-app" up -d`: compose costruisce le immagini, crea la rete di default del progetto e avvia i container di tutti i servizi.

Parte III

Software and Design Pattern

Capitolo 10

Lezione 17-18 Design Patterns

Software Development Lifecycle

La progettazione con i pattern si colloca all'interno del ciclo di vita dello sviluppo software, che va dai requisiti alla manutenzione.

- **Requirements Engineering**: i requisiti si raccolgono con interviste agli stakeholder, storie e scenari, personas; si specificano con use case, linguaggio naturale e modelli di dominio.
- **System Design**: si definisce l'architettura del sistema, allocando i requisiti ai sottosistemi software e i sottosistemi alle risorse hardware (architectural pattern); si definiscono i sottosistemi e i loro oggetti applicando i software design pattern, con l'usability engineering (wireframing ad alta fedeltà, mock-up).
- **Implementation**: ogni sottosistema è implementato con codice sorgente e altri artefatti, seguendo Clean Code, framework e ORM, con attenzione alla qualità del software.
- **Testing**: si garantisce che il software soddisfi i clienti con code inspection, functional testing (unit, integration, system) e usability testing.
- **Operation and Maintenance**: il sistema è messo in uso pratico; la manutenzione serve a correggere errori non scoperti nelle fasi precedenti e ad adattare il software a cambiamenti dei requisiti o del suo ambiente.

Riuso di codice e riuso di design

Codice e design si riusano per non reinventare la ruota e per sfruttare il lavoro precedente, ma con requisiti diversi.

- **Riuso di codice**: richiede codice pulito, elegante, comprensibile, generale e stabile.
- **Riuso di design**: richiede una comprensione precisa dei design comuni e ricorrenti.

I sistemi object-oriented sfruttano strutture di design ricorrenti che promuovono **astrazione**, **flessibilità**, **modularità** ed **eleganza**, e in esse risiede una preziosa conoscenza di progettazione. Il problema è catturare, comunicare e applicare questa conoscenza per il riuso.

Che cos'è un design pattern

Definition

Design pattern

Soluzione comune a un problema ricorrente nella progettazione. Astrae una struttura di design ricorrente, che comprende classi e/o oggetti con le loro dipendenze, strutture, interazioni e convenzioni; la specifica esplicitamente e distilla l'esperienza di progettazione.

Un design pattern ha quattro parti fondamentali: **nome**, **problema**, **soluzione**, **conseguenze e trade-off** della sua applicazione. È inoltre indipendente dal linguaggio e dall'implementazione, ed è una «micro-architettura»: non si applica meccanicamente, perché lo sviluppatore deve tradurre la soluzione in termini concreti nel contesto dell'applicazione.

La storia del concetto si sintetizza in tre tappe.

- L'architetto **Christopher Alexander** introduce l'idea con *A Pattern Language: Towns, Buildings, Construction* (1977).
- La **Gang of Four** (Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides) pubblica *Design Patterns: Elements of Reusable Object-Oriented Software* (1995).
- Seguono molte conferenze, simposi e libri; i cataloghi principali sono quello GoF e **POSA** (*Pattern-Oriented Software Architecture*, Buschmann et al., 1996).

I design pattern sono importanti per quattro ragioni.

- **Codificano il buon design**: distillano e diffondono esperienza, aiutano novizi ed esperti e astraiono come pensare il design.
- **Danno nomi espliciti alle strutture di design**: creano un vocabolario comune, riducono la complessità e aumentano l'espressività.

- **Catturano e preservano le informazioni di design**: articolano le decisioni in modo succinto e migliorano la documentazione.
- **Facilitano il restructuring/refactoring**: i pattern sono interrelati e aggiungono flessibilità.

Classificazione dei pattern

I pattern si distinguono per complessità, livello di dettaglio e scala di applicabilità, e in ingegneria del software si separano in due livelli.

- **Architectural pattern**: pattern di alto livello il cui ambito è la struttura dell'intero sistema software; possono essere implementati in qualsiasi linguaggio di programmazione. **REST** può essere considerato un architectural pattern.
- **Design pattern** in senso stretto: pattern di medio/basso livello che si applicano alle scelte di progettazione dentro componenti o moduli e risolvono problemi ricorrenti del design e dell'implementazione object-oriented.

In base allo scopo o **intent** si distinguono tre categorie di design pattern.

- **Creazionali**: meccanismi per creare oggetti, con l'obiettivo di aumentare flessibilità, riuso del codice esistente e manutenibilità.
- **Strutturali**: modi di assemblare oggetti e classi in strutture più grandi, mantenendo le strutture flessibili ed efficienti.
- **Comportamentali**: gestione della comunicazione efficace e dell'assegnazione di responsabilità tra oggetti.

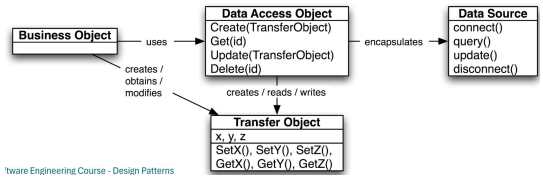
La classificazione GoF incrocia scopo e scope (class/object).

Scope	Creazionali	Strutturali	Comportamentali
Class	Factory Method	Adapter (class)	Interpreter, Template Method
Object	Abstract Factory, Builder, Prototype, Singleton	Adapter (object), Bridge, Composite, Decorator, Flyweight, Facade, Proxy	Chain of Responsibility, Command, Iterator, Mediator, Memento, Observer, State, Strategy, Visitor

Data Access Object

Il pattern DAO affronta la variabilità dell'accesso ai dati.

- **Problema**: l'accesso ai dati varia molto a seconda del tipo di memorizzazione (DBMS relazionale, DBMS NoSQL, flat file, ecc.) e dell'implementazione del fornitore.
- **Soluzione**: il pattern **Data Access Object (DAO)** astrae e incapsula ogni accesso alla sorgente dati; ogni DAO gestisce la connessione con la sorgente per ottenere e memorizzare dati e implementa il meccanismo di accesso richiesto per lavorare con essa.



Si definisce una classe DAO per ciascuna classe entità, che astrae le operazioni **CRUD** (Create, Retrieve, Update, Delete); i metodi «utili» dipendono dalla classe di dominio e dall'applicazione (per `LocationDao`: `findById`, `findByName`, `find`, `save`, `delete`). I benefici sono tre.

- Permette di «collegare» implementazioni di memorizzazione diverse con impatto minimo sul resto del sistema.
- Disaccoppia il layer di persistenza.
- Favorisce e supporta il riuso del codice.

Singleton

L'intento del Singleton è garantire che una classe abbia solo un'istanza e fornire un punto di accesso globale ad essa. Si applica in due casi.

- Quando deve esistere esattamente una istanza di una classe, accessibile da un punto di accesso ben noto.
- Quando l'unica istanza deve essere estendibile per sottoclassazione e i client devono poter usare l'istanza estesa senza modificare il proprio codice.

La struttura si realizza con un costruttore privato e un'operazione statica di accesso all'istanza.

```java
public class Singleton {
  private static Singleton istanza = null;
  private Singleton() {}
  public static Singleton getSingleton() {
    if (istanza == null) { istanza = new Singleton(); }
    return istanza;
  }
}
```

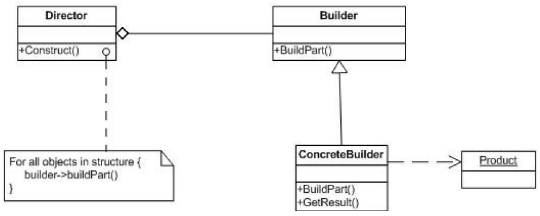
Pur usando una sola istanza, modifiche all'operazione di accesso possono permettere un numero variabile di istanze (ad esempio una classe che ammette fino a tre istanze). Le conseguenze dell'applicazione sono le seguenti.

- Riduce l'inquinamento del namespace (namespace pollution).
- Rende facile cambiare idea e permettere più di un'istanza.
- Presenta gli stessi difetti di una variabile globale se usato male.
- L'implementazione può essere meno efficiente di una globale e presenta insidie di concorrenza (concurrency pitfalls).

Builder

Builder e Factory formano una famiglia di pattern che affronta il problema delle classi complesse per cui il meccanismo object-oriented base dei costruttori non è sufficiente a garantire flessibilità ed efficienza. Lo scopo del Builder è rendere la costruzione di un oggetto flessibile e decomponibile, separandola dal costruttore effettivo dell'oggetto.

Nella soluzione il singolo prodotto (`Product`) è costruito dal `ConcreteBuilder`, mentre la classe `Director` può gestire la costruzione di un insieme di oggetti `Product`.



Il pattern si usa quando sussistono queste condizioni.

- La creazione di un prodotto complesso deve essere indipendente dalle parti specifiche che lo compongono.
- Il processo di creazione non deve conoscere il processo di assemblaggio (come le parti vengono unite per costituire il prodotto).
- Il processo di creazione deve permettere rappresentazioni diverse per l'oggetto costruito.

Nell'uso tipico il prodotto è separato da un oggetto Builder che gestisce tutte le varianti di costruzione in modo flessibile: i metodi `with...` restituiscono il Builder stesso, così le chiamate si innestano; il metodo `build()` valida e invoca il costruttore package-private del prodotto.

```Java
Computer gamingPC = new ComputerBuilder("AMD Ryzen 7", 32)
    .withDedicatedGPU("AMD Radeon RX 7800")
    .withSSD(2000)
    .includeOS()
    .build();
```

La logica di validazione sta nel Builder, non nella classe del prodotto: ad esempio `build()` solleva una `IllegalStateException` se la RAM è inferiore a 4GB, quindi un PC «obsoleto» non viene proprio costruito.

Il pattern è molto usato nelle librerie Java standard: `java.lang.StringBuilder` per costruire stringhe, `java.util.Locale.Builder` per oggetti locale complessi (lingua, paese, script, varianti), `java.time.format.DateTimeFormatterBuilder` per i formatter data-ora. La stessa catena di chiamate fluenti si usa nelle API REST, ad esempio per creare un JWT con `JWT.create().withIssuer(...).withClaim(...).sign(...)` e per costruire la risposta con `Response.status(...).entity(...).build()`.

Factory Method

L'intento del Factory Method è istanziare oggetti senza esporre al client la logica di creazione, riferendosi all'oggetto appena creato tramite un'interfaccia comune.

La motivazione nasce dai framework: usano classi astratte per definire e mantenere relazioni tra oggetti e devono creare oggetti, ma conoscono solo le classi astratte, che non possono istanziare; il factory method incapsula la conoscenza di quale sottoclasse creare, spostandola fuori dal framework.

Il pattern si applica in due situazioni.

- Quando una classe non può anticipare la classe degli oggetti che deve creare.
- Quando le classi delegano la responsabilità a una di diverse sottoclassi ausiliarie e si vuole localizzare la conoscenza di quale sottoclasse sia la delegata.

I partecipanti sono tre: il **Product** definisce l'interfaccia degli oggetti creati dal factory method, il **ConcreteProduct** implementa l'interfaccia del prodotto, il **Creator** dichiara il factory method che restituisce un oggetto di tipo `Product`.

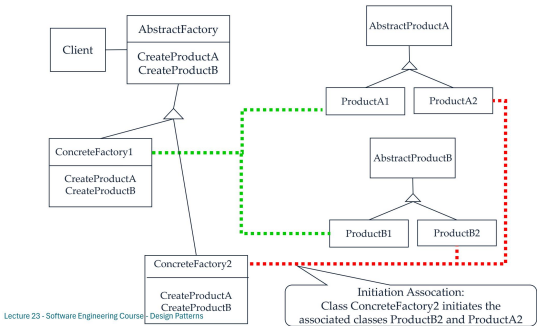
La fabbrica incapsula la scelta dell'istanza concreta, che dipende da una condizione di contesto.

```Java
public class VehicleFactory {
  public Vehicle instantiateVehicle() {
    // logica per scegliere l'istanza
    if (weather.rains()) return new Car();
    else return new Bike();
  }
}
```

La motivazione si vede ripartendo dal system design: se `Traveler` ha un attributo di tipo `Car`, c'è un forte accoppiamento e la classe non è riusabile con altri mezzi né robusta a cambiamenti di `Car`. Dichiarando invece un attributo di tipo `Vehicle` (interfaccia con `move()`) e iniettando il mezzo con `setV(...)`, `Traveler` funziona con qualsiasi implementazione (`Car`, `Bike`).

Abstract Factory

La motivazione dell'Abstract Factory è l'indipendenza dalle famiglie di prodotti: un toolkit di interfaccia utente deve supportare più standard di look-and-feel (X, Windows Vista, il Finder di MacOS) oppure un sistema di facility management per la casa intelligente deve supportare diversi sistemi di controllo (Instabus di Siemens, Metasys di Johnson & Control, lo standard proprietario Zumtobe). In entrambi i casi si vuole un'unica interfaccia, indipendente dal produttore.



L'applicabilità del pattern copre quattro esigenze.

- **Indipendenza da inizializzazione o rappresentazione**: il sistema non deve dipendere da come i suoi prodotti sono creati, composti o rappresentati.
- **Indipendenza dal produttore**: il sistema va configurato con una tra molteplici famiglie di prodotti; inoltre si può voler fornire una libreria di classi al cliente senza rivelare quale prodotto particolare si sta usando.
- **Vincoli su prodotti correlati**: una famiglia di prodotti correlati è progettata per essere usata insieme e il vincolo va imposto.
- **Gestione del cambiamento in arrivo**: si usa una particolare famiglia di prodotti ma ci si aspetta che la tecnologia sottostante cambi presto, con nuovi prodotti sul mercato.

Nell'esempio GUI una fabbrica astratta, selezionata leggendo la configurazione, decide quale famiglia di widget istanziare.

```java
abstract class GUIFactory {
  public static GUIFactory getFactory() {
    int sys = readFromConfigFile("OS_TYPE");
    if (sys == 0) return new WinFactory();
    else return new OSXFactory();
  }
  public abstract Button createButton();
}
```

Il client ottiene la factory con `getFactory()`, crea il bottone con `createButton()` e lo usa tramite l'astrazione `Button`, senza mai dipendere dalle classi concrete `WinButton` o `OSXButton`.

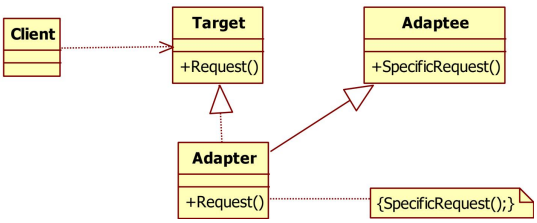
Pattern strutturali

Adapter

Lo scopo dell'Adapter è convertire l'interfaccia di una classe in un'altra interfaccia attesa dal client, permettendo la cooperazione di classi che altrimenti avrebbero interfacce incompatibili; il sinonimo è Wrapper. La motivazione è riusare una classe esistente e già collaudata la cui interfaccia però non corrisponde a quella richiesta.

I partecipanti sono quattro.

- **Client**: la classe che vuole usare il servizio da adattare.
- **Target**: l'interfaccia che vorremmo avere dal servizio da usare.
- **Adaptee**: la classe esistente che fornisce il servizio.
- **Adapter**: la classe che adatta l'interfaccia fornita dall'adaptee all'interfaccia Target.



L'Adapter può essere usato in tre casi.

1. Si vuole usare una classe esistente (`Adaptee`) con interfaccia incompatibile.
2. Si vuole creare una classe riusabile che dovrà collaborare con classi non prevedibili al momento della sua creazione.
3. Si vuole riusare un insieme di sottoclassi esistenti, ma adattare l'interfaccia creando una sottoclasse per ciascuna è scomodo: meglio creare un adapter per la classe padre.

L'adapter incapsula l'adaptee e ne converte i parametri verso l'interfaccia richiesta.

```java
public class Adapter implements Target {
  public Adaptee adaptee = new Adaptee();
  public double evaluate(double weight, double height) {
    return adaptee.calcolaIMC(weight / 0.453, height / 0.0254);
  }
}
```

L'uso tipico è con servizi e microservizi: il design del software porta a progettare un'interfaccia; un microservizio esistente fornisce il servizio con la propria interfaccia; si implementa un Adapter che interagisce con il fornitore del servizio, converte i parametri (eventualmente rielaborandoli) e li fornisce al Target, che può essere anch'esso una interfaccia. Non si modifica direttamente l'Adaptee perché se ne possiede solo l'interfaccia o perché è già usato da altri.

Adapter di classe e adapter a oggetti

Le due varianti del pattern si distinguono per il legame tra Adapter e Adaptee.

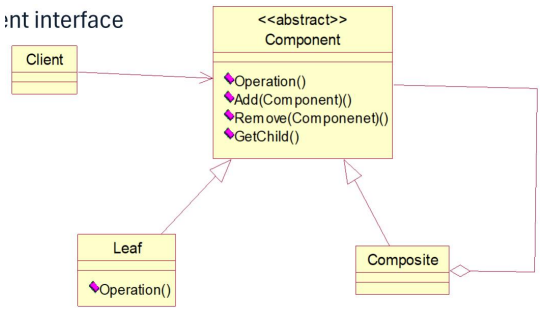
- **Adapter di classe**: utile quando una o più classi esistenti forniscono una parte o tutti i servizi necessari ma non usano l'interfaccia richiesta (es. un lettore audio che suona solo mp3 riusa, tramite `MediaAdapter`, un lettore avanzato capace di suonare vlc e mp4).
- **Adapter a oggetti**: la **delega** lega Adapter e Adaptee, mentre l'**ereditarietà di interfaccia** specifica l'interfaccia della classe Adapter; Target e Adaptee (spesso detta sistema legacy) esistono prima dell'Adapter. È usato spesso per fornire una nuova interfaccia a componenti legacy esistenti (interface engineering, reengineering); ad esempio `ServicesEnumeration` adatta un `RegisteredServices` all'interfaccia `Enumeration` delegando `hasMoreElements()` e `nextElement()`.

Composite

Lo scopo del Composite è comporre oggetti in strutture ad albero che rappresentano gerarchie tutto-parte, facendo sì che i client trattino allo stesso modo oggetti compositi e oggetti singoli.

I partecipanti sono quattro.

- **Component**: dichiara l'interfaccia degli oggetti e il comportamento di default, e un'interfaccia per accedere e gestire i componenti figli.
- **Leaf**: implementa gli oggetti component senza figli e il loro comportamento.
- **Composite**: definisce il comportamento dei componenti con figli, memorizza i componenti figli e implementa le operazioni sui figli dichiarate dall'interfaccia Component.
- **Client**: usa gli oggetti compositi tramite l'interfaccia Component.



Il caso tipico è un file system, dove una `Directory` può contenere `File` e altre directory (anche vuote) e l'operazione `ls()` viene invocata ricorsivamente e polimorficamente sui figli tramite l'interfaccia `AbstractFile`.

```java
class Directory implements AbstractFile {
  public void ls() {
    System.out.println(EsempioComposite.indentazione + nome);
    for (int i = 0; i < listaFile.size(); ++i) {
      AbstractFile obj = (AbstractFile) listaFile.get(i);
      obj.ls(); // polimorfismo
    }
  }
}
```

Le conseguenze dell'applicazione sono le seguenti.

- Permette di definire gerarchie di classi formate da oggetti primitivi e compositi: i primitivi si compongono in oggetti più complessi, componibili ricorsivamente, e dovunque il client si aspetta un primitivo si può usare indifferentemente un composito.

- Semplifica il client: strutture composite e oggetti singoli si usano in modo uniforme, e il client in genere non sa (né deve saperlo) se sta lavorando con una foglia o con un composito.
- È flessibile: è più facile aggiungere nuovi tipi di componenti, perché nuove sottoclassi di Leaf o Composite funzionano automaticamente nelle strutture esistenti con il codice client.
- È spesso usato per istanziare la gerarchia di widget di una GUI.

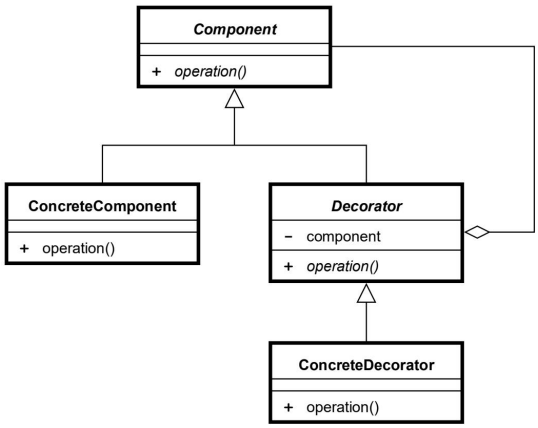
Decorator

Il pattern Decorator permette di creare oggetti complessi aggiungendo funzionalità a oggetti più semplici: l'oggetto complesso include quello semplice e lo «decora». Il meccanismo ha quattro caratteristiche.

- L'oggetto originale è passato come parametro al costruttore del decorator, che restituisce l'oggetto «decorato».
- Più decorator possono essere concatenati tra loro, aggiungendo funzionalità in modo incrementale: la classe complessa è rappresentata dall'ultimo anello della catena.
- La concatenazione segue una composizione arbitraria: il numero di comportamenti possibili dell'oggetto composito cresce con legge combinatoria al crescere dei decorator disponibili.
- È un'alternativa all'ereditarietà singola o multipla: con l'ereditarietà l'aggiunta di funzionalità è statica secondo i legami della gerarchia di classi, e a run-time non si può ottenere una combinazione arbitraria di funzionalità né aggiungerle/rimuoverle.

Nello schema concettuale intervengono quattro ruoli.

- **Component** è l'astrazione/interfaccia.
- **ConcreteComponent** è l'implementazione base del componente.
- **Decorator** è una classe astratta che include un Component e ne eredita l'operazione.
- **ConcreteDecorator** migliora il Component: ne assume tutte le caratteristiche (ha un attributo component) e può modificarne i metodi in override.



Un decorator concreto incapsula l'oggetto decorato e ne estende il comportamento in override.

```java
public class Mozzarella extends PizzaDecorator {
  public Mozzarella(Pizza pizza) { super(pizza); }
  public String getDescription() {
    return pizza.getDescription() + ", Mozzarella";
  }
  public double getCost() { return pizza.getCost() + 1.50; }
}
```

Nelle librerie Java il pattern è usato da `Reader` con `BufferedReader(Reader in)`, `InputStream` con `InputStreamReader(InputStream in)` e `BufferedInputStream(InputStream in)`.

```java
BufferedReader in = new BufferedReader(new InputStreamReader(System.in));
```

Vantaggi e svantaggi del pattern si bilanciano così.

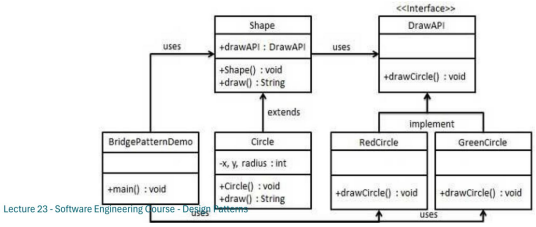
- **Vantaggio**: combinando i decorator si ottengono combinazioni illimitate di chiamate al builder, senza dichiarare le classi per ogni combinazione («prosciutto e funghi», «margherita», «margherita doppia mozzarella», ecc.).
- **Svantaggio**: la costruzione di un oggetto richiede di istanziare più oggetti e chiamare più costruttori, quindi il pattern è inefficiente dal punto di vista computazionale e, di conseguenza, anche da quello energetico.

Bridge

> *Decouple an abstraction from its implementation so that the two can vary independently.*
>
> – Gamma et al.

Il Bridge permette di decidere dinamicamente tra implementazioni diverse di un'interfaccia. È utile quando variano spesso sia la classe sia ciò che la classe fa (la classe stessa può essere pensata come l'implementazione, ciò che può fare come l'astrazione); quando una classe varia spesso il pattern diventa molto utile, perché i cambiamenti al codice si fanno facilmente con una conoscenza minima e preliminare del programma.

Nell'esempio un cerchio può essere disegnato in colori diversi usando lo stesso metodo della classe astratta ma diverse classi implementatrici del bridge.



L'astrazione mantiene un riferimento all'implementatore, ricevuto nel costruttore.

```java
public abstract class Shape {
  protected DrawAPI drawAPI;
  protected Shape(DrawAPI drawAPI) { this.drawAPI = drawAPI; }
  public abstract void draw();
}
```

Conseguenza: il modo in cui un cerchio viene disegnato può variare indipendentemente dalla classe `Circle`, e la classe `Circle` può variare senza impatti sull'implementazione di disegno.

Adapter e Bridge a confronto

Entrambi i pattern nascondono i dettagli dell'implementazione sottostante, ma si collocano in momenti diversi del ciclo di vita del design.

- **Adapter**: mira a far collaborare componenti non correlati; si applica a sistemi già progettati (reengineering, interface engineering).
- **Bridge**: si usa a priori nel design, per far variare indipendentemente astrazioni e implementazioni; è il green field engineering di un «sistema estendibile», in cui nuovi elementi possono essere aggiunti allo «zoo di oggetti» anche se non noti in fase di analisi o di system design.

Facade

Definition

Facade

Interfaccia unificata verso un insieme di oggetti di un sottosistema. La facciata definisce un'interfaccia di livello più alto che rende il sottosistema più facile da usare, astrae i dettagli più crudi (*gory details*) e permette di fornire un'architettura chiusa.

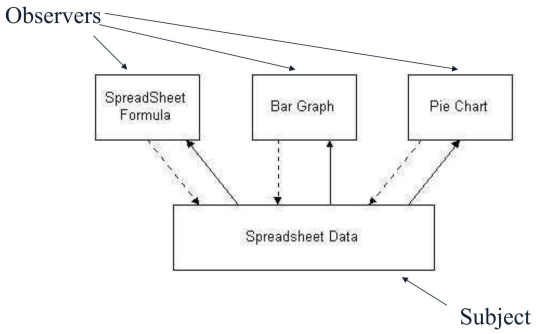
La facciata incapsula gli oggetti del sottosistema ed espone operazioni di livello superiore: ad esempio `ShapeMaker` possiede `circle`, `rectangle` e `square` ed espone `drawCircle()`, `drawRectangle()` e `drawSquare()`, ciascuna che delega all'oggetto interno.

La struttura ideale di un sottosistema combina Facade e Adapter/Bridge: un sottosistema consiste di un oggetto interfaccia, di un insieme di oggetti di dominio applicativo (oggetti entità) che modellano entità reali o sistemi esistenti, alcuni dei quali sono interfacce verso sistemi esistenti, e di uno o più oggetti di controllo. I due ruoli chiave sono realizzati con i pattern.

- **Interface object**: realizzato con un Facade, fornisce l'interfaccia del sottosistema.
- **Interfaccia verso sistemi esistenti**: realizzata con Adapter o Bridge, fornisce l'interfaccia al sistema esistente (sistema legacy), che non è necessariamente object-oriented.

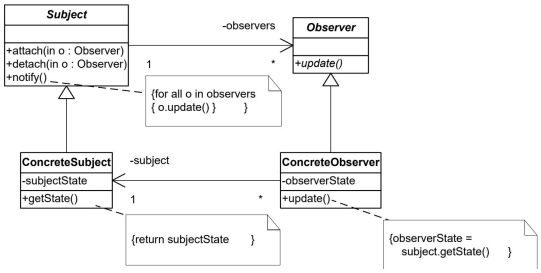
Observer

L'intento dell'Observer è definire una dipendenza uno-a-molti tra oggetti, così che quando un oggetto cambia stato tutti i suoi dipendenti sono notificati e aggiornati automaticamente. L'esempio concettuale è un unico dato (es. `SpreadSheet Data`) osservato da più viste (formula, grafico a barre, grafico a torta) che si aggiornano al variare del dato.



Il pattern si applica in tre casi.

- Quando un'astrazione ha due aspetti, uno dipendente dall'altro.
- Quando un cambiamento a un oggetto richiede di cambiarne altri e non si sa quanti oggetti vanno cambiati.
- Quando un oggetto deve notificarne altri senza fare assunzioni su chi siano.



Nella struttura il soggetto astratto conserva la lista degli osservatori, offre la registrazione e notifica i cambiamenti invocando `aggiorna()` su ciascun osservatore; il soggetto concreto, in `setStato(...)`, effettua la chiamata extra `update()` rispetto a una normale classe; l'osservatore concreto è legato al soggetto che vuole osservare e in `aggiorna()` legge lo stato aggiornato.

```java
public abstract class Soggetto {
  protected ArrayList<Osservatore> osservatori;
  public void registraOsservatore(Osservatore o) {
    osservatori.add(o);
  }
  protected void update() {
    for (int i = 0; i < osservatori.size(); i++)
      osservatori.get(i).aggiorna();
  }
  public abstract void setStato(int s);
}
```

Le conseguenze dell'applicazione sono cinque.

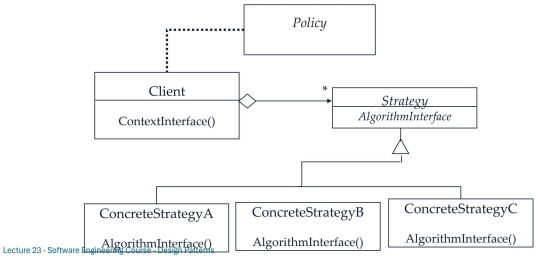
- **Modularità**: subject e observer possono variare indipendentemente.
- **Estensibilità**: si possono definire e aggiungere un numero qualsiasi di observer.
- **Personalizzabilità**: observer diversi forniscono viste diverse del subject.
- **Aggiornamenti inattesi**: gli observer non si conoscono tra loro.
- **Overhead di aggiornamento**: possono servire hint.

Gli aspetti di implementazione riguardano la mappatura subject-observer, i riferimenti pendenti (dangling references), l'evitare protocolli di aggiornamento specifici dell'observer (modelli push e push/pull) e la registrazione esplicita delle modifiche di interesse.

Strategy

Per lo stesso compito esistono in genere molti algoritmi diversi (es. spezzare un flusso di testo in righe, fare il parsing di un insieme di token in un albero di sintassi astratta, ordinare una lista di clienti). Algoritmi diversi sono appropriati in momenti diversi; non si vogliono supportare tutti se non servono; un nuovo algoritmo va aggiunto facilmente senza disturbare l'applicazione che lo usa.

La **Policy** decide quale Strategy è migliore in funzione del Context corrente.



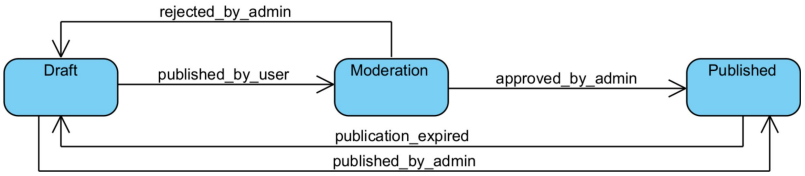
Il pattern si applica in due casi.

- Molte classi correlate differiscono solo per il comportamento: Strategy permette di configurare una singola classe con uno tra molti comportamenti.
- Servono varianti di un algoritmo che scambiano spazio con tempo: tutte le varianti si implementano come gerarchia di classi di algoritmi, selezionabili a run-time.

In un'applicazione database, ad esempio, le strategie di ordinamento `BubbleSort`, `QuickSort` e `ShellSort` implementano la stessa interfaccia e sono intercambiabili rispetto alla classe `Database` che le usa.

State

Classi e sottosistemi possono essere visti come sistemi reattivi: reagiscono a stimoli esterni (eventi) in base al loro stato corrente. Il caso motivante è una classe `Document` il cui comportamento è descritto da uno statechart con stati `Draft`, `Moderation` e `Published` e transizioni come la pubblicazione da parte dell'utente, l'approvazione o il rifiuto da parte dell'admin e la scadenza della pubblicazione.



L'implementazione diretta con condizionali sullo stato sembra gestibile.

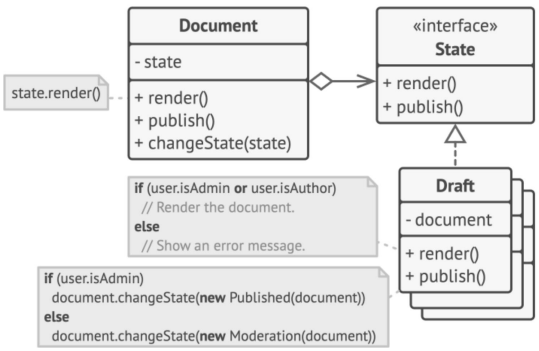
```java
public void publish() {
  switch (state) {
    case "draft": state = "moderation"; break;
    case "moderation":
      if ("admin".equals(user.getRole())) state = "published";
      break;
  }
}
```

Error

Condizionali mostruosi

Aggiungendo più stati e più transizioni, quasi tutti i metodi finiscono per contenere condizionali mostruosi; ogni modifica alla logica di transizione richiede di cambiare i condizionali di stato in ogni metodo.

Il pattern State risolve il problema: creare nuove classi per tutti i possibili stati di un oggetto e delegare a queste classi il comportamento specifico dello stato.



State e Strategy a confronto

State è in certa misura simile a Strategy: in entrambi l'implementazione di certe operazioni è delegata a oggetti «pluggabili». La differenza chiave riguarda la consapevolezza reciproca.

- Nel pattern **State** i singoli stati possono conoscersi e avviare transizioni da uno stato all'altro: uno stato può decidere quale sarà il prossimo.
- Nel pattern **Strategy** le strategie quasi mai si conoscono: forniscono implementazioni separate e indipendenti della stessa operazione.

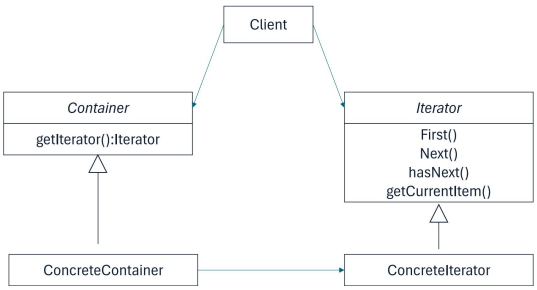
Iterator

La situazione di partenza è un gruppo di oggetti su cui si vuole iterare senza conoscere la rappresentazione sottostante; magari in più modi (in avanti, all'indietro, saltando elementi, in base ai valori nella struttura) e magari contemporaneamente, in due o più modi. Il pattern è molto comune in Java e .NET.

Un **iteratore** è un oggetto che permette di scorrere tutti gli elementi o le parti contenuti in un altro oggetto, tipicamente un container o una lista; in certi contesti, come i database, è detto **cursor**. Il pattern separa l'implementazione dell'iterazione dalla lista stessa: l'Iterator è un'interfaccia nota, quindi non serve esporre i dettagli della struttura dati sottostante.

I partecipanti sono quattro.

- **Iterator interface**: metodi semplici per attraversare gli elementi della lista.
- **Concrete Iterator**: classe che implementa Iterator.
- **Iteratee interface**: definisce un'interfaccia per creare la lista che verrà iterata.
- **Concrete Iteratee**: crea un iteratore concreto per il proprio tipo di dati.



Nella collaborazione il client chiede al container un iteratore con `getIterator()`: `Iterator` e `Container` sono le interfacce, il `ConcreteContainer` crea il `ConcreteIterator` per i propri dati.

Le conseguenze riguardano cinque questioni di design.

- Chi controlla l'iterazione: iterator interni ed esterni.
- Chi definisce l'algoritmo: può stare nell'iterator o nell'iteratee.
- La robustezza.
- Gli operatori aggiuntivi (es. `Previous`, `SkipTo`).
- Se gli iterator hanno accesso privilegiato ai dati.

I pattern correlati sono tre.

- **Composite**: gli iterator si possono applicare a strutture ricorsive.
- **Factory Method**: per istanziare la sottoclasse concreta di iterator.

- **Memento**: usato con l'iterator, cattura lo stato di un'iterazione.

Dependency Injection

Definition

> **Dependency Injection**
> **Pratica** in cui le risorse da cui un componente dipende sono passate (iniettate) dall'esterno invece che istanziate dal componente stesso durante la propria esecuzione.

Il componente dichiara, ad esempio tramite il costruttore, il tipo (eventualmente generico) e la quantità di tutte le risorse da cui dipende; i riferimenti alle risorse richieste vengono poi passati (iniettati), ad esempio tramite il costruttore.

Java

```java
public class MyClass {
  private Logger logger;
  public MyClass(Logger logger) {
    this.logger = logger;
    logger.info("This is a log message.");
  }
}
```

`MyClass` può scrivere in un log e l'oggetto su cui scrive è quello di tipo `Logger` passato come parametro. Le conseguenze del pattern sono tre.

- Rende la classe più generica, separando il suo comportamento dall'oggetto specifico che le viene passato.
- Introduce una dipendenza tra un oggetto e una classe, che dal punto di vista del coupling può non essere ottimale.
- È detta anche **inversion of control**, perché richiede di creare prima l'oggetto dipendente e poi quello che lo userà, andando in direzione opposta, ad esempio, al modello BCE.

Dependency Injection e testing

La Dependency Injection è fondamentale per il testing di isolamento e di integrazione: permette di eseguire un test su un oggetto ricevendo in input un oggetto fittizio su cui agire, al posto dell'oggetto concreto, che può non essere disponibile o non essere ancora testato.

- **Esempio**: per testare una classe che chiama un servizio non ancora disponibile, se la classe segue il pattern riceve via costruttore un riferimento all'oggetto servizio; in test si crea un servizio fittizio e lo si passa alla classe, che così si può eseguire anche senza il servizio originale.
- **Controesempio**: se la classe non segue il pattern, il riferimento al servizio è hardcoded o istanziato dentro la classe sotto test e non può essere modificato al momento del test.

Riuso, ereditarietà e delega

I due scopi dell'ereditarietà

L'ereditarietà persegue due scopi diversi: la descrizione di tassonomie e il riuso del codice.

- **Identificazione di tassonomie** (usata nell'analisi dei requisiti): si identificano oggetti del dominio applicativo legati gerarchicamente, con l'obiettivo di rendere il modello di analisi più comprensibile; nelle tassonomie l'ereditarietà si trova per specializzazione o per generalizzazione.
- **Specifica dei servizi** (usata nell'object design): con l'obiettivo di aumentare la riusabilità e migliorare modificabilità ed estensibilità.

Eredità di implementazione e di interfaccia

Esistono due forme di ereditarietà, con scopi diversi.

- **Implementation inheritance** (anche detta eredità di classe): si eredita da una classe esistente con alcune o tutte le operazioni già implementate, per estendere le funzionalità dell'applicazione riusando quelle della classe padre.

- **Interface inheritance** (anche detta subtyping): si eredita da una classe astratta con tutte le operazioni specificate ma non ancora implementate.

Il riuso per implementazione inheritance nasce quando una classe molto simile a quella desiderata è già implementata e fa quasi quello che serve: avendo già una classe `List` e dovendo realizzare uno `Stack`, si può sottoclassare `Stack` da `List` fornendo i tre metodi `Push()`, `Pop()` e `Top()`.

Error

Problemi dell'implementazione inheritance

- L'ereditarietà è il massimo accoppiamento possibile tra due classi.
- Con l'ereditarietà la sottoclasse espone anche i metodi della superclasse: alcune operazioni ereditate possono avere comportamenti indesiderati (es. cosa succede se l'utente dello `Stack` chiama `Remove()` invece di `Pop()`?).

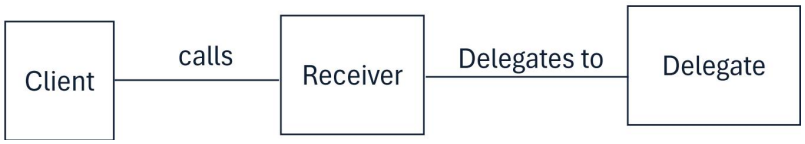
Delega

Definition

Delega

Meccanismo che rende la composizione (ad esempio l'aggregazione) potente per il riuso quanto l'ereditarietà: nella gestione di una richiesta sono coinvolti due oggetti e l'oggetto ricevente delega le operazioni al proprio delegato.

Lo sviluppatore può assicurarsi che l'oggetto ricevente non permetta al client di usare male l'oggetto delegato. La differenza operativa è netta: l'ereditarietà estende una classe base con una nuova operazione o ne sovrascrive una; la delega intercetta un'operazione e la invia a un altro oggetto.



Delega ed ereditarietà di implementazione si confrontano su pro e contro.

	Delega	Eredità di implementazione
Pro	Flessibilità: ogni oggetto è sostituibile a run-time con un altro dello stesso tipo	Semplicità d'uso, supporto di molti linguaggi, facilità di implementare nuove funzionalità
Contro	Inefficienza: gli oggetti sono incapsulati	Espone la sottoclasse ai dettagli della classe padre; ogni cambiamento nell'implementazione del padre forza il cambiamento della sottoclasse (e la ricompilazione di entrambe)

Warning

Euristiche di design

- Evitare l'ereditarietà di implementazione e usare sempre l'eredità di interfaccia.
- Una sottoclasse non deve mai nascondere operazioni implementate in una superclasse.
- Se si è tentati di usare l'ereditarietà di implementazione, usare invece la delega.

Osservazioni e conclusioni

Warning

Cautela nell'uso dei pattern

I pattern sistematizzano soluzioni standard per una classe di problemi: non vanno trattati come soluzioni su misura del proprio contesto specifico, né usati quando esistono soluzioni più semplici; l'abuso produce gerarchie di classi dai nomi mostruosi (es. `BaseEnterpriseVisitorOutputPrintableContextGeneratorSingletonMiddlewareStrategyConfiguratorFactoryReturner`).

Le tre categorie si riassumono per fuoco e problemi risolti.

Categoria	Fuoco	Problemi risolti
Strutturali	Come gli oggetti sono composti in strutture più grandi	Realizzare nuove funzionalità da funzionalità esistenti; fornire flessibilità ed estensibilità
Comportamentali	Algoritmi e assegnazione di responsabilità agli oggetti	Accoppiamento troppo stretto a un algoritmo particolare; gestione del comportamento stato-dipendente
Creazionali	Creazione di oggetti complessi	Nascondere come gli oggetti complessi sono creati e assemblati

Due osservazioni chiudono la lezione.

- I pattern permettono di progettare a un livello più astratto, trattando molte interazioni tra classi e oggetti come un'unità; spesso sono benefici dopo il design iniziale e sono bersagli per il refactoring delle classi.
- Il **variation-oriented design** considera quali aspetti del design sono variabili, identifica i pattern applicabili, varia i pattern per valutare i trade-off e ripete.

La conclusione è metodologica: prima si definisce il problema, poi si cerca la soluzione; poiché i pattern sono soluzioni a problemi, non vanno cercati prima di aver definito il problema.

Capitolo 11

Lezione 19 Refactoring

Definizione e motivazioni

> *"Any fool can write code that a computer can understand. Good programmers write code that humans can understand."*
>
> – Martin Fowler

La comprensibilità del codice da parte degli umani è la motivazione alla base del refactoring, che Fowler definisce in due sensi.

Definition

Refactoring

Modifica apportata alla struttura interna del software per renderlo più facile da comprendere ed economico da modificare, senza cambiarne il comportamento osservabile. **Refactor** (verbo) significa ristrutturare il software applicando una serie di refactoring senza cambiarne il comportamento osservabile.

Rinominare variabili, metodi e classi

Un esempio tipico di refactoring è la ridenominazione di variabili, metodi e classi per chiarezza, che migliora la comprensibilità del codice senza cambiarne il comportamento.

Java

```java
int n = numberOfRentals();
int n2 = numberOfDaysSinceStart();      // Nope
double number = n / n2;

int numberOfRentals = numberOfRentals();
int numberOfDays = numberOfDaysSinceStart();   // Better
double averageRentalsPerDay = numberOfRentals / numberOfDays();
```

Il catalogo di Fowler

Info

Il catalogo di Fowler

Il catalogo è una raccolta di piccole trasformazioni applicabili al codice, ciascuna con motivazione, spiegazione, varianti ed esempi, come *Extract interface*, *Move method*, *Pull up constructor body*. Contiene **22 bad smells**, cioè cose a cui fare attenzione ("anti-pattern"), e **72 refactoring**, cioè cosa fare quando li si trovano. I refactoring spesso esistono in **coppie di duali**, come *Replace inheritance with delegation* e *Replace delegation with inheritance*. Come per il libro dei GoF sui design pattern, le voci del catalogo si sovrappongono e i confini concettuali sfumano.

Code smells

Definition

Code smell

Indicazione che nel sistema esistono problemi di design.

Le tecniche di refactoring servono a correggere i code smells; la maggior parte è abbastanza semplice e oggi gode di buon supporto negli IDE. Nuovi smell e nuovi refactoring vengono "scoperti" di continuo, quindi l'elenco dei nomi è piuttosto lungo. Il ciclo di lavoro tipico parte da codice sporco contenente bad smells e arriva a codice pulito applicando il refactoring, che migliora il codice senza cambiarne il comportamento.

Tipologie di bad smells

I bad smells si raggruppano in cinque categorie secondo la loro natura.

Bloaters

I **bloaters** sono porzioni di codice, metodi e classi cresciute fino a proporzioni gigantesche, tali da essere difficili da gestire; di solito non compaiono subito ma si accumulano durante l'evoluzione del programma, soprattutto se nessuno si sforza di eradicarli. Esempi: *Long Method*, *Large Class*, *Primitive Obsession*, *Long Parameter List*, *Data Clumps*.

Object-Orientation Abusers

Gli **object-orientation abusers** sono tutti smell derivanti dall'applicazione incompleta o errata dei principi della programmazione orientata agli oggetti. Esempi: *Alternative Classes with Different Interfaces*, *Refused Bequest*, *Switch Statements*, *Temporary Field*.

Change Preventers

I **change preventers** sono smell per cui cambiare qualcosa in un punto del codice richiede molte modifiche anche in altri punti, rendendo lo sviluppo molto più complicato e costoso. Esempi: *Divergent Change*, *Parallel Inheritance Hierarchies*, *Shotgun Surgery*.

Dispensables

Un **dispensable** è qualcosa di inutile e superfluo la cui assenza renderebbe il codice più pulito, più efficiente e più facile da capire. Esempi: *Comments*, *Duplicate Code*, *Data Class*, *Dead Code*, *Lazy Class*, *Speculative Generality*.

Couplers

I **couplers** sono smell che contribuiscono a un accoppiamento eccessivo tra classi o che mostrano cosa accade quando l'accoppiamento è sostituito da una delega eccessiva. Esempi: *Feature Envy*, *Incomplete Library Class*, *Message Chains*, *Middle Man*.

Debito tecnico

Definizione

Definition

Debito tecnico
Condizione in cui il codice soddisfa i requisiti funzionali ma è subottimale o realizzato in fretta ("quick and dirty").

Il debito tecnico si manifesta ad esempio come codice "sporcato" dai bad smells, algoritmi inefficienti o design trascurato. Può essere corretto in code review oppure generare TODO o nuove issue nel issue tracker; capirlo, misurarlo e comunicarlo è critico nell'industria del software.

Analogia del debito

Il debito non è sempre negativo, perché nella vita reale conta consegnare codice funzionante.

> *"Shipping first time code is like going into debt. A little debt speeds development so long as it is paid back promptly with a rewrite – the danger occurs when the debt is not repaid. Every minute spent on not-quite-right code counts as interest on that debt."*
>
> – Ward Cunningham

Le domande decisive da porsi sono tre:

- Ci si può permettere di lavorare senza debito?
- Ci si può permettere di pagarne gli interessi?
- Si riesce a tenere debito e interessi gestibili?

Debito strategico e intenzionale

La definizione originale di Cunningham considerava solo il debito intenzionale: il debito può essere "pianificato" oppure "capitato", e in entrambi i casi può essere strategico, secondo l'idea di consegnare subito e gestire le conseguenze dopo. Esempi di debito intenzionale o strategico: design non modulare, implementazione volutamente troppo semplice o troppo complessa, indifferenza alle prestazioni, mancanza di generalità o estensibilità, mancanza di scalabilità.

Debito non strategico e non intenzionale

Il debito si accumula anche involontariamente, perché nessun processo di sviluppo è perfetto. Esempi di debito non intenzionale o non strategico: bad smells, memory leak, copertura di test insufficiente, implementazione involontariamente complessa, architettura rigida o fragile, colli di bottiglia nelle prestazioni o nella scalabilità, codice trascurato o difficile da mantenere.

Conseguenze del debito

Un debito tecnico eccessivo produce effetti a cascata sullo sviluppo.

- Un debito eccessivo significa troppo tempo speso a pagare gli interessi.
- Rende imprevedibile la fase di pianificazione del software e aumenta il rischio di investimento.
- Rallenta il lavoro futuro.
- Genera più bug, più costosi da correggere.
- Produce sviluppatori frustrati e infelici.

Misurazione

Strumenti di qualità come **SonarQube** possono fornire una stima accurata del debito tecnico accumulato in un repository software.

Il processo di refactoring

Metafora di base

Il refactoring parte da una base di codice esistente e la migliora: cambia la struttura interna (in-the-small fino a in-the-medium) preservando la semantica complessiva, cioè riarrangia i "fattori" mantenendo invariato il "prodotto" finale. L'idea è migliorare il codice in qualche modo significativo, ad esempio:

- Ridurre il codice quasi duplicato.
- Migliorare coesione e accoppiamento.
- Migliorare parametrizzazione, comprensibilità, manutenibilità, flessibilità, astrazione, efficienza.

Il ciclo di refactoring

Il pattern base per refactorare un programma funzionante prevede tre passi:

1. Scegliere il bad smell peggiore.
2. Selezionare un refactoring che lo affronti.
3. Applicare il refactoring.

A ogni giro del ciclo si selezionano refactoring che migliorano il codice; il comportamento del programma non cambia, quindi il programma resta in uno stato funzionante: il ciclo migliora il codice mantenendo il comportamento. La parte più insidiosa del processo è identificare lo smell: all'inizio conviene partire dalle cose facili, come spezzare routine lunghe o rinominare per chiarezza, così i problemi restanti si vedono e si correggono più facilmente.

Refactoring e unit test

Il refactoring dipende fortemente da una buona suite di **unit test**: si refactora, poi si eseguono i test automatici per verificare che il comportamento sia davvero preservato. Senza buoni unit test gli sviluppatori tendono a evitare il refactoring, per paura di rompere qualcosa.

Perché refactorare

Nella realtà è estremamente difficile ottenere il design "giusto" la prima volta, per vari motivi:

- È difficile capire a fondo il dominio del problema.
- È difficile capire i requisiti degli utenti, anche quando sono chiari per l'utente stesso.
- È difficile prevedere come il sistema evolverà tra X anni.
- Il design originale spesso si rivela inadeguato.
- Il sistema col tempo diventa fragile e più difficile da cambiare.

Il refactoring aiuta a manipolare il codice in un ambiente sicuro che preserva il comportamento, a ricreare una situazione in cui l'evoluzione è possibile e a comprendere il codice esistente. I suoi benefici principali sono:

- **Migliora il design** del software: senza refactoring il design decade, e il codice mal progettato richiede più righe per fare le stesse cose, spesso perché fa la stessa cosa in punti diversi.
- **Rende il software più facile da capire**, perché prima o poi qualcun altro dovrà leggere il codice.
- **Aiuta a trovare i bug**.
- **Aiuta a programmare più in fretta**.

Refactoring tipici

I refactoring tipici si dividono in tre categorie secondo l'elemento coinvolto.

Class Refactorings	Method Refactorings	Attribute Refactorings
add (sub)class to hierarchy	add method to class	add variable to class
rename class	rename method	rename variable
remove class	remove method	remove variable
Extract class	push method down	push variable down
	push method up	pull variable up
	add parameter to method	create accessors
	move method	abstract variable
	extract code in new method	
	replace parameter with method	

Extract Method

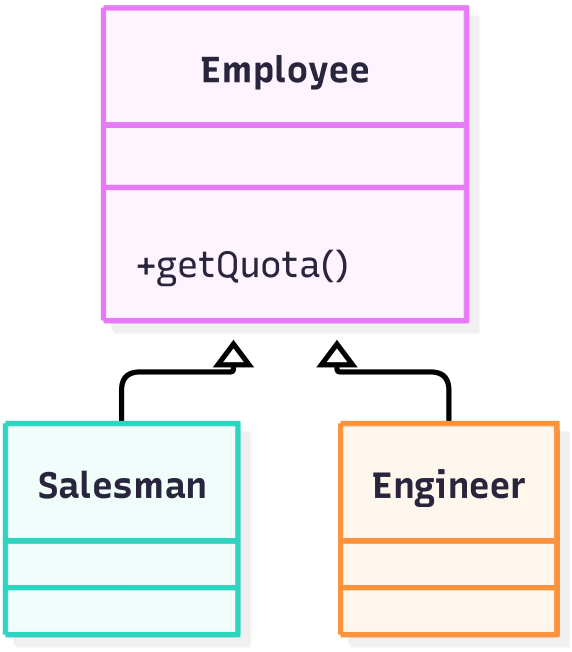
Extract Method estrae un frammento di codice in un nuovo metodo, invocato al posto del codice estratto: le stampe dei dettagli, prima inline in `printOwing`, diventano il metodo `printDetails`.

```java
void printOwing() {
    printBanner();
    printDetails(getOutstanding());
}

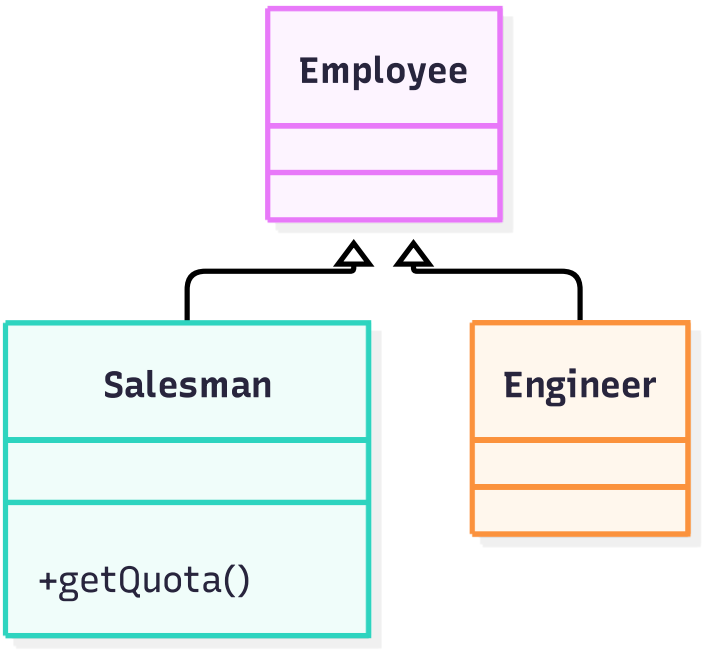
void printDetails(double outstanding) {
    System.out.println("amount " + outstanding);
}
```

Push method down

Si applica quando il comportamento di una superclasse è rilevante solo per alcune delle sue sottoclassi: il metodo va spostato in quelle sottoclassi. Nel diagramma `getQuota()` è inizialmente definito in `Employee` anche se interessa solo `Salesman`:

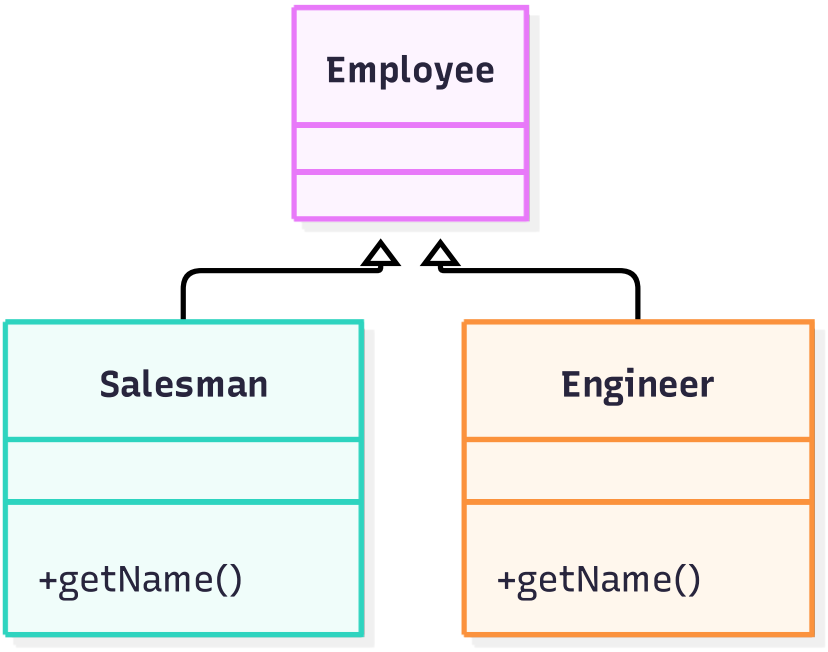


Dopo il refactoring il metodo vive solo nella sottoclasse che lo usa:

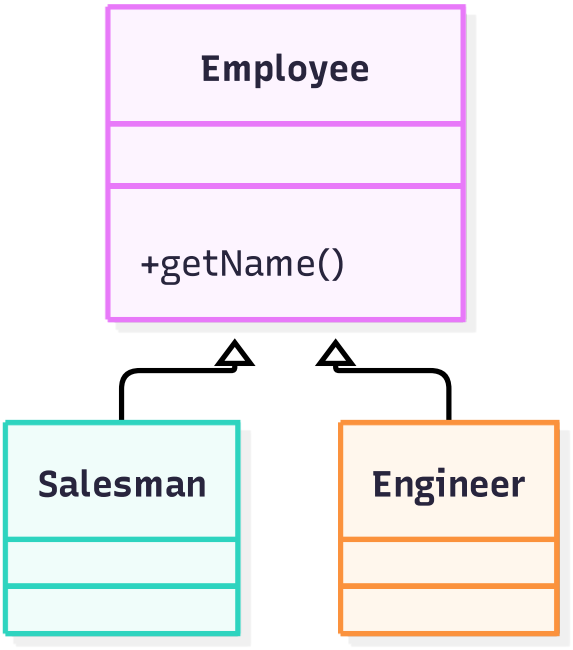


Push method up

Si applica quando più sottoclassi hanno metodi con risultati identici: i metodi vanno spostati nella superclasse. Inizialmente `getName()` è duplicato in entrambe le sottoclassi:



Dopo il refactoring il metodo è definito una sola volta nella superclasse:



Add parameter to method

Si applica quando un metodo necessita di più informazioni dal chiamante: si aggiunge un parametro per un oggetto che può passare queste informazioni, ad esempio `getContact()` diventa `getContact(Date)`.

Replace Parameter with Method

Si applica quando un oggetto invoca un metodo e ne passa il risultato come parametro a un altro metodo, mentre anche il ricevente potrebbe invocare il metodo direttamente: il parametro `discountLevel` sparisce perché il ricevente può calcolarselo da sé.

```java
int basePrice = _quantity * _itemPrice;
double finalPrice = discountedPrice(basePrice);
```

Bad smells e rimedi

Per ciascun bad smell del catalogo esistono refactoring raccomandati che lo eliminano.

Duplicated Code

Se la stessa struttura di codice compare in più punti, il programma migliorerà trovando un modo di unificarla. I casi principali sono tre:

- La stessa espressione in due metodi della stessa classe si risolve applicando *Extract Method* e invocando il codice da entrambi i punti.
- La stessa espressione in due sottoclassi fratelle si risolve con *Extract Method* in entrambe le classi e poi *Pull Up Field*.
- Il codice duplicato in due classi non correlate si affronta con *Extract Class* in una classe e usando il nuovo componente nell'altra.

Long Method

Quanto più una procedura è lunga, tanto più è difficile da capire; i programmi a oggetti vivono al meglio e più a lungo con metodi corti. Quasi sempre basta *Extract Method* per accorciare un metodo; se l'estrazione richiede molti parametri, si può creare una nuova classe che li incapsuli.

Large Class

Quando una classe cerca di fare troppo, lo mostra con troppe variabili d'istanza, e dove ci sono troppe variabili d'istanza il codice duplicato non è lontano; anche una classe con troppo codice è un terreno fertile per la duplicazione. In entrambi i casi funzionano *Extract Class* ed *Extract Subclass*.

Long Parameter List

Alla vecchia scuola si passava tutto come parametro, cosa comunque migliore dei dati globali; con gli oggetti non serve passare tutto ciò di cui il metodo ha bisogno, ma abbastanza perché possa arrivare a tutto. Le liste di parametri lunghe sono difficili da capire, incoerenti, difficili da usare e in continua evoluzione man mano che servono nuovi dati. Si risolve così:

- *Replace Parameter with Method*, quando il dato può essere ottenuto con una richiesta a un oggetto già noto.
- *Introduce Parameter Object*, quando un gruppo di parametri va naturalmente insieme e si trasforma in un oggetto.

```java
public void marry(String name, int age, boolean gender,
    String name2, int age2, boolean gender2) {...}
// diventa
public void marry(Person person1, Person person2) {...}
```

Divergent Change

La **divergent change** si verifica quando una classe è comunemente modificata in modi diversi per ragioni diverse: quasi certamente si violano i principi di astrazione chiave e di separazione delle responsabilità. Si sistema identificando tutto ciò che cambia per una particolare causa e applicando *Extract Class* per raccoglierlo in un unico posto.

Shotgun Surgery

La **shotgun surgery** si verifica quando ogni modifica di un certo tipo richiede molte piccole modifiche in molte classi diverse: la stessa frequenza di cambiamento in oggetti diversi, in particolare scollegati, è tipicamente il risultato della

copy-paste programming. Con i cambiamenti sparsi è facile dimenticarne uno importante. Si risolve con *Move Method* e *Move Field* per concentrare tutte le modifiche in una sola classe; se nessuna classe esistente è un buon candidato, la si crea.

Feature Envy

Definition

> **Feature envy**
>
> **Smell** in cui un metodo sembra più interessato a una classe diversa da quella in cui si trova, tipicamente per i suoi dati: fa un uso massiccio di dati e metodi dell'altra classe.

Si risolve con *Move Method* per spostare il metodo nella classe che lo interessa; se solo una parte del metodo fa un uso massiccio delle caratteristiche dell'altra classe, si applica *Extract Method* per estrarre le parti che vi appartengono.

Data Clumps

I **data clumps** sono gli stessi tre o quattro dati che compaiono insieme in molti posti, come campi in un paio di classi o parametri in molte firme di metodi: gruppi di dati che stanno davvero insieme dovrebbero diventare un oggetto a sé, come `x`, `y` che diventano `Point`. Per i campi si usa *Extract Class* per trasformare il gruppo in un oggetto; per i parametri si usano *Introduce Parameter Object* o *Preserve Whole Object* per snellirli.

Primitive Obsession

La **primitive obsession** è la riluttanza a introdurre piccole classi per cose facilmente rappresentate con primitivi: numeri di telefono, codici di avviamento, somme di denaro, intervalli con limiti superiore e inferiore. Se la primitiva necessita di dati o comportamenti aggiuntivi, ad esempio una formattazione speciale come `(215)898-0587` o `19104-6389`, va trasformata in una classe.

Switch Statements

Nella maggior parte dei casi in cui si vede uno `switch` sul tipo si dovrebbe considerare il polimorfismo:

> *L'essenza del polimorfismo è che invece di chiedere a un oggetto di che tipo è e poi invocare un comportamento in base alla risposta, si invoca direttamente il comportamento: l'oggetto, a seconda del suo tipo, fa la cosa giusta.*

Si applica *Extract Method* per estrarre lo `switch` e poi *Move Method* per portarlo nella classe dove serve il polimorfismo.

Parallel Inheritance Hierarchies

Le **parallel inheritance hierarchies** sono in realtà un caso particolare di shotgun surgery: ogni volta che si crea una sottoclasse di una classe, occorre creare una sottoclasse di un'altra. Secondo Cunningham si può rimuovere il parallelo refactorando una o entrambe le gerarchie fino a rendere i membri congruenti e poi collassandole a coppie, oppure si può rimuovere la duplicazione tra i paralleli definendo responsabilità distinte raffinate da ciascuna gerarchia e ricollocando i metodi di conseguenza. Con *Move Method* e *Move Field* la gerarchia della classe che si riferisce all'altra scompare.

Lazy Class

Ogni classe creata costa tempo e denaro da mantenere e capire: una classe che non si guadagna il suo mantenimento va eliminata. Per sottoclassi che fanno troppo poco si usa *Collapse Hierarchy*; per componenti quasi inutili si usa *Inline Class*.

Speculative Generality

La **speculative generality** si presenta quando qualcuno dice "penso che un giorno ci servirà la capacità di fare questo" e servono hook e casi speciali per gestire cose non richieste; si rileva facilmente quando gli unici utilizzatori di una classe o metodo sono i test case. Si combatte così:

- *Collapse Hierarchy* per le classi astratte che non fanno abbastanza.
- *Inline Class* per la delega non necessaria.
- *Remove Parameter* per i metodi con parametri inutilizzati.
- *Rename Method* per i metodi con nomi astratti e incomprensibili.

Temporary Field

Il **temporary field** è un oggetto in cui una variabile d'istanza è impostata solo in certe circostanze: il codice diventa difficile da capire, perché di solito ci si aspetta che un oggetto usi tutte le sue variabili. Si risolve con *Extract Class*, che crea una casa per queste variabili orfane spostandovi tutto il codice che le usa.

Message Chains

Le **message chains** si verificano quando un client chiede a un oggetto un altro oggetto, che a sua volta chiede un altro oggetto e così via, come in `intermediate.getProvider().doSomething()`: navigando in questo modo il client assume la struttura della gerarchia.

```C++
color = vehicle->getChassis()->getBody()->getShell()
    ->getMaterial()->getProperties()->getColor();
```

Middle Man

Una delle caratteristiche principali degli oggetti è l'incapsulazione, che spesso comporta la delega; a volte però la delega va troppo in là, come quando metà dei metodi di una classe sono semplici deleghe a un'altra classe. I rimedi possibili sono tre:

- *Remove Middle Man*, per parlare direttamente all'oggetto che sa davvero cosa sta succedendo: il client, che prima invocava `Person.getManager()` come semplice delega a `Department`, ottiene il `Department` da `Person` e invoca direttamente `getManager()`.
- *Inline Method*, se solo pochi metodi fanno poco, per inlinearli nel chiamante.
- *Replace Delegation with Inheritance*, se c'è un comportamento aggiuntivo, per trasformare il middle man in una sottoclasse dell'oggetto reale.

Osservazioni finali

Refactoring vs restructuring

Info

Refactoring vs restructuring

Il termine "refactoring" è spesso usato a sproposito: se un sistema resta rotto per un paio di giorni mentre si "refattorizza", o se si parla di refactoring di un documento, si sta facendo restructuring. **Lo restructuring** è qualsiasi riarrangiamento di parti di un tutto: un termine molto generale che non implica alcun modo particolare di procedere. **Il refactoring** è invece una tecnica molto specifica, fondata su piccole trasformazioni che preservano il comportamento (esse stesse chiamate refactoring). Facendo refactoring, il sistema non dovrebbe restare rotto per più di pochi minuti alla volta, e non si vede come applicarlo a qualcosa che non ha un comportamento ben definito.

Parte IV

Progettazione Orientata agli Oggetti e Clean Code

Parte V

Verification, Validation & Testing

Parte VI

Processi di Sviluppo e Interfacce